

Hichip Linux SDK

本文档主要介绍hclinux sdk的基本知识，目的让开发者熟悉系统的架构、开发和调试流程。

1. hclinux sdk资料介绍

1.1 文档资料

1.1.1 gitlab资料

gitlab网址: <https://gitlab.hichiptech.com:62443/>

资料说明:

类别	仓库	重点说明	说明
sw	hclinux_sdk		hclinux sdk代码
	hcrtos_sdk		hcrtos sdk代码
	DI		buildroot编译需要的第三方软件包
		hcrtos编译工具	Codescape.GNU.Tools.Package.2019.09-03-2.for.MIPS32.MTI.Bare.Metal.Ubuntu-18.04.5.x86_64.tar.gz
	Tools		存放PC工具，主要存放固件烧写工具，以及bootlogo.m2v制作工具
		GDB	固件烧写工具，ejtag方式烧写
		ffmpeg-4.4-essentials_build	bootlogo.m2v制作工具
hw16xx	Datasheet		存放IC的datasheet
	Reference Design		存放硬件参考设计

S

sw

Group ID: 13

Leave group

Subgroups and projects

Shared projects

Archived projects

> H hclinux_sdk

> H hcrtos_sdk

D DI

T Tools

H

hw16xx

Group ID: 55

Leave group

Subgroups and projects

Shared projects

Archived projects

D

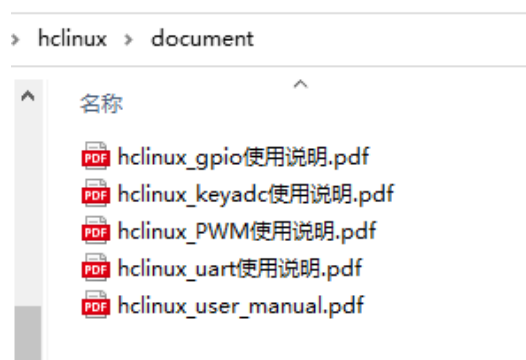
Datasheet

R

Reference Design

1.1.2 sdk使用说明文档

使用说明文档在SDK中，位于：hclinux\document



1.2 SDK下载方法

1.2.1 sdk下载

下载hclinux仓库

```
1 git clone https://gitlab.hichiptech.com:62443/sw/hclinux_sdk/hclinux.git
2 cd hclinux
```

切换分支

```
1 #git branch -av //查看分支
2
3 main 6d29e28 Initial commit
4 remotes/origin/HEAD -> origin/main
5 remotes/origin/hclinux-2022.03.y f807c0e Fix mjpeg bootlogo compile issue
6 remotes/origin/hclinux-2022.05.y 6f17d92 2022.05.y (Update submodule commits)
7 remotes/origin/hclinux-2022.06.y 21d1e39 install S01wifiprobe
8 remotes/origin/hclinux-2022.08.y 858d3f2 tag-hclinux-2022.08.y.1 (Update submodule
commits)
9 remotes/origin/hclinux-2022.10.y 73cb181 disable hdmirx by default in case user has no
hdmirx plugin
10 remotes/origin/main 6d29e28 Initial commit
11
12 #git checkout hclinux-2022.10.y //切换到想要的分支
```

下载子仓库的代码

```
1 | git pull
2 | git submodule update --init --recursive
```

1.2.2 dl仓库下载

dl仓库文件应该放到hclinux/buildroot目录下

```
1 | cd buildroot
2 | rm -rf dl //删除buildroot目录原有的dl文件夹
3 | git clone https://gitlab.hichiptech.com:62443/sw/dl.git //下载dl仓库
```

buildroot编译时会去国外网站下载的第三方软件包到dl目录，**如果没有网络或者网速不好**，可以事先下载dl仓库，这样编译时就不会去下载了。

1.2.2 硬件资料下载

```
1 | git clone https://gitlab.hichiptech.com:62443/hw16xx/reference-design.git
2 | cd reference-design
```

IC datasheet和原理图都放在这个仓库。

1.2.3 PC工具下载

```
1 | git clone https://gitlab.hichiptech.com:62443/sw/tools.git
2 | cd tools
```

PC工具和教学视频都放在这个仓库，包括固件烧写工具。

2. 开发环境设置

2.1 主机配置说明

hclinux对主机的要求如下：

操作系统：ubuntu **英文版**

系统版本：Ubuntu 18.04.5 LTS 或更高版本

硬盘：大于100G（hclinux一个方案编译后大约20G左右）

内存：大于4G（至少2G，否则会编译非常慢）

hclinux基于Ubuntu 18.04.5 LTS建立主机端开发环境。桌面版和服务器版Ubuntu均可作为开发环境，由于桌面版自带的工具较为丰富，建议使用**桌面版Ubuntu**。如果采用虚拟机，考虑到电脑性能，建议使用**服务器版**。

注意：

- 1.必须使用**ubuntu**下载**SDK**吗，不要使用**windows**，原因是**windows**会自动修改**sdk**中某些文件的文件名造成编译报错。
- 2.**ubuntu**必须是英文版，原因是某些编译工具使用系统调用获得返回值是中文，造成编译报错。

2.2 ubuntu环境搭建

2.2.1 设置Ubuntu的系统语言为英文

如果系统语言不是英文，可以通过下面方式修改

```
1 | 1. 编辑文件
2 | vi /etc/default/locale
3 |
4 | 2. 修改成英文配置：
5 | LANG="en_US.UTF-8"
6 | LANGUAGE="en_US:en"
7 |
8 | 3. 然后重启Ubuntu，即可恢复为英文的语言环境。
```

2.2.2 安装编译工具

安装编译工具脚本位于：hclinux\support\scripts\setup\setup_tools.sh

安装编译工具：

```
1 | source support/scripts/setup/setup_tools.sh
```

```
javen@ubuntu:~/linux/hclinux$ source support/scripts/setup/setup_tools.sh
Get:1 http://mirrors.huaweicloud.com/repository/ubuntu bionic InRelease [242 kB]
Get:2 http://mirrors.huaweicloud.com/repository/ubuntu bionic-updates InRelease [88.7 kB]
Get:3 http://mirrors.huaweicloud.com/repository/ubuntu bionic-backports InRelease [83.3 kB]
Get:4 http://mirrors.huaweicloud.com/repository/ubuntu bionic-security InRelease [88.7 kB]
Get:5 http://mirrors.huaweicloud.com/repository/ubuntu bionic/main amd64 Packages [1,019 kB]
Get:6 http://mirrors.huaweicloud.com/repository/ubuntu bionic/main i386 Packages [1,007 kB]
Get:7 http://mirrors.huaweicloud.com/repository/ubuntu bionic/main Translation-en [516 kB]
Get:8 http://mirrors.huaweicloud.com/repository/ubuntu bionic/restricted i386 Packages [9,156 B]
Get:9 http://mirrors.huaweicloud.com/repository/ubuntu bionic/restricted amd64 Packages [9,184 B]
Get:10 http://mirrors.huaweicloud.com/repository/ubuntu bionic/restricted Translation-en [3,584 B]
Get:11 http://mirrors.huaweicloud.com/repository/ubuntu bionic/universe i386 Packages [8,531 kB]
Get:12 http://mirrors.huaweicloud.com/repository/ubuntu bionic/universe amd64 Packages [8,570 kB]
Get:13 http://mirrors.huaweicloud.com/repository/ubuntu bionic/universe Translation-en [4,941 kB]
Get:14 http://mirrors.huaweicloud.com/repository/ubuntu bionic/multiverse amd64 Packages [151 kB]
Get:15 http://mirrors.huaweicloud.com/repository/ubuntu bionic/multiverse i386 Packages [144 kB]
Get:16 http://mirrors.huaweicloud.com/repository/ubuntu bionic/multiverse Translation-en [108 kB]
Get:17 http://mirrors.huaweicloud.com/repository/ubuntu bionic-updates/main i386 Packages [1,598 kB]
Get:18 http://mirrors.huaweicloud.com/repository/ubuntu bionic-updates/main amd64 Packages [2,875 kB]
Get:19 http://mirrors.huaweicloud.com/repository/ubuntu bionic-updates/main Translation-en [525 kB]
70% [19 Translation-en 15.6 kB/525 kB 3%]
```



```
1 | cd buildroot/d1
2 | sudo tar -xzf Codescape.GNU.Tools.Package.2019.09-03-2.for.MIPS32.MTI.Bare.Metal.Ubuntu-18.04.5.x86_64.tar.gz -C /opt
```

查看是否安装成功

```
javen@ubuntu:~/linux/hclinux$ ls -l /opt/mips32-mti-elf/2019.09-03-2/
total 24
drwxrwxr-x 2 1001 1001 4096 Sep 28 18:20 bin
drwxrwxr-x 2 1001 1001 4096 Sep 28 18:04 include
drwxrwxr-x 3 1001 1001 4096 Sep 28 18:20 lib
drwxrwxr-x 3 1001 1001 4096 Sep 28 18:04 libexec
drwxrwxr-x 5 1001 1001 4096 Sep 28 18:10 mips-mti-elf
drwxrwxr-x 6 1001 1001 4096 Sep 28 18:20 share
```

3. 设置第三方软件的本地下载路径

hclinux SDK在编译过程中会下载一些第三方软件包，一般会从公开网络进行下载。有时候由于网络问题导致下载很慢或者无法下载，则可以用其他的方式提前下载好第三方软件包。然后能过hclinux的配置选择从已经下载的路径进行三方软件的下载。

dl仓库的下载方法参考：《1.2.2 DI仓库下载》

4. sdk介绍

4.1 目录结构

一级目录	二级目录	n级目录	说明
board			方案板级配置目录，存放板级配置
	hichip/hc16xx/common		通用配置，hichip公版配置，建议用户新建自己的方案目录，尽量不要去修改common中的配置
		busybox-configs	busybox配置文件
		ddrinit	linux系统用到ddr配置文件
		ddrinit/gdb	HiChipGDB PC工具用到的ddr配置文件
		dts	板级配置文件
		kernel-configs	kernel配置文件
		rootfs_overlay	外部rootfs配置，在打包固件时会覆盖原有rootfs
		logo.m2v	开机图片/动画
		post-build.sh	固件打包脚本
	hichip/hc16xx/c3100_emmc		方案板级配置，针对某一个方案，参考《4.2 方案介绍》
boot			boot相关配置
	hcboot		hcboot编译配置
buildroot			buildroot构建系统

一级目录	二级目录	n级目录	说明
	dl		存放编译时下载的开源包
	output		编译输出目录
		build	存放编译时的软件包源码
		host	包含为主机构建的工具和 目标工具链
		images	存放所有编译生成的映像文件
		staging	存放编译产生的临时文件和链接
		target	编译生成的rootfs目标目录
configs			存放方案配置
document			存放使用说明文档
linux			外部linux配置目录
package			<i>SOURCE</i> 目录下软件包编译配置
patches			buildroot原生软件包补丁 包括linux kernel补丁
SOURCE			hichip开发的软件包
	applications		
		hccastapp	7宫格UI的demo app
		hcscreenapp	无线投屏main app
	avp		hcrtos源代码，也是hcboot源码
		components /applications /apps-bootloader	<i>hcbootbootloader</i> 源码
		configs	<i>hcrtos</i> 方案配置
		ffmpeg	ffmpeg源码
		getevent	获取 input 输入事件
		libcast	dlina\miracast\aircast库
		hccast	dlina\miracast\aircast中间件
		hc-examples	存放demo程序，多媒体/ hdmi rx/wifi test等
		linux-drivers	海奇开发的 <i>linux</i> 驱动
		rtl8188fu	wifi驱动源码
		wpa_supplicant	wpa_supplicant源码
support			存放编译相关的脚本
toolchain			存放编译工具链
Config.in			SOURCE目录下软件包 链接进buildroot配置
local.mk			SOURCE目录下软件包 链接进buildroot编译

4.2 方案介绍

方案	板级配置	方案配置
无线投屏	common\dts	hichip_hc16xx_db_a3100_v10_hcscreen_defconfig hichip_hc16xx_db_a3300_v10_hcscreen_defconfig hichip_hc16xx_db_a5100_v10_128M_hcscreen_defconfig hichip_hc16xx_db_a5100_v10_hcscreen_defconfig
投影仪	projector\c2\dts	hichip_hc16xx_cb_d3100_v10_projector_cast_c2_defconfig hichip_hc16xx_db_d3000_v10_projector_cast_defconfig hichip_hc16xx_db_d3100_v10_projector_cast_defconfig
hdmi rx无线投屏 发送端	a3300_hwd_source	hichip_hc16xx_db_a3300_v10_hdmi_wifi_source_defconfig
hdmi rx无线投屏 收端端	common\dts	hichip_hc16xx_db_a3300_v10_hdmi_wifi_sink_defconfig
nor+emmc方案	c3100_emmc	hichip_hc16xx_db_c3100_v20_emmc_defconfig
带文件系统的数据分区	d3100_p1	hichip_hc16xx_db_d3100_v20_p1_defconfig

4.2 SDK编译

参考《16.1 SDK 配置、编译和打包》

5. 固件烧录

5.1 烧录-with-GDB

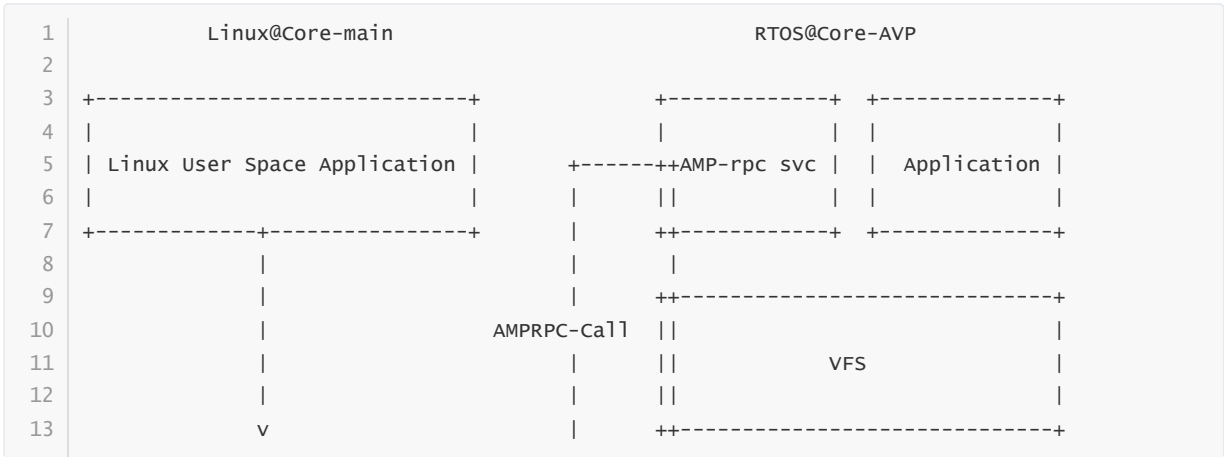
gdb烧录方法参考《LINUX_HiChipGDB使用手册.pdf》

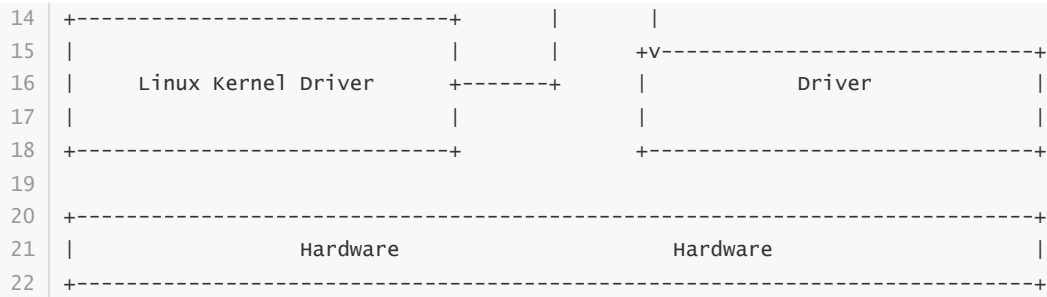
5.2 烧录-in-USB

请参考教学视频

6. 有关设计

6.1 海奇linux sdk软件框架：





- 双CPU架构设计，主CPU运行Linux OS，而辅CPU海奇称之为AVP（Audio/Video协处理）运行RTOS。
- AVP主要处理Video/Audio/HDMI-Tx/HDMI-Rx/Display等与音视频相关的驱动程序处理。而其他功能均由主CPU处理。
- 双核之间通过异构AMPRPC远程调用进行通信，RPC通信的API参考posix API设计。AVP是海奇开发的基于FreeRTOS的RTOS系统，内部有完整的驱动程序框架，支持posix标准的VFS接口。在AVP系统内同样会有字符设备，块设备等驱动程序框架，对驱动访问基于posix open/read/write等接口进行。对于异构的双CPU软件架构，如果Linux操作系统的内核驱动想要访问AVP的驱动接口，可以通过如下API进行访问。

- avp_open()
- avp_close()
- avp_read()
- avp_write()
- avp_ioctl()
- avp_poll()
 - 尽量避免调用avp_poll()，如果调用此API的timeout为非零，将会阻塞住一路RPC后台服务线程（默认main-core/avp-core均有8路RPC后台服务线程）
- avp_work_notifier_setup_proxy()
 - 有些情况下，需要监听AVP RTOS驱动程序的一些事件，比如音频、视频播放结束的通知，hdm插入、拔出通知等信息时，可以通过这个API监听通知事件，当事件发生时，会调用回调函数。
 - 如果不主动调用avp_work_notifier_teardown()则事件一旦发生，会重复调用回调函数。
- avp_work_notifier_onehot_setup_proxy()
 - 类似于avp_work_notifier_setup_proxy()，不同的是，当事件发生一次后，会主动取消该监听事件。
- avp_work_notifier_teardown()
 - 取消尚存的监听事件

7. Buildroot

海奇Linux SDK基于Buildroot构建。关于Buildroot的资料，可查阅Buildroot官网获取更多信息(<https://buildroot.org>)

Buildroot自带的文档信息可以通过如下命令生成：

```

1 $ cd buildroot
2 $ make manual
3 $ tree output/docs/manual/ -L 1
4 buildroot/output/docs/manual/
5 |— docbook-xsl.css
6 |— github_hash_mongrel2.png
7 |— logo.png
8 |— manual.chunked
9 |— manual.epub
10 |— manual.html
11 |— manual.pdf
12 |— manual.text

```

该命令依赖主机端安装如下工具：

```
1 $ sudo apt install asciidoc (version 8.6.3 or higher)
2 $ sudo apt install w3m
3 $ sudo apt install python
4 $ sudo apt install dblatex
```

Buildroot作为海奇Linux SDK的子软件包，帮助管理依赖关系，编译构建rootfs等。Buildroot内部管理有许多可用于嵌入式开发的第三方软件包，如有需要，可以通过menuconfig配置并使用这些三方软件包。

由于Buildroot会从开源网站上下载所需要的开源第三方软件包，下载的软件包会放在buildroot/dl目录下。为了加快开发速度，一旦下载完成后，可以将下载好的文件（buildroot/dl）制作一个本地的http文件服务，并配置Buildroot首先从本地的镜像服务器获取软件包，当在一个全新的SDK工作空间编译时，可以节省下载时间。比如，

安装与配置apache2

```
1 $ sudo apt install apache2
2 $ sudo ln -s /path/to/mirror/dl /var/www/html/dl
3 $ sudo /etc/init.d/apache2 restart
4 $ rsync -aux buildroot/dl/* /var/www/html/dl/
```

设置Buildroot配置

```
1 $ cd buildroot
2 $ make menuconfig
3 > Build options > Mirrors and Download locations
4 (http://<ip-addr-of-local-apache2-server>/dl) Primary download site
5 [ ] Only allow downloads from primary download site
6 (http://sources.buildroot.net) Backup download site
7 (https://cdn.kernel.org/pub) Kernel.org mirror
8 (http://ftpmirror.gnu.org) GNU Software mirror
9 (http://rocks.moonscript.org) LuaRocks mirror
10 (http://cpan.metacpan.org) CPAN mirror (Perl packages)
```

AVP的编译框架是参考buildroot设计的，它同样也会在编译各软件包的过程中下载一些第三方开源软件。需要修改AVP的配置文件从本地镜像中下载第三方软件以节省下载时间。

AVP的配置文件位于

```
1 $ cat hclinux/SOURCE/avp/configs/hichip_hc16xx_db_a3300_linux_defconfig
2 BR2_PRIMARY_SITE="http://hichip01/dl/"
```

如果已经编译过AVP软件包，则可以通过AVP的menuconfig来修改AVP配置

```
1 $ cd buildroot
2 $ make avp-menuconfig
3 > Mirrors and Download locations
4 (http://<ip-addr-of-local-apache2-server>/dl/) Primary download site
5 [ ] Only allow downloads from primary download site
6 (http://sources.buildroot.net) Backup download site
7 (https://cdn.kernel.org/pub) Kernel.org mirror
8 (http://ftpmirror.gnu.org) GNU Software mirror
9 (http://rocks.moonscript.org) LuaRocks mirror
10 (http://cpan.metacpan.org) CPAN mirror (Perl packages)
11 () global patch directories
```

如果要修改AVP的默认配置文件，可以将buidroot/output/build/avp-custom/output/.config内容更新到hclinux/SOURCE/avp/configs/hichip_hc16xx_db_a3300_linux_defconfig

8. linux sdk配置设计

8.1 总配置与支持的Demo Board

一般来说，由于Buildroot本身的设计，它的配置文件都放在Buildroot/configs目录。为了便于未来升级buildroot，海奇Linux SDK将海奇开发的软件包和配置文件独立于buildroot放置。buildroot支持通过BR2_EXTERNAL变量导入外部的配置文件与软件包。

顶层总配置在hclinux/configs目录

```
1 $ cd hclinux
2 $ ls -l configs
3 hichip_hc16xx_db_a3000_v10_defconfig
4 hichip_hc16xx_db_a3100_v10_defconfig
5 hichip_hc16xx_db_a3100_v20_defconfig
6 hichip_hc16xx_db_a3200_v10_defconfig
7 hichip_hc16xx_db_a3300_v10_defconfig
8 hichip_hc16xx_db_b3100_v20_defconfig
9 hichip_hc16xx_db_c3100_v20_defconfig
10 hichip_hc16xx_db_d3100_v20_defconfig
```

根据海奇的Demo Board分别为每块支持的板子有一个配置文件。

8.2 总配置内会指定其它有关不同的配置

buildroot所管理编译的软件包有些是需要有配置文件的，比如，编译uboot需要有一个uboot的配置文件，编译linux kernel需要有一个linux kernel的配置文件，编译busybox需要有一个busybox的配置文件，编译avp需要有一个avp的配置文件等等。

8.2.1 hcboot的配置

```
1
2 $ cd buildroot
3 $ make hcboot-menuconfig
```

8.2.2 DTS的配置

海奇Linux SDK使用device tree进行一些设备驱动的描述。为了便于统一管理，海奇DTS文件统一放在(hclinux/board/hichip/hc16xx/dts)目录

```
1 BR2_LINUX_KERNEL_CUSTOM_DTS_PATH="$(BR2_EXTERNAL_HCLINUX_PATH)/board/hichip/hc16xx/dts/hc16xx-db-a3300-squashfs.dts $(BR2_EXTERNAL_HCLINUX_PATH)/board/hichip/hc16xx/dts/hc16xx-common.dtsi $(BR2_EXTERNAL_HCLINUX_PATH)/board/hichip/hc16xx/dts/hc16xx-dba33000-avp.dtsi $(BR2_EXTERNAL_HCLINUX_PATH)/board/hichip/hc16xx/dts/hc16xx-da330000.dtsi"
```

由于海奇hc16xx系列芯片的双核架构设计以及对应的异构操作系统架构设计，DTS文件内部分为两个部分的配置

- 一部分的设备节点用于AVP RTOS使用，该部分设备节点信息均在DTS的/hcrtos/路径下，具体参考样例(board/hichip/hc16xx/dts/hc16xx-db-a3300-avp.dtsi)

```
1 $ cat board/hichip/hc16xx/dts/hc16xx-db-a3300-avp.dtsi
2 /{
3     hcrtos {
4         ...
5     };
6 };
```

- 其余部分的设备节点用于Linux kernel使用

以目前的设计，uboot内部禁用了DTS功能，uboot没有使用DTS用于uboot内部驱动程序的适配，所以有些适配性的工作需要修改uboot内部的代码，比如适配norflash的pinmux，或者调试串口的pinmux功能。当前uboot仅做最基本的固件加载及启动工作。

uboot启动AVP和linux kernel时，会从norflash加载DTB到指定的内存地址（0x85ff0000）。uboot将DTB传递给AVP及linux kernel的方式有所不同。

uboot的启动命令定义在uboot的配置文件中

```
1 $ cat board/hichip/hc16xx/uboot-configs/hc16xx_db_a3300_defconfig
2 CONFIG_BOOTCOMMAND="mw.b 0xb880006b 0x1; mw.l 0xb8800004 ${fdtaddr}; sf probe; mtdparts
  default; sf read ${fdtaddr} dtb 0x8000; sf read ${loadaddr} avp 0x100000; bootm
  ${loadaddr}; sf read ${loadaddr} linux 0x400000; bootm ${loadaddr} - ${fdtaddr}"
```

```
1 $ cat SOURCE/u-boot/include/configs/hc16xx.h
2 #define CONFIG_LOADADDR          0x86000000
3 #define CONFIG_EXTRA_ENV_SETTINGS \
4     "serverip=172.16.100.95\0" \
5     "ipaddr=172.16.100.200\0" \
6     "fdtaddr=0x85ff0000\0"
```

从uboot的启动命令来看，mw.l 0xb8800004 \${fdtaddr};

DTB (也即fdt)的加载地址为0x85ff0000。uboot会通过0xb8800004寄存器传递DTB加载地址给AVP。AVP启动后会从该寄存器取得DTB地址并进行FDT初始化。

而启动linux内核则采用bootm *loadaddr*—{fdtaddr}的方式将fdtaddr传递给linux kernel。

海奇Linux SDK的DTS当前默认是在编译linux kernel的时候编译生成DTB并安装到buildroot/output/images目录。通过menuconfig可以确认这一点。

```
1 $ cd buildroot
2 $ make menuconfig
3 > kernel
4   ($(BR2_EXTERNAL_HCLINUX_PATH)/board/hichip/hc16xx/dts/hc16xx-db-a3300-squashfs-4k.dts
   $(BR2_EXTERNAL_HCLINUX_PATH)/board/hichip/hc16xx/dts/hc16xx-common.dtsi
   $(BR2_EXTERNAL_HCLINUX_PATH)/board/hichip/hc16xx/dts/hc16xx-db-a3300-avp-4k.dtsi
   $(BR2_EXTERNAL_HCLINUX_PATH)/board/hichip/hc16xx/dts/hc16xx-dba3300.dtsi) Out-of-tree
   Device Tree Source file paths
```

buildroot在编译linux kernel时会同步生成DTB。所以，当DTS有变化被修改后，需要通过重新编译linux kernel来生成新的DTB

```
1 $ cd buildroot
2 $ make linux-rebuild
```

8.2.3 linux kernel的配置

linux kernel 的配置主要分为initramfs以及squashfs两种不同根文件系统下的内核配置，可自行修改

```
1 BR2_LINUX_KERNEL_CUSTOM_CONFIG_FILE="$(BR2_EXTERNAL_HCLINUX_PATH)/board/hichip/hc16xx/kernel-
  configs/kernel-squashfs.config"
```

在buildroot编译完linux后，如果想要修改linux的配置，buildroot可以通过make linux-menuconfig的方式进行。修改完配置后会更新buildroot/output/build/linux-5.12.4/.config

如果希望将修改后的linux配置文件更新为默认的linux配置文件，需要将buildroot/output/build/linux-5.12.4/.config的内容更新到hclinux/board/hichip/hc16xx/kernel-configs/下对应的配置文件。

9. 支持的功能

9.1 linux kernel

module	此版本是否支持	说明
linux-kernel version	Yes	4.4.186 (usb/sdio驱动在5.12.4上仍存在编译问题, 暂时移除5.12.4)
nandflash controller	Yes	
spi sflash controller	Yes	
spi master controller	Yes	
usb-host	Yes	支持usb2.0/usb3.0
usb-device	Yes	
sdio 3.0	Yes	
mac/ethernet	Yes	
i2c	Yes	
fast-uart		
2D GE		
ADC	Yes	
Advance Security		
eFuse	Yes	
IR-rx	Yes	
GPIO	Yes	
PinCtrl	Yes	
PWM		
SGDMA		
Alsa		
Sound	Yes	通过AVP实现sound输出功能
audio decoder	Yes	aac,mp3,adpcm,amr,opus,pcm,vorbis,wma,flac,cookie,ape通过AVP实现audio decoder功能
audio encoder		
video decoder	Yes	h264(up to 4k),mpeg2,mpeg4,VC1,mjpeg,VP8,wmv,RV,BMP 通过AVP实现视频解码功能
framebuffer	Yes	支持double-buffer, 支持上下或左右滚动

module	此版本是否支持	说明
Display Engine	Yes	2k/4k output, 通过AVP实现2k/4k显示引擎功能
CVBS output	Yes	
VGA output		
RGB output	Yes	
LVDS output	Yes	
MIPI DSI output	Yes	
TV Encoder	Yes	
TV Decoder	Yes	
HDMI Tx	Yes	
HDMI Rx	Yes	
CVBS-in	Yes	
DVP-in		
MMZ	Yes	从内核分配大片内存, 或者在不播放音视频的时候, 从mmz分配大片内存
persistentmem	Yes	基于norflash特性实现的支持损耗均衡和掉电数据损坏恢复机制的虚拟永久内存驱动。用于保存音量或其他重要系统数据。hcboot实现同样驱动支持同步访问。

hclinux/configs里面的默认配置已经更换成linux 4.4.186的内核。要把内核版本配置改成linux 5.12.4。如下方法可以更改内核版本。

由于linux上的应用程序依赖内核, 如果内核版本改变, 建议重新编译整个SDK, 如下所示

```

1  $ cd buildroot
2  $ rm -rf output
3  $ make BR2_EXTERNAL=$PWD/../../hichip_hc16xx_db_a3300_squashfs_defconfig
4  $ make menuconfig
5  > kernel
6      kernel version (Latest version (5.12)) --->
7          (X) Latest version (5.12)
8          ( ) Latest CIP SLTS version (4.19.182-cip45)
9          ( ) Latest CIP RT SLTS version (4.19.165-cip41-rt18)
10         ( ) Custom version
11         ( ) Custom tarball
12         ( ) Custom Git repository
13  $ make all

```

9.1.2 完整linux kernel source code

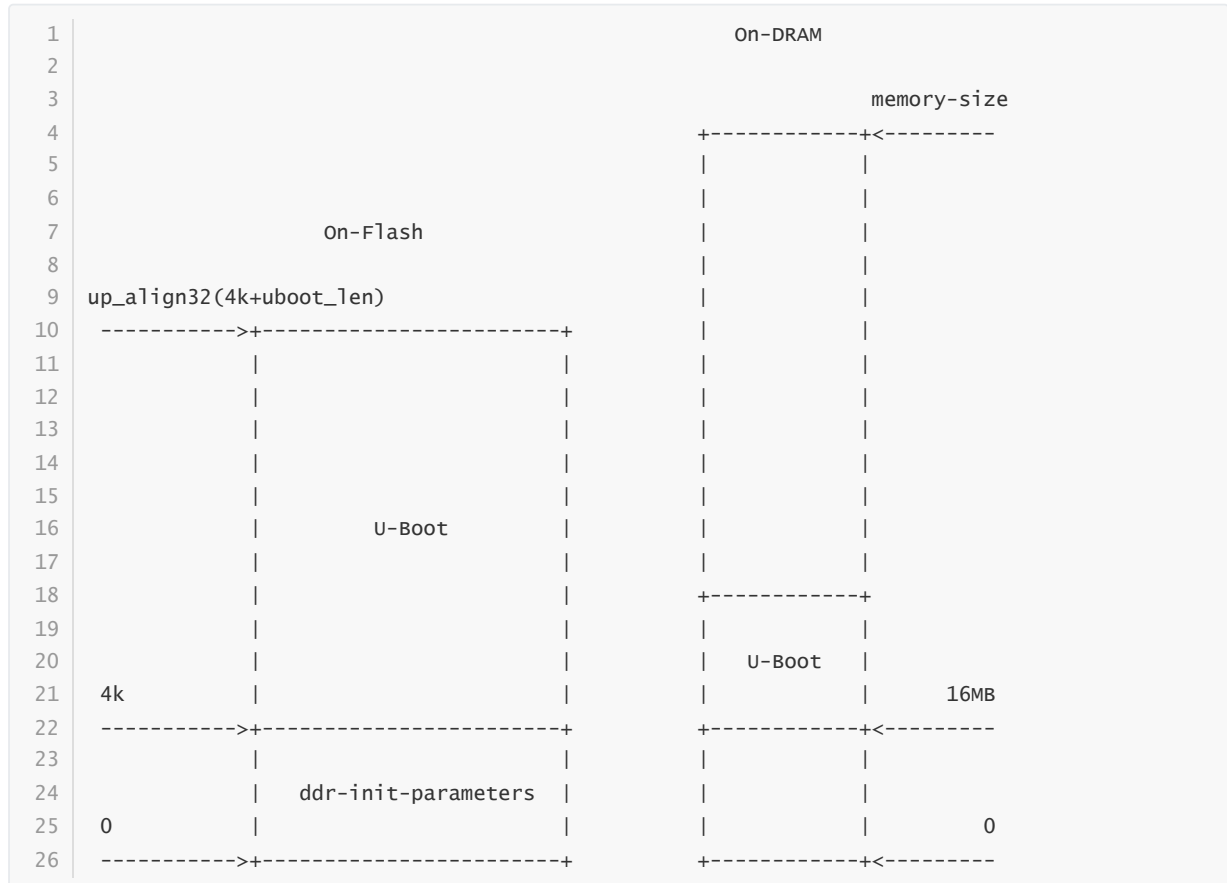
9.2 Bootloader

9.2.1 U-Boot

9.2.1.1 从bootrom到U-Boot

buildroot/output/images/bootloader.bin文件由u-boot.bin和ddr初始化文件合并而成。

ddr初始化文件位于board/hichip/hc16xx/ddrinit目录，每个文件都是4KB。在norflash上ddr初始化文件与U-Boot的mapping方式如下所示。



芯片内部的bootrom程序会从norflash前面4KB获取ddr初始化数据并对内存进行初始化。然后加载U-Boot到内存的16MB位置并运行。ddr初始化文件内部除了ddr参数以外，还有其他参数，包括uboot在norflash上的位置（默认位于4KB位置），uboot的size，以及uboot将被加载到内存的哪一个地址运行（默认加载到16MB位置）。由于U-Boot的size会经常变化，所以在Linux SDK的post-build.sh脚本中合并生成bootloader.bin时，会动态修改ddr初始化文件并将uboot的真实size (向上32字节对齐)进行更新。参考脚本：

```
1 $ cat hclinux/board/hichip/hc16xx/post-build.sh
2 if [ -f ${BINARIES_DIR}/u-boot ] && [ -f ${BINARIES_DIR}/u-boot.bin ] && [ -f
  ${BR2_EXTERNAL_BOARD_DDRINIT_FILE} ] ; then
3     message "Generating bootloader.bin ..."
4     fddrinit=$(basename ${BR2_EXTERNAL_BOARD_DDRINIT_FILE})
5
6     uboot_sz=$(wc -c ${BINARIES_DIR}/u-boot.bin | awk '{print $1}')
7     ((uboot_sz_align=((uboot_sz + 31) / 32) * 32))
8     uboot_sz_hex=$(printf %06x $uboot_sz_align)
9
10    ${BR2_EXTERNAL_HCLINUX_PATH}/support/scripts/binmodify.py \
11        --input ${BR2_EXTERNAL_BOARD_DDRINIT_FILE} --output ${BINARIES_DIR}/${fddrinit} \
12        --offset 0x814 --fill 0x${uboot_sz_hex:0:2}
13    ${BR2_EXTERNAL_HCLINUX_PATH}/support/scripts/binmodify.py \
14        --input ${BINARIES_DIR}/${fddrinit} \
15        --offset 0x819 --fill 0x${uboot_sz_hex:2:2}
16    ${BR2_EXTERNAL_HCLINUX_PATH}/support/scripts/binmodify.py \
17        --input ${BINARIES_DIR}/${fddrinit} \
```

```

18     --offset 0x818 --fill 0x${uboot_sz_hex:4:2}
19
20     uboot_ep_noncache=$(readelf -h ${BINARIES_DIR}/u-boot | grep Entry | awk '{print $NF}'
| sed 's/0x8/0xa/')
21     ${BR2_EXTERNAL_HCLINUX_PATH}/build/scripts/binmodify.py \
22         --input ${BINARIES_DIR}/${fddrinit} \
23         --offset 0x810 --fill 0x${uboot_ep_noncache:4:2}
24     ${BR2_EXTERNAL_HCLINUX_PATH}/build/scripts/binmodify.py \
25         --input ${BINARIES_DIR}/${fddrinit} \
26         --offset 0x811 --fill 0x${uboot_ep_noncache:2:2}
27
28     cat ${BINARIES_DIR}/${fddrinit} ${BINARIES_DIR}/u-boot.bin >
${BINARIES_DIR}/bootloader.bin
29     message "Generating bootloader.bin done!"
30 fi

```

9.2.1.2 uboot启动AVP

由于uboot运行在core-main，而AVP是需要运行在core-avp的，此处巧妙利用了uboot standalone app的启动方式。在制作avp.ulmage的时候，将avp制作成os为u-boot，image-type为standalone。如下所示：

```

1 $ cat board/hichip/hc16xx/post-build.sh
2 if [ -f ${BINARIES_DIR}/avp.bin ] ; then
3     echo "Generating avp.uImage ..."
4     gzip -kf9 ${BINARIES_DIR}/avp.bin > ${BINARIES_DIR}/avp.bin.gz
5     ${HOST_DIR}/bin/mkimage -A mips -O u-boot -T standalone -C gzip -n avp -e ${avp_ep}
-a ${avp_load} -d ${BINARIES_DIR}/avp.bin.gz ${BINARIES_DIR}/avp.uImage
6 fi

```

而AVP的启动代码判断自己运行在core-main的时候，将真正去启动core-AVP，启动完成后，像api调用一个重新返回uboot。从而uboot可以继续加载并启动linux-kernel。

通过uboot的启动命令配置可以看出，uboot通过寄存器0xb880006b传递一个数字"0x1"给AVP。而AVP就是通过该数字决定是否像api调用一个重新返回uboot还是不返回进入deadloop。此方法便于使用GDB下载avp.out单独调试，因为GDB下载avp.out单独调试AVP时，core-main进入deadloop，core-AVP运行AVP RTOS。

```

1 $ cat board/hichip/hc16xx/uboot-configs/hc16xx_db_a3300_defconfig
2 CONFIG_BOOTCOMMAND="mw.b 0xb880006b 0x1; mw.l 0xb8800004 ${fdtaddr}; sf probe; mtdparts
default; sf read ${fdtaddr} dtb 0x8000; sf read ${loadaddr} avp 0x100000; bootm
${loadaddr}; sf read ${loadaddr} linux 0x400000; bootm ${loadaddr} - ${fdtaddr}"

```

9.2.1.3 压缩与解压

在制作ulmage的时候可以指定不同的压缩方式或者不压缩。

9.2.1.4 uboot driver

module	此版本是否支持	说明
usb-host	Yes	仅支持pio mode，有些U盘读写速度比较慢，有时候对某些U盘读取会出错
spl flash controller	Yes	
nandflash controller	Yes	

注：spi-nor/spi-nand boot only

9.2.2 hcboot

U-boot不支持开机show-logo的功能。但U-boot支持nandflash驱动程序。

hcboot基于海奇HCRTOS开发的bootloader，支持开机show-logo功能。默认配置会编译hcboot。

hcboot支持两种格式的logo, jpeg和mpeg2。

9.3 其他功能

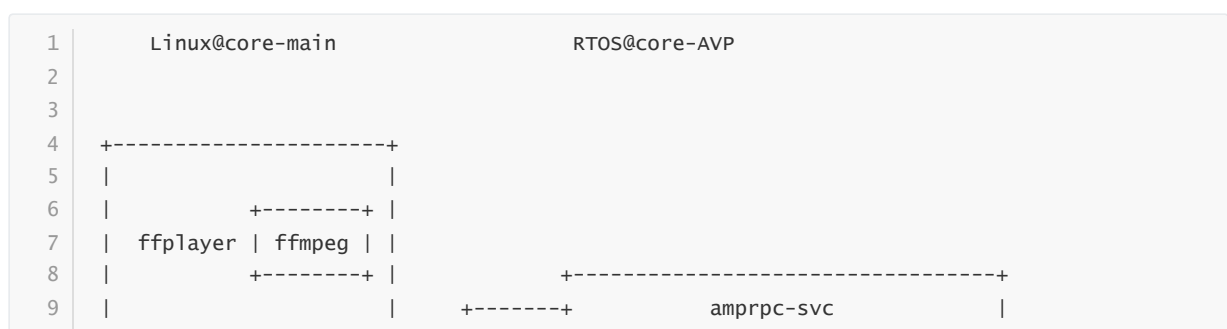
module	此版本是否支持	说明	参考代码
本地、网络多媒体播放	Yes	本地多媒体播放基于ffmpeg开发 demux: mp3 aac ac3 avi asf amr ape amrnb amrwb dts dvbsub dvbtxt eac3 flac flv h261 h263 h264 hevc hls mjpeg m4v mov mpegps mpegts mpegtsraw mjpeg_2000 mpegvideo mjpeg matroska ogg pcm_alaw pcm_f32be pcm_f32le pcm_f64be pcm_f64le pcm_mulaw pcm_s16be pcm_s16le pcm_s24be pcm_s24le pcm_s32be pcm_s32le pcm_s8 pcm_u16be pcm_u16le pcm_u24be pcm_u24le pcm_u32be pcm_u32le pcm_u8 rm rtp rtsp spdif wav parser: aac aac_latm ac3 flac gif h261 h263 h264 hevc jpeg2000 mjpeg mpeg4video mpegaudio mpegvideo opus vc1 vorbis protocols: file ftp data hls http httpproxy mmsh mmst rtmp rtmpt rtp tcp srtp udp	SOURCE/ffplayer-example
DLNA	Yes		SOURCE/libdlna-porting
MiraCast	N/A		
Airplay	N/A		
lvgl	Yes		SOURCE/lvgl
RTL8192CU	Yes	AP mode, Station Mode, P2P mode (目前尚存在稳定性的问题)	

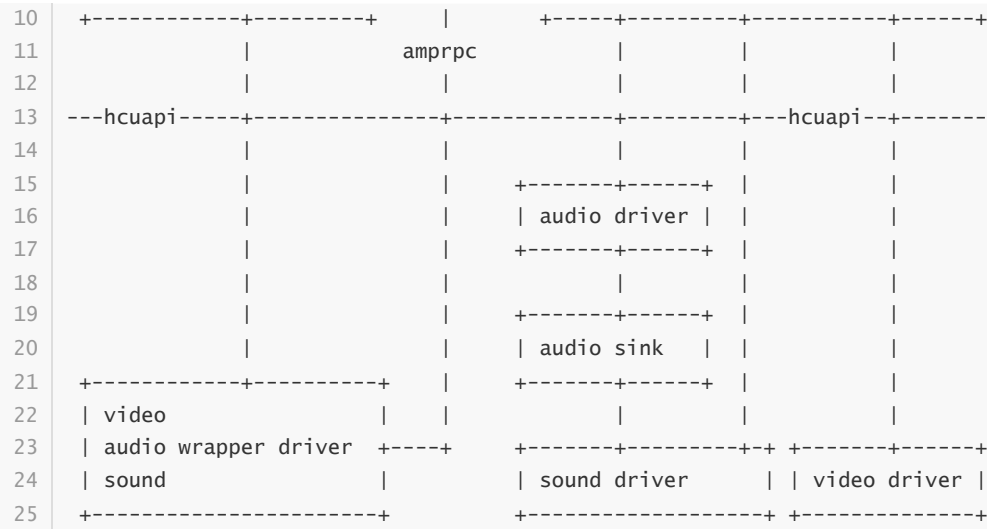
9.4 本地、网络多媒体播放

海奇Linux SDK的本地、网络多媒体播放器（ffplayer）基于开源软件ffmpeg开发设计。利用ffmpeg中的协议解析，容器解析以及demux功能获取到audio/video的packat之后，音视频解码输出与同步则基于AVP提供的audio/video驱动接口实现。

多媒体播放功能的开发即可以基于海奇ffplayer的接口，也可以基于海奇AVP提供的底层audio/video驱动接口实现。

多媒体音视频驱动框架





- Linux内核中的video/audio/sound wrapper driver本身并未实现具体功能，hcuapi中所定义的接口均通过amrprc调用并由RTOS@core-AVP去实现。
- 这里的sound driver并不是alsa驱动，而是基于hcuapi定义的音频解码输出驱动。(未来将提供独立的alsa驱动程序。alsa驱动不能与hcuapi中的audio/sound同时使用，alsa驱动由于其框架本身的原因，不能在驱动层面做音视频同步)
- 目前已支持音频多路独立解码输出。最终会利用audio sink模块进行声音混合输出。目前audio sink模块没有支持resample功能，所以，当进行多路声音混合解码输出时，要求samplerate保持一致。
- 多媒体播放的输入接口，比如文件系统，网络协议等均依靠ffmpeg所支持的协议完成。

9.4.1 ffplayer接口

```

1  $ tree hclinux/SOURCE/prebuilt/usr/
2  hclinux/SOURCE/prebuilt/usr/
3  |— include
4  |   └─ ffplayer.h
5  └─ lib
6     └─ libffplayer.so
7
8  $ cd hclinux/buildroot
9  $ make menuconfig
10 > External options > prebuilts
11     *- ffplayer

```

基于ffplayer提供的example位于hclinux/SOURCE/ffplayer-example

9.4.2 AVP底层audio/video驱动接口

底层audio/video驱动接口的使用方法及调用流程将在未来提供example示范代码。相关的hcuapi接口定义如下：

```

1  $ tree hclinux/SOURCE/linux-drivers/include/uapi/hcuapi/
2  hclinux/SOURCE/linux-drivers/include/uapi/hcuapi/
3  |— auddec.h
4  |— avsync.h
5  |— codec_id.h
6  |— common.h
7  |— iocbase.h
8  |— kumsgq.h
9  |— snd.h
10  /* 由于海奇提供了auddec.h用于支持audio的解码与输出， 此处snd.h一般仅用来get/set音量
    (SND_IOCTL_SET_VOLUME/SND_IOCTL_GET_VOLUME) */
11  |— viddec.h
12  |— vidmp.h

```

9.4.3 mplayer

ffmpeg-example软件包如果被选上，则会生成mplayer可执行程序，它是基于ffmpeg所写的一个测试本地、网络多媒体播放的例子程序，供参考。利用mplayer可执行程序，可以演示本地、网络多媒体播放功能。

```
1 | # mplayer /* 启动mplayer测试程序 */
2 | mp:# play /mnt/usb/test.mkv -s 0 -t 0.1 -d 1 -b 1 /* -s 2表示不做A/V同步(Free Run); -b 1表示做buffering; -d mode(enum IMG_DIS_MODE)表示图片输出模式; -t time 从文件的某个时间点开始播放, time < 1时, 为总时间的百分比, 大于1时, 单位为秒*/
3 | mp:# scan /mnt/usb/media /* 表示扫描/mnt/usb/media文件夹, 并循环播放该文件夹中的媒体文件, 并以audio作为同步参考 */
4 | mp:# scan /mnt/usb/media 0 /* 表示扫描/mnt/usb/media文件夹, 并循环播放该文件夹中的媒体文件, 0表示不做A/V同步 */
5 | mp:# help /* 打印help */
6 | mp:# exit /* 退出mplayer */
```

mplayer内部的其他测试命令可以参考help指令。

9.4.4 使用extractor工具抽取h264视频数据

ffmpeg-example样例程序提供了如何直接基于A/V驱动进行音视频ES数据的播放流程。首先用extractor工具对视频文件抽取视频数据，然后用样例程序将其播放出来。extractor基于linux host主机端的ffmpeg开发。

H264视频抽取工具extractor在目录hclinux/SOURCE/ffmpeg-example/video-extractor, 使用该工具可以抽取H264数据，并在每一帧之前加4个字节，表示这一帧数据的大小，运行命令如下：

```
1 | extractor <输入视频文件> <输出h264文件>
```

如果需要自行编译extractor，可以参考

```
1 | hclinux/SOURCE/ffmpeg-example/video-extractor/README
```

9.4.5 播放9.4.4抽取出来的h264文件

```
1 | #mplayer
2 | mp:#h264_es_play <h264文件>
3 | mp:#h264_es_stop
```

9.5 DLNA

9.6 AirPlay

9.7 Miracast

9.8 Wifi

目前支持标准的USB驱动，其他Wifi驱动可基于标准USB驱动进行移植。

支持如下四种Wifi:

- USB wifi
 - rtl8188fu (2.4G wifi)
 - rtl8811cu (5G wifi)
- SDIO wifi

- o rtl8723as (2.4G wifi)
- o rtl8822cs (5G wifi)

9.9 开机show-logo/show-av

默认采用hcboot，支持开机show logo功能。默认采用的是mpeg2的logo文件。hcboot还支持jpeg格式的logo文件。

9.10 USB移动存储设备自动mount/umount

支持U盘，移动硬盘等USB移动存储设备插入和拔出时自动mount/umount。

支持FAT32, NTFS, exFAT文件系统格式。

相关脚本如下所示：

```
1 $ cd hclinux
2 $ tree SOURCE/prebuilts/etc/
3 SOURCE/prebuilts/etc/
4 |— init.d
5 |   |— s10mdev
6 |— mdev
7 |   |— mount-helper.sh
8 |   |— mount.sh
9 |— mdev.conf
10
```

10. 有关不同板子的配置

10.1 DDR配置

ddr配置文件放在hclinux/board/hichip/hc16xx/ddrinit/, 通过menuconfig配置选择不同的ddr参数文件，该文件会被hclinux/board/hichip/hc16xx/post-build.sh利用并在make all的时候用于生成最终的bootloader.bin。冷启动的时候bootrom会利用该配置对DDR进行初始化。

```
1 $ tree hclinux/board/hichip/hc16xx/ddrinit/
2 hclinux/board/hichip/hc16xx/ddrinit/
3 |— aux_code_ddr2_128M_1066MHZ.abs
4 |— aux_code_ddr2_128M_800MHZ.abs
5 |— aux_code_ddr2_64M_800MHZ.abs
6 |— aux_code_ddr3_128M_1066MHZ.abs
7 |— aux_code_ddr3_128M_1333MHZ.abs
8 |— aux_code_ddr3_128M_1464MHZ.abs
9 |— aux_code_ddr3_128M_1600MHZ.abs
10 |— aux_code_ddr3_128M_800MHZ.abs
11 |— aux_code_ddr3_256M_1066MHZ.abs
12
13 $ cd hclinux/buildroot
14 $ make menuconfig
15 > External options
16 ({BR2_EXTERNAL_HCLINUX_PATH}/board/hichip/hc16xx/ddrinit/aux_code_ddr3_128M_800MHZ.abs)
   DDRinit file
```

用GDB进行调试的时候，需要对GDB工具进行配置，以设置内存初始化参数。GDB工具所使用的DDR参数配置文件在如下路径，根据不同的板子选择不同的参数文件。

```
1 $ cd hclinux
2 $ tree board/hichip/hc16xx/ddrinit/gdb/
3 board/hichip/hc16xx/ddrinit/gdb/
4 |— H1600_0001_ddr2_Clk800MHz_128M_20200923.abs
```

```

5 | └─ H1600_0002_ddr2_clk1066MHz_128M_20211201.abs
6 | └─ H1600_0003_ddr3_clk800MHz_128M_20201021.abs
7 | └─ H1600_0004_ddr3_clk1066MHz_128M_20201021.abs
8 | └─ H1600_0005_ddr3_clk1066MHz_256M_20211111.abs
9 | └─ H1600_0006_ddr3_clk1333MHz_128M_20211201.abs
10 | └─ H1600_0007_ddr3_clk1333MHz_256M_20211201.abs
11 | └─ H1600_0008_ddr3_clk1600MHz_128M_20201021.abs
12 | └─ H1600_0009_ddr3_clk1600MHz_256M_20201021.abs
13 | └─ H1600A_ddr2_1066M_128M.abs
14 | └─ H1600A_ddr2_1066M_256M.abs
15 | └─ H1600A_ddr2_800M_128M.abs
16 | └─ H1600A_ddr2_800M_256M.abs
17 | └─ H1600A_ddr3_1066M_128M_176pin.abs
18 | └─ H1600A_ddr3_1066M_128M.abs
19 | └─ H1600A_ddr3_1066M_256M_176pin.abs
20 | └─ H1600A_ddr3_1066M_256M.abs
21 | └─ H1600A_ddr3_1333M_128M.abs
22 | └─ H1600A_ddr3_1333M_256M.abs
23 | └─ H1600A_ddr3_1600M_128M.abs
24 | └─ H1600A_ddr3_1600M_256M.abs
25 | └─ H1600A_ddr3_800M_128M_176pin.abs
26 | └─ H1600A_ddr3_800M_256M_176pin.abs

```

10.2 调试串口配置

以hc16xx-db-a3300为例，在hclinux/board/hichip/hc16xx/dts/hc16xx-db-a3300.dtsi文件中指定了serial0设备描述，此处说明使用串口3（四个串口标号为0/1/2/3）

```

1 | $ cat hclinux/board/hichip/hc16xx/dts/hc16xx-db-a300.dtsi
2 | aliases {
3 |     serial0 = &uart3;
4 | };

```

另外，在hclinux/board/hichip/hc16xx/dts/hc16xx-db-a300-squashfs.dts中指定了linux内核早期的串口使用的串口外设地址0x18818900，对应串口3。

```

1 | $ cat hclinux/board/hichip/hc16xx/dts/hc16xx-db-a300-squashfs.dts
2 | &chosen {
3 |     bootargs = "root=/dev/mtdblock4 rw rootfstype=squashfs init=/linuxrc
4 |     console=ttyS0,115200n8 earlycon=uart8250,mmio,0x18818900,115200n8 no_console_suspend";
5 | };

```

对应其他串口的的外设地址可以查阅datasheet。

uart port	外设地址
uart 0	0x18818300 (bootargs这里使用物理地址，linux内核会映射到虚拟地址)
uart 1	0x18818600
uart 2	0x18818800
uart 3	0x18818900

第三，如果使用的是U-boot，则需要在SOURCE/u-boot/board/hichip/hc16xx/board.c中有关调试串口的pinmux配置，请参考datasheet查阅有关pinmux配置所对应的寄存器地址及其值。如果使用的是hcboot，则修改相应DTS配置即可，不需要改hcboot源码。

```

1 | $ cat hclinux/SOURCE/u-boot/board/hichip/hc16xx/board.c
2 | #if !CONFIG_IS_ENABLED(OF_CONTROL)

```

```

3
4 /* 此处指定调试串口的外设基地址，此处要用虚拟地址0xb88xxxxx */
5 #if CONFIG_IS_ENABLED(BOARD_HC16XX_DB_A300_V10) || \
6     CONFIG_IS_ENABLED(BOARD_HC16XX_SB_A300_V10)
7 static const struct ns16550_plat hc16xx_serial[] = {
8     { .base = 0xb8818900, .reg_shift = 0,
9       .clock = 1843200, .fcr = UART_FCR_DEFVAL, },
10 };
11 #else
12 static const struct ns16550_plat hc16xx_serial[] = {
13     { .base = 0xb8818300, .reg_shift = 0,
14       .clock = 1843200, .fcr = UART_FCR_DEFVAL, },
15 };
16 #endif
17
18 U_BOOT_DRVINFOS(hc16xx_uarts) = {
19     { "ns16550_serial", &hc16xx_serial[0] },
20 };
21 #endif
22
23 /* 此处为调试串口的pinmux及strapin配置，具体pinmux配置可查阅datasheet */
24 #if CONFIG_IS_ENABLED(BOARD_EARLY_INIT_F)
25 int board_early_init_f(void)
26 {
27     #if CONFIG_IS_ENABLED(BOARD_HC16XX_DB_A300_V10) || \
28         CONFIG_IS_ENABLED(BOARD_HC16XX_SB_A300_V10)
29         writeb(4, (void *)0xb88004ac);
30         writeb(4, (void *)0xb88004ad);
31     #elif CONFIG_IS_ENABLED(BOARD_HC16XX_DB_C300_V10)
32         writeb(4, (void *)0xb8800576);
33         writeb(4, (void *)0xb8800577);
34         writel(0x00040000, (void *)0xb8800094); /* c300的板子使用了uart0，但不是默认的pin脚，
35         所以需要将strapin的默认uart0 pin脚绑定关系解除 */
36     #endif
37     return 0;
38 }
39 #endif

```

第四，以hichip_hc16xx_db_a300_squashfs_defconfig为例，uart的pinmux配置在DTS中进行正确设置。

```

1 $ cat hclinux/board/hichip/hc16xx/dts/hc16xx-db-a300.dtsi
2 &uart3 {
3     status = "okay";
4     pinctrl-names = "default";
5     pinctrl-0 = <&pctl_uart3>;
6 };
7
8
9 &pinctrl {
10     functions {
11         pctl_uart3: pctl_uart3 {
12             name = "pctl_uart3";
13             groups = "GPIO_L_12", "GPIO_L_13";
14             field-value-0 = <0x4>;
15             field-value-1 = <0x4>;
16         };
17     };
18 };

```

10.3 Uboot内部USB驱动

Uboot内部的USB默认使用port 1，如果要使用port 0，则需要修改对应代码 (u32 ulUsbUsePort=0x01;)

```
1 $ cat hclinux/SOURCE/u-boot/board/hichip/hc16xx/board.c
2 #ifdef CONFIG_USB_MUSB_HICHIP
3 void hc16xx_evm_musb_init(void)
4 {
5     /*
6      * Set up USB clock/mode in the DEVCONF2 register.
7      * USB2.0 PHY reference clock is 13 MHz
8      */
9     u32 ulUsbUsePort=0x01;
10    u32 ulUsbBase=hc16xx_get_base(ulUsbUsePort);
11    hc16xx_set_port(ulUsbUsePort);
12    musb_register(&musb_plat, &musb_board_data,
13                (void *)ulUsbBase);
14 }
15 #endif
16
```

10.4 Uboot内部norflash驱动

可以通过Uboot的menuconfig来配置选上不同的norflash型号的驱动程序。如果已经编译过完整的sdk，此时可以通过如下命令修改uboot的配置，然后重新编译uboot。

```
1 $ cd hclinux/buildroot
2 $ make uboot-menuconfig
3 > Device Drivers > MTD Support > SPI Flash Support
4 [*] SPI Flash Core Interface support
5 (0) SPI Flash default bus identifier
6 (0) SPI Flash default Chip-select
7 (3) SPI Flash default mode (see include/spi.h)
8 (1000000) SPI Flash default speed in Hz
9 [ ] SFDP table parsing support for SPI NOR flashes
10 [ ] SPI flash Bank/Extended address register support
11 [*] Unlock the entire SPI flash on u-boot startup
12 [ ] SPI DUAL flash memory support
13 [ ] Atmel SPI flash support
14 [ ] EON SPI flash support
15 [*] GigaDevice SPI flash support
16 [ ] ISSI SPI flash support
17 [*] Macronix SPI flash support
18 [ ] Spansion SPI flash support
19 [ ] STMicro SPI flash support
20 [ ] SST SPI flash support
21 [*] winbond SPI flash support
22 [ ] XMC SPI flash support
23 [*] Use small 4096 B erase sectors
24 [*] SPI Flash MTD support
25 [ ] SPI flash MTD support for SPL

```

Uboot内的spi flash controller驱动程序有默认的pinmux配置，以及默认的1线，2线，4线配置，如果需要不同的配置，需要修改对应代码。

```
1 $ cat hclinux/SOURCE/u-boot/drivers/spi/hc_spi.c
2 struct hc_spi_pinmux {
3     int pin;
4     char mux;
5 };
6
```

```

7 static void *regpinmux[4] = {
8     (void *)HC_PINMUX_IOBASE + 0x4a0,
9     (void *)HC_PINMUX_IOBASE + 0x4e0,
10    (void *)HC_PINMUX_IOBASE + 0x520,
11    (void *)HC_PINMUX_IOBASE + 0x560,
12 };
13
14 /* 修改此处pin脚, 及其对应pinmux select值, pin脚序号从0~127 (对于L/B/R/T四组, 可以参考
15    hclinux/SOURCE/linux-drivers/include/uapi/hcuapi/pinpad.h) */
16 static struct hc_spi_pinmux spi_pinmux[] = {
17     {110, 1},
18     {111, 1},
19     {112, 1},
20     {113, 1},
21     {114, 1},
22     {115, 1},
23 };
24
25 static int pinmux_configure(int pin, char mux)
26 {
27     void *reg = regpinmux[pin / 32] + pin % 32;
28
29     writeb(mux, reg);
30
31     return 0;
32 }
33
34 static void hc_spi_pinmux_set(void)
35 {
36     int i;
37
38     for (i = 0; i < ARRAY_SIZE(spi_pinmux); i++) {
39         pinmux_configure(spi_pinmux[i].pin, spi_pinmux[i].mux);
40     }

```

```

1 $ cat hclinux/SOURCE/u-boot/drivers/spi/hc_spi.c
2 struct spi_slave *spi_setup_slave(unsigned int bus, unsigned int cs,
3                                     unsigned int max_hz, unsigned int mode)
4 {
5     ...
6     hs->slave.mode = SPI_RX_SLOW | SPI_TX_BYTE;
7     //hs->slave.mode = SPI_RX_DUAL | SPI_TX_DUAL;
8     //hs->slave.mode = SPI_RX_QUAD | SPI_TX_BYTE;
9     ...
10 }

```

当前uboot的spi sflash controller驱动Rx支持1/2/4线, 而Tx仅支持1线。(Linux kernel的spi sflash controller驱动无此问题), 但需要在DTS中设置spi-tx-bus-width/spi-rx-bus-width。如下所示:

```

1 $ cat hclinux/board/hichip/hc16xx/dts/hc16xx-db-a300.dtsi
2
3 &spi_flash {
4     spi-tx-bus-width = <1>;
5     spi-rx-bus-width = <1>;
6
7     partitions {
8         compatible = "fixed-partitions";
9         #address-cells = <1>;
10

```


10.4.1 不支持的norflash型号

可以在uboot命令行中通过sf probe命令探测norflash型号

```
1 => sf probe
2 SF: Detected gd25q128 with page size 256 Bytes, erase size 4 KiB, total 16 MiB
```

如果遇到不支持的型号，可以看到其device id号为0x684018

- 0x68表示厂家

```
1 => sf probe
2 (NULL udevice *): unrecognized JEDEC id bytes: 68, 40, 18
```

- 如果norflash的厂家在uboot中已经有支持，一般直接在该厂家下添加相应id即可，
- 如果uboot中没有相应厂家，则咨询厂家，其norflash与uboot/linux中哪些著名厂家是兼容的，在兼容的厂家列表中添加相应id即可
- 如果找不到厂家支持，则可以尝试在device id中寻找厂家不同，但其他id相同的，参考其配置添加id。这种方法并不十分保险，需要找norflash的datasheet研究并确认其特性。
- 以博雅norflash为例，博雅售后告之其norflash与gigadevice是兼容的，于是，在gigadevice的型号列表中添加博雅

```
1 diff --git a/drivers/mtd/spi/spi-nor-ids.c b/drivers/mtd/spi/spi-nor-ids.c
2 index 4ae067a655..9821c39bae 100644
3 --- a/drivers/mtd/spi/spi-nor-ids.c
4 +++ b/drivers/mtd/spi/spi-nor-ids.c
5 @@ -123,7 +123,7 @@ const struct flash_info spi_nor_ids[] = {
6         SPI_NOR_HAS_LOCK | SPI_NOR_HAS_TB)
7     },
8     {
9 -         INFO("boya25q128", 0x684018, 0, 64 * 1024, 256,
10 +         INFO("by25q128", 0x684018, 0, 64 * 1024, 256,
11             SECT_4K | SPI_NOR_DUAL_READ | SPI_NOR_QUAD_READ |
12             SPI_NOR_HAS_LOCK | SPI_NOR_HAS_TB)
13     },
```

- 相应的，linux中添加id可以参考

```
1 $ cat hclinux/patches/linux-5.12.4/0009-adding-by25q128-ids.patch
2 diff --git a/drivers/mtd/spi-nor/gigadevice.c b/drivers/mtd/spi-nor/gigadevice.c
3 index 447d84b..63eb61d 100644
4 --- a/drivers/mtd/spi-nor/gigadevice.c
5 +++ b/drivers/mtd/spi-nor/gigadevice.c
6 @@ -45,6 +45,9 @@ static const struct flash_info gigadevice_parts[] = {
7     { "gd25q128", INFO(0xc84018, 0, 64 * 1024, 256,
8         SECT_4K | SPI_NOR_DUAL_READ | SPI_NOR_QUAD_READ |
9         SPI_NOR_HAS_LOCK | SPI_NOR_HAS_TB) },
10 + { "by25q128", INFO(0x684018, 0, 64 * 1024, 256,
11 +     SECT_4K | SPI_NOR_DUAL_READ | SPI_NOR_QUAD_READ |
12 +     SPI_NOR_HAS_LOCK | SPI_NOR_HAS_TB) },
13     { "gd25q256", INFO(0xc84019, 0, 64 * 1024, 512,
14         SECT_4K | SPI_NOR_DUAL_READ | SPI_NOR_QUAD_READ |
15         SPI_NOR_4B_OPCODES | SPI_NOR_HAS_LOCK |
16     },
```

10.5 Uboot内部nandflash驱动

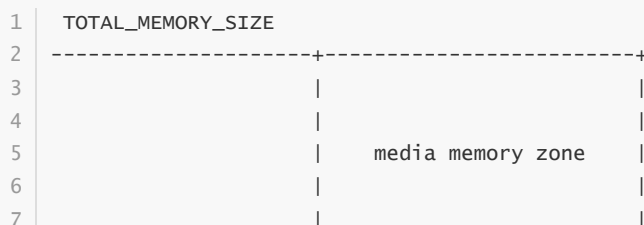
与nandflash驱动相关的内容主要是pinmux，不同的板或芯片可能有不同的pinmux。需要根据不同的板进nandflash pinmux设置。

```
1 $ cat hclinux/SOURCE/u-boot/drivers/mtd/nand/raw/hc_nand.c
2 #if CONFIG_IS_ENABLED(BOARD_HC16XX_DB_C200_V10)
3 static struct hc_nfc_pinmux nfc_pinmux[] = {
4     {37, 0x4e5, 2},
5     {38, 0x4e6, 2},
6     {39, 0x4e7, 2},
7     {40, 0x4e8, 2},
8     {41, 0x4e9, 2},
9     {42, 0x4ea, 2},
10    {43, 0x4eb, 2},
11    {44, 0x4ec, 2},
12    {45, 0x4ed, 2},
13    {46, 0x4ee, 2},
14    {47, 0x4ef, 2},
15    {48, 0x4f0, 2},
16    {49, 0x4f1, 2},
17    {50, 0x4f2, 2},
18    {51, 0x4f3, 2},
19 };
20 #elif CONFIG_IS_ENABLED(BOARD_HC16XX_SB_C300_V10)
21 static struct hc_nfc_pinmux nfc_pinmux[] = {
22     {42, 0x4ea, 1},
23     {43, 0x4eb, 1},
24     {44, 0x4ec, 1},
25     {45, 0x4ed, 1},
26     {46, 0x4ee, 1},
27     {47, 0x4ef, 1},
28     {48, 0x4f0, 1},
29     {49, 0x4f1, 1},
30     {50, 0x4f2, 1},
31     {51, 0x4f3, 1},
32     {52, 0x4f4, 1},
33     {53, 0x4f5, 1},
34     {54, 0x4f6, 1},
35     {55, 0x4f7, 1},
36     {56, 0x4f8, 1},
37 };
38 #else
39 #error "Not support nandflash with the board"
40 #endif
41
```

10.6 内存空间配置

hclinux方案采用双CPU方案，主CPU运行linux操作系统，Slave CPU（称之为AVP处理器）运行海奇HCRTOS操作系统。

内存空间从低地址到高地址分为三段：





名称	起始地址	结束地址	说明
linux	0	CONFIG_LINUX_MEMORY_SIZE	用于linux运行的内存空间
hcartos system memory	CONFIG_LINUX_MEMORY_SIZE	CONFIG_LINUX_MEMORY_SIZE + HCRTOS_SYSMEM_SIZE	用于avp rtos系统运行的内存空间
media memory zone	CONFIG_LINUX_MEMORY_SIZE + HCRTOS_SYSMEM_SIZE	CONFIG_MEMORY_SIZE	用于video解码时动态分配的空间

- linux的内存地址从0开始
- HCRTOS@AVP的内存地址从CONFIG_LINUX_MEMORY_SIZE开始，其地址根据HCRTOS_MMZ0_SIZE和HCRTOS_SYSMEM_SIZE自动计算
- HCRTOS@AVP的系统内存空间大小由DTS配置。MMZ0的内存空间也由DTS配置。参考 board/hichip/hc16xx/dts/hc16xx-db-a300.dtsi, hc16xx-db-a300-avp.dtsi:

```

1 $ cat hclinux/board/hichip/hc16xx/dts/hc16xx-db-a300.dtsi
2 #define CONFIG_AVP_MEMORY_SIZE (HCRTOS_SYSMEM_SIZE + HCRTOS_MMZ0_SIZE)
3 #define CONFIG_LINUX_MEMORY_SIZE (CONFIG_MEMORY_SIZE - CONFIG_AVP_MEMORY_SIZE)
4 #define CONFIG_LINUX_MEMORY_OFFSET (0)
5
6 /{
7     #address-cells = <1>;
8     #size-cells = <1>;
9     memory {
10         device_type = "memory";
11         reg = <CONFIG_LINUX_MEMORY_OFFSET CONFIG_LINUX_MEMORY_SIZE>;
12     };

```

```

1 $ cat hclinux/board/hichip/hc16xx/dts/hc16xx-db-a300-avp.dtsi
2 /{
3     hcrtos {
4         #address-cells = <1>;
5         #size-cells = <1>;
6         memory-mapping {
7             #address-cells = <1>;
8             #size-cells = <1>;
9             sysmem {
10                 reg = <HCRTOS_SYSMEM_OFFSET HCRTOS_SYSMEM_SIZE>;
11             };
12
13             mmz0 {
14                 id = <0>;
15                 reg = <HCRTOS_MMZ0_OFFSET HCRTOS_MMZ0_SIZE>;
16             };
17     };
18

```

- 对于MMZ0来说，其主要用于HCRTOS@AVP内video驱动进行video解码，如2k/4k系统可根据实际情况配置不同的内存空间。当没有进行解码时MMZ0的内存是闲置的。未来将通过hcuapi的接口释放出来让Linux内核驱动可以获取该处内存。比较典型的应用是，在内存吃紧的时候，如果没有播放视频，则可以利用其来实现高清OSD显示或多画面滑动OSD显示。
- mmz0支持0~10个，可根据实际应用分配和利用独立的内存空间。
- Linux内核的load address是根据DTS里面的内存空间分配而动态变化的。在编译Linux内核的时候会根据具体所使用的DTS内存配置参数配置内核config。具体实现如下所示，最终会修改内核.config中的CONFIG_PHYSICAL_START变量。

```

1 $ cat hclinux/linux/linux-ext-fixup-load-addr.mk
2 define LINUX_PHYSICAL_START_FROM_DTS
3     ...
4 endef
5 LINUX_PRE_BUILD_HOOKS += LINUX_PHYSICAL_START_FROM_DTS

```

- AVP的load address同样是根据DTS里面的内存空间分配而动态变化的。在编译AVP的时候会根据具体所使用的DTS内存配置参数配置AVP的load address。具体实现如下所示。

```

1 $ cat hclinux/SOURCE/avp/components/applications/apps-avp/apps-avp.mk
2 ifneq ($(CONFIG_CUSTOM_DTS_PATH),)
3 define APPS_AVP_ENTRY_ADDR_FROM_DTS
4 ...
5 endef
6 else ifeq ($(BR2_PACKAGE_APPS_AVP),y)
7 $(info "CONFIG_CUSTOM_DTS_PATH is not defined, not able to update entry address of avp
  application")
8 endif
9 APPS_AVP_PRE_BUILD_HOOKS += APPS_AVP_ENTRY_ADDR_FROM_DTS

```

10.7 驱动程序头文件

hclinux SDK的linux内核驱动程序都放在SOURCE/linux-drivers

驱动程序的头文件可用于应用程序引用开发

```

1 $ tree hclinux/SOURCE/linux-drivers/include/uapi/hcuapi/
2 SOURCE/linux-drivers/include/uapi/hcuapi/
3 ├── amprpc.h      /* linux kernel与AVP之间进行远程调用的接口 */
4 ├── auddec.h      /* audio decoder驱动 */
5 ├── audsink.h     /* audio sink驱动，介于audio decoder与sound driver之间，以支持多路解码混合输出
  */
6 ├── avevent.h     /* 音视频驱动事件 */
7 ├── avsync.h      /* 音频视频同步 */
8 ├── codec_id.h    /* 音频视频codec id，参考ffmpeg定义 */
9 ├── common.h      /* 公共定义 */
10 ├── dis.h         /* Display */
11 ├── fb.h          /* framebuffer，linux标准以外海奇自定义的framebuffer接口 */
12 ├── gpio.h        /* 用于RTOS@AVP的gpio驱动(RTOS only) */
13 ├── hdmi.h        /* HDMI */
14 ├── iocbase.h     /* Unify IO Command Base */
15 ├── kshm.h        /* linux kernel与AVP之间进行大数据共享，常用于音视频播放时以共享memory方式传递A/V
  stream */
16 ├── kumsgq.h      /* 用于linux user-space / kernel-space之间的message消息通知 */
17 ├── notifier.h    /* 用于linux kernel与AVP之间由AVP向linux kernel发送的通知事件 */
18 ├── pinmux        /* 用于RTOS@AVP的pinmux驱动(RTOS only) */
19 |   ├── hc15xx_pinmux.h
20 |   └── hc16xx_pinmux.h
21 ├── pinmux.h      /* 用于RTOS@AVP的pinmux驱动(RTOS only) */
22 ├── pinpad.h      /* 用于RTOS@AVP的pinmux驱动(RTOS only) */
23 ├── ramdisk.h     /* 用于RTOS@AVP的ramdisk驱动(RTOS only) */
24 ├── sci.h         /* 用于RTOS@AVP的串口驱动(RTOS only) */
25 ├── snd.h         /* sound驱动，一般由audio sink向sound输出声音，应用不直接向sound输出声音。但可用
  于设置音量 */
26 ├── tvtype.h      /* 电视制式定义 */
27 ├── viddec.h      /* video decoder */
28 ├── vidmp.h       /* 多媒体播放所需参数定义 */
29 └──
30 .h      /* 虚拟串口，用于linux与AVP进行通信。linux可通过该虚拟串口获取RTOS@AVP的打印输出及发送命令 */

```

由于有些头文件是用于linux内核与AVP处理器进行RPC通信的，应用程序开发可依实际情况引用正确的头文件。

10.8 GDB调试工具

下载gdb.rar，内部有用于GDB调试工具的内存初始化文件(gdb\DDR3)

参考文档WIN GDB使用手册.pdf

下载ICE Driver.rar，内部有jtag调试工具的驱动程序，以及readme.txt说明安装步骤。安装好后，建议重启电脑使用。

自hclinux-2021.12.y之后，在sdk内部就包含了相应ddr初始化文件供GDB调试工具使用。

```
1 hclinux$ tree board/hichip/hc16xx/ddrinit/gdb/
2 board/hichip/hc16xx/ddrinit/gdb/
3 |— H1600_0001_ddr2_Clk800MHz_128M_20200923.abs
4 |— H1600_0002_ddr2_Clk1066MHz_128M_20211201.abs
5 |— H1600_0003_ddr3_Clk800MHz_128M_20201021.abs
6 |— H1600_0004_ddr3_Clk1066MHz_128M_20201021.abs
7 |— H1600_0005_ddr3_Clk1066MHz_256M_20211111.abs
8 |— H1600_0006_ddr3_Clk1333MHz_128M_20211201.abs
9 |— H1600_0007_ddr3_Clk1333MHz_256M_20211201.abs
10 |— H1600_0008_ddr3_Clk1600MHz_128M_20201021.abs
11 |— H1600_0009_ddr3_Clk1600MHz_256M_20201021.abs
```

10.9 buildroot patch管理

参考buildroot手册hclinux/buildroot/output/docs/manual/manual.html，第9.8节，第19章。先要生成手册文档。

```
1 $ cd buildroot
2 $ make manual
```

海奇Linux SDK的patch放在hclinux/patches目录

```
1 $ tree hclinux/patches/
2 patches/
3 |— ffmpeg
4 |   └─ 0001-ffmpeg-aac-parser.patch
5 └─ linux
   |— 0001-add-hichip-hc16xx-arch-support.patch
   |— 0002-fix-hichip-pinctrl.patch
   |— 0003-fix-r4k-dma-cache.patch
   |— 0004-fix-uart-tx-hang-issue.patch
   |— 0006-adding-new-nand-ids.patch
   |— 0007-adding-hcd drivers.patch
   └─ 0008-adding-hichip-musb.patch
```

未来海奇针对发布的SDK可能会修复一些Bug，除少部分以binary形式发布的库以外，其它以源文件发布的代码均按照patch的方式进行修复。

基于海奇Linux SDK进行二次开发的过程中，尽量建立自己的board及相关的配置文件，不对原SDK的配置文件作修改，便于未来适配海奇发布的patch包。

11. 如何将sdk配置由squashfs改成initramfs

hclinux中会有一些demo board的默认配置

```
1 hclinux$ tree configs/
2 configs/
3 |— hichip_hc16xx_db_a300_initramfs_defconfig
4 |— hichip_hc16xx_db_a300_squashfs_4k_defconfig
5 |— hichip_hc16xx_db_a300_squashfs_defconfig
6 |— hichip_hc16xx_db_a3200_squashfs_defconfig
7 |— hichip_hc16xx_db_b300_initramfs_defconfig
8 |— hichip_hc16xx_db_b300_squashfs_defconfig
9 |— hichip_hc16xx_db_c300_initramfs_defconfig
10 └─ hichip_hc16xx_db_c300_squashfs_defconfig
```

其中，带有squashfs字眼的，其配置的根文件系统格式为squashfs，为只读文件系统，存储在norflash分区的mtddblock4，需要norflash中烧录各分区才能够启动。且需要从norflash启动。

其中，带有initramfs字眼的，其配置的根文件系统格式为initramfs，为可读写文件系统，掉电不会保存的。可用于GDB download到内存运行(download.ini)

```
1 | hclinux/buildroot/output/images/download.ini
```

squashfs与initramfs配置的差异中，有关wifi的配置是可以从squashfs配置中同步到initramfs配置中去的。同步过去后initramfs的配置就支持wifi rtl8192cu了。

```
1 | diff --git a/configs/hichip_hc16xx_db_a300_initramfs_defconfig
  b/configs/hichip_hc16xx_db_a300_initramfs_defconfig
2 | index eb97cd2..435e057 100644
3 | --- a/configs/hichip_hc16xx_db_a300_initramfs_defconfig
4 | +++ b/configs/hichip_hc16xx_db_a300_initramfs_defconfig
5 | @@ -2162,9 +2162,28 @@ BR2_PACKAGE_MONGREL2_LIBC_SUPPORTS=y
6 |  # BR2_PACKAGE_WIREGUARD_LINUX_COMPAT is not set
7 |  # BR2_PACKAGE_WIREGUARD_TOOLS is not set
8 |  # BR2_PACKAGE_WIRELESS_REGDB is not set
9 | -# BR2_PACKAGE_WIRELESS_TOOLS is not set
10 | +BR2_PACKAGE_WIRELESS_TOOLS=y
11 | +BR2_PACKAGE_WIRELESS_TOOLS_LIB=y
12 |  # BR2_PACKAGE_WIRESHARK is not set
13 | -# BR2_PACKAGE_WPA_SUPPLICANT is not set
14 | +BR2_PACKAGE_WPA_SUPPLICANT=y
15 | +BR2_PACKAGE_WPA_SUPPLICANT_NL80211=y
16 | +BR2_PACKAGE_WPA_SUPPLICANT_WEXT=y
17 | +# BR2_PACKAGE_WPA_SUPPLICANT_WIRED is not set
18 | +BR2_PACKAGE_WPA_SUPPLICANT_IBSS_RSN=y
19 | +BR2_PACKAGE_WPA_SUPPLICANT_AP_SUPPORT=y
20 | +BR2_PACKAGE_WPA_SUPPLICANT_WIFI_DISPLAY=y
21 | +# BR2_PACKAGE_WPA_SUPPLICANT_MESH_NETWORKING is not set
22 | +BR2_PACKAGE_WPA_SUPPLICANT_AUTOSCAN=y
23 | +BR2_PACKAGE_WPA_SUPPLICANT_EAP=y
24 | +BR2_PACKAGE_WPA_SUPPLICANT_HOTSPOT=y
25 | +# BR2_PACKAGE_WPA_SUPPLICANT_DEBUG_SYSLOG is not set
26 | +BR2_PACKAGE_WPA_SUPPLICANT_WPS=y
27 | +# BR2_PACKAGE_WPA_SUPPLICANT_WPA3 is not set
28 | +BR2_PACKAGE_WPA_SUPPLICANT_CLI=y
29 | +BR2_PACKAGE_WPA_SUPPLICANT_WPA_CLIENT_S0=y
30 | +BR2_PACKAGE_WPA_SUPPLICANT_PASSPHRASE=y
31 | +BR2_PACKAGE_WPA_SUPPLICANT_CTRL_IFACE=y
32 | +# BR2_PACKAGE_WPA_SUPPLICANT_DBUS is not set
33 |  # BR2_PACKAGE_WPA_TOOLS is not set
34 |  # BR2_PACKAGE_XINETD is not set
35 |  # BR2_PACKAGE_XL2TP is not set
```

12. 如何从U盘运行系统

- 有时需要从U盘运行整个系统，建议从U盘运行系统的时候采用initramfs方式。这里先说明一下从U盘运行initramfs配置的系统
- 编译sdk的时候，用configs/hichip_hc16xx_db_XXX_initramfs_defconfig

```
1 | $ cd buildroot
2 | $ make BR2_EXTERNAL=$PWD/./ hichip_hc16xx_db_a300_initramfs_defconfig
3 | $ make all
```

- 用gdb download buildroot/output/images/u-boot，并运行uboot，进入uboot命令行
- 在uboot命令中依次输入命令，后一个命令要等前一个命令正确执行完

```

1 => mw.b 0xb880006b 0x1;
2 => mw.l 0xb8800004 0x85ff0000;
3 => usb start;
4 => fatload usb 0 0x85ff0000 dtb.bin;
5 => fatload usb 0 0x86000000 avp.uImage;
6 => bootm 0x86000000;
7 => fatload usb 0 0x86000000 vmlinux.uImage;
8 => bootm 0x86000000;

```

- 如果已经先编译了squashfs配置的sdk，后面想改成initramfs，则可以基于现有已完成编译的sdk，进行如下修改切换成initramfs的配置

```

1 $ cd buildroot
2 $ make BR2_EXTERNAL=$PWD/./ hichip_hc16xx_db_a300_initramfs_defconfig
3 $ make linux-reconfigure all

```

13. 关于4K解码与输出

- 目前，4K解码需要256MB的内存，默认configs/hichip_hc16xx_db_a300_xxx_4k_defconfig配置的DDR参数即为256MB的开发板。
- 目前，所有配置默认输出都是2k输出。而不论128MB或256MB均可以支持2k输出或4k输出。系统运行起来后，有如下几种方式切换输出制式

- 方式一：
 - 进入busybox后，直接输入命令

```
1 # echo "dis tvsys -c 0 -l 1 -d 1 -t 24 -p 1" > /dev/virtuart
```

- 如果要切换回2k制式输出，则输入命令

```
1 # echo "dis tvsys -c 0 -l 1 -d 1 -t 11 -p 1" > /dev/virtuart
```

- 方式二：
 - 进入busybox后，输入avpconsole，进行avp-core的rtos命令行界面

```

1 # avpconsole
2 D/main [01-01 00:00:00.011 t:01,0] (logger_init:54)

```

```

1 eelog initialized
2
3 I/WQUEUE [01-01 00:00:00.012 t:01,0] (work_start_highpri:151)Starting high-
  priority kernel worker thread(s)
4 I/WQUEUE [01-01 00:00:00.012 t:01,0] (work_start_rpcsvc:151)Starting rpc kernel
  worker thread(s)
5 I/WQUEUE [01-01 00:00:00.015 t:01,0] (work_start_lowpri:149)Starting low-priority
  kernel worker thread(s)
6 get_vp_init_dac_info 11
7 get_vp_init_dac_info_sd 1
8 get_vp_init_dac_info 11
9
10 =====<vdec_s3601_profile_level>API-Intro
11
12 =====<vdec_s3601_sync_mode>API-Intro
13 E/NO_TAG [01-01 00:00:00.254 t:14,0]
  (hdmi_hot_plug_in_callback:129)hdmi_hot_plug_in_callback
14 E/NO_TAG [01-01 00:00:00.281 t:14,0]
  (hdmi_edid_ready_callback:119)hdmi_edid_ready_callback
15 hc1600a@dbA300#
16 hc1600a@dbA300#

```



```
17 |
18 | ...
```

- 输入命令切换输出制式为4k输出制式

```
1 | hc1600a@dbA300# dis tvsys -c 0 -l 1 -d 1 -t 24 -p 1
```

(相应的2k输出制式命令为)

```
1 | dis tvsys -c 0 -l 1 -d 1 -t 11 -p 1
```

- 输入命令退出avpconsole, 回到busybox

```
1 | hc1600a@dbA300# quit
```

- 方式三:

- 在自己开发的应用程序中调用hcuapi的接口, 设置不同的制式输出, hcuapi的接口定义在

```
1 | SOURCE/linux-drivers/include/uapi/hcuapi/dis.h
```

- example, 上述方法一的命令就是如下example的实现。

```
1 static int tvsys_test(int argc, char *argv[])
2 {
3     struct dis_tvsys tvsys = { 0 };
4     int fd = -1;
5     long cmd = -1;
6     int opt;
7
8     opterr = 0;
9     optind = 0;
10
11     while ((opt = getopt(argc, argv, "c:l:d:t:p:")) != EOF) {
12         switch (opt) {
13             case 'c':
14                 cmd = atoi(optarg);
15                 break;
16             case 'l':
17                 tvsys.layer = atoi(optarg);
18                 break;
19             case 't':
20                 tvsys.tvtype = atoi(optarg);
21                 break;
22             case 'd':
23                 tvsys.distype = atoi(optarg);
24                 break;
25             case 'p':
26                 tvsys.progressive = atoi(optarg);
27                 break;
28             default:
29                 break;
30         }
31     }
32
33     fd = open("/dev/dis", O_WRONLY);
34     if (fd < 0) {
35         return -1;
36     }
37
38     if (tvsys.layer != DIS_LAYER_MAIN && tvsys.layer != DIS_LAYER_AUXP) {
```

```

39     printf("error layer %d\n", tvsys.layer);
40     return -1;
41 }
42
43 if (tvsys.distype != DIS_TYPE_SD && tvsys.distype != DIS_TYPE_HD &&
44     tvsys.distype != DIS_TYPE_UHD) {
45     printf("error display type %d\n", tvsys.distype);
46     return -1;
47 }
48
49 switch (cmd) {
50 case 0:
51     cmd = DIS_SET_TVSYST;
52     break;
53 case 1:
54     cmd = DIS_SET_ZOOM;
55     break;
56 default:
57     break;
58 }
59
60 ioctl(fd, cmd, &tvsys);
61
62 close(fd);
63
64 return 0;
65 }

```

- 方式四

- 如果在busybox命令行中已经启动了mplayer example测试程序，则，可以在mplayer测试程序的命令行中输入命令进行制式切换，如

```

1 # mplayer /* 启动mplayer测试程序 */
2 mp:#
3 mp:# avp dis tvsys -c 0 -l 1 -d 1 -t 24 -p 1 /* 切换为4k输出 */
4 mp:# avp dis tvsys -c 0 -l 1 -d 1 -t 11 -p 1 /* 切换为2k输出，具体参数参考
    SOURCE/linux-drivers/include/uapi/hcuapi/dis.h */

```

14. 配置系统可见内存大小以及不运行avp rtos系统

- 以下以A3200平台为例，修改系统可见内存大小，以下示例中是修改系统可见内存位64M：

```

1 diff --git a/board/hichip/hc16xx/dts/hc16xx-db-a3200-squashfs.dts
  b/board/hichip/hc16xx/dts/hc16xx-db-a3200-squashfs.dts
2 index 461550a..06d3e3f 100644
3 --- a/board/hichip/hc16xx/dts/hc16xx-db-a3200-squashfs.dts
4 +++ b/board/hichip/hc16xx/dts/hc16xx-db-a3200-squashfs.dts
5 @@ -1,5 +1,4 @@
6 #include "hc16xx-common.dtsi"
7 -#include "hc16xx-db-a3200-avp.dtsi"
8 #include "hc16xx-db-a3200.dtsi"
9
10 &chosen {
11 diff --git a/board/hichip/hc16xx/dts/hc16xx-db-a3200.dtsi
  b/board/hichip/hc16xx/dts/hc16xx-db-a3200.dtsi
12 index dcf74bb..dc816cb 100644
13 --- a/board/hichip/hc16xx/dts/hc16xx-db-a3200.dtsi
14 +++ b/board/hichip/hc16xx/dts/hc16xx-db-a3200.dtsi
15 @@ -1,6 +1,7 @@
16 #define CONFIG_AVP_MEMORY_SIZE (HCRTOS_SYMMEM_SIZE + HCRTOS_MMZ0_SIZE)
17 -#define CONFIG_LINUX_MEMORY_SIZE (CONFIG_MEMORY_SIZE - CONFIG_AVP_MEMORY_SIZE)
18 +#define CONFIG_LINUX_MEMORY_SIZE 0x04000000

```

```
19  #define CONFIG_LINUX_MEMORY_OFFSET (0)
20  +#define HCRTOS_SYSEMEM_OFFSET 0
21
22  /{
23      #address-cells = <1>;
```

运行make linux-rebuild编译

- 删除uboot中运行avp rtos系统的配置：
进入buildroot目录，运行make uboot-menuconfig

```
1  Boot options>
2      bootcmd value
```

删除下面配置

```
1  sf read ${loadaddr} avp 0x100000; bootm ${loadaddr};
```

运行make menuconfig，去掉avp选项：

```
1  External options>
2      [] avp
```

运行make uboot-rebuild编译

15、LCD显示屏配置说明

15.1 屏幕验证配置汇总

配置	RGB	LVDS	MIPI
hichip_hc16xx_db_b300_v10_defconfig	*	1024×600(VESA)1920×1080(VESA 默认)	*
hichip_hc16xx_db_b3100_v20_defconfig	800×480(RGB565 默认)	1024×600(VESA)1920×1080(VESA)	*
hichip_hc16xx_db_b3120_v20_defconfig	800×480(RGB565 默认)	1024×600(VESA)1920×1080(VESA)	*
hichip_hc16xx_db_c300_v10_defconfig		1024×600(VESA)1920×1080(VESA)	1080×1920(默认)1920×1200
hichip_hc16xx_db_c300_v20_defconfig	800×480(RGB666 默认)	*	1920×1200
hichip_hc16xx_db_c3000_v10_defconfig		1024×600(VESA 默认)	
hichip_hc16xx_db_c3100_v20_defconfig	*	*	1920×1200(默认)
hichip_hc16xx_db_c5200_v10_defconfig		1920×1080(VESA 默认)	
hichip_hc16xx_db_d3000_v10_defconfig		1024×600(VESA 默认)	
hichip_hc16xx_db_d3100_v20_defconfig	*	1024×600(VESA 默认)	*
hichip_hc16xx_db_d5200_v11_defconfig	800×480(RGB888)	1024×600(VESA 默认)1920×1080(VESA)	1920×1200

15.2 显示屏快速配置说明

```
1  1、前言说明
2  为了方便快速切换lcd显示屏，目前将所有点亮过的所有lcd显示屏都放
   在"board\hichip\hc16xx\common\dts\lcd\"的目录下了，需要配置lcd可以在这里进行参考；
3
4  2、修改默认配置下的lcd显示屏
5  例如board\hichip\hc16xx\common\dts\hc16xx-db-d3100-v10.dts默认配置的是800×480的RGB显示屏，如果
   需要改成1024×600的LVDS显示屏，可以做以下修改；
6
7  // #include "lcd_rgb_800_480_rgb565.dtsi" /*注释掉默认的配置*/
8  #include "lcd_lvds_1024_600_vesa.dtsi" /*添加新的lcd显示屏配置*/
9
10 3、编译
```

```
11 cd buildroot/
12 make linux-rebuild hcboot-kernel-rebuild hcboot-all hcboot-rebuild all
13
14 4、版本说明
15 只有在2022年12月份后的版本才能支持
```

15.3 RGB 与LVDS的配置说明

```
1  步骤一：配置
2  cd buildroot/
3  rgb可以参考配置
4  make BR2_EXTERNAL=$PWD/./ hichip_hc16xx_db_b3100_v20_defconfig
5
6  lvds可以参考配置
7  make BR2_EXTERNAL=$PWD/./ hichip_hc16xx_db_d3100_v20_defconfig
8
9  步骤二：hcboot配置说明
10 说明：
11    * 为了能够提前显示屏幕的内容，已经支持在hcboot里面show logo了。
12    * rgb与lvds相关驱动默认在hcboot里面，可以进行关闭或者打开；
13    * 如果已经在hcboot配置logo了，可以忽略步骤三，因为在linux上做了判断(is_bootlogo_showing())，可
    以忽略LVDS或者RGB的配置，前提是在hcboot打开了lvds的配置。
14    * 使用rgb需要打开lvds的配置
15  2.1 lvds的配置（默认打开）
16  cd buildroot
17  make hcboot-menuconfig
18    x There is no help available for this option.
19    x Symbol: CONFIG_DRV_LVDS [=y]
20    x Type   : bool
21    x Prompt: lvds
22    x   Location:
23    x     -> Components
24    x -> kernel (BR2_PACKAGE_KERNEL [=y])
25    x -> Drivers
26    x   Defined at drivers:155
27    x   Depends on: BR2_PACKAGE_KERNEL [=y]
28
29  2.2 bootloader show logo（默认打开）
30  x Symbol: CONFIG_BOOT_SHOWLOGO [=n]
31    x Type   : bool
32    x Prompt: Support boot show logo/av
33    x   Location:
34    x     -> Components
35    x     -> Applications Configuration
36    x   Defined at source:5x
37    x   Depends on: BR2_PACKAGE_APPS_BOOTLOADER [=y] (y=打开 n=关闭)
38
39  步骤三：linux上打开lvds的配置（默认打开）
40    * 使用rgb需要打开lvds的配置
41  cd buildroot/
42  make linux-menuconfig
43    | There is no help available for this option.
44    | Symbol: HC_LVDS [=y]
45
46    | Type   : boolean
47
48    | Prompt: lvds config driver
49
50    |   Location:
```

```

48 | -> Device Drivers
49 | -> HC drivers
50 | Defined at drivers/hcd drivers/kconfig:119
51
52 步骤四：配置DE
53 例如 在board\hichip\hc16xx\common\dts\hc16xx-db-b3100-v20.dts中，配置的是rgb565的lcd显示屏。
54 #include "lcd_rgb_800_480_rgb565.dtsi"
55 打开board\hichip\hc16xx\common\dts\lcd\lcd_rgb_800_480_rgb565.dtsi的文件
56 &DE {
57     tvtype = <15>;//一般配置为15
58     VPInitInfo {
59         rgb-cfg{
60             b-rgb-enable = <1>;//2、使能RGB设置为 1
61             /*
62              * 0: MODE_PRGB
63              * 1: MODE_SRGB
64              */
65             rgb-mode = <0>;//3、设置为并行模式
66             /*
67              * 0: MODE_PRGB_888_10bit
68              * 1: MODE_PRGB_666
69              */
70             prgb-mode = <0>; //4、设置为RGB888的模式
71             lcd-width = <800>;//5、lcd-width与h-active-len需要一致
72             b-disable-ejtag = <1>;
73             b-dlpc-enale = <0>;
74             timing-para {
75                 /* bool type, 0 or 1*/
76                 b-enable = <1>;//6、使能为，使能之后才能配置以下的参数
77                 /*
78                  * VPO_RGB_CLOCK_27M = 1,
79                  * VPO_RGB_CLOCK_33M = 2,
80                  * VPO_RGB_CLOCK_49M = 3,
81                  * VPO_RGB_CLOCK_66M = 4,
82                  * VPO_RGB_CLOCK_74M = 5,
83                  * VPO_RGB_CLOCK_85M = 6,
84                  * VPO_RGB_CLOCK_108M = 7,
85                  * VPO_RGB_CLOCK_6_6M = 8,
86                  * VPO_RGB_CLOCK_9M = 9,
87                  * VPO_RGB_CLOCK_39_6M = 10,
88                  * VPO_RGB_CLOCK_74_25M = 11,//H1600
89                  * VPO_RGB_CLOCK_148_5M = 12,//H1600
90                  */
91                 output-clock = <1>;//7、output-clock = h-total-len * v-total-len * 频率
(50~60H)
92
93                                     8、按照屏厂提供参数填写相关屏参
94             h-total-len = <1026>;//h-total-len = h-active-len + h-front-len + h-sync-
len+h-back-len
95             v-total-len = <525>;//v-total-len = v-active-len + v-front-len + v-sync-
len+v-back-len
96             h-active-len = <800>;
97             v-active-len = <480>;
98
99             h-front-len = <210>;
100             h-sync-len = <6>;
101             h-back-len = <10>;
102
103             v-front-len = <22>;
104             v-sync-len = <3>;
105             v-back-len = <20>;

```

```

106         /* bool type, 0 or 1*/
107         h-sync-level = <0>;
108         /* bool type, 0 or 1*/
109         v-sync-level = <0>;
110         frame-rate = <500000>;//9、设置帧率 50HZ
111     };
112 };
113 };
114 };
115
116
117 步骤五：模式设置为RGB
118 5.1 配置RGB
119 board\hichip\hc16xx\common\dts\lcd\lcd_rgb_800_480_rgb565.dtsi //配置rgb565 linux的节点
120 &lvs {
121     /*
122     * "lvs"      : LVDS screen
123     * "rgb888"   : RGB screen
124     * "rgb666"   : RGB screen
125     * "rgb565"   : RGB screen
126     * "i2so"     : I2S OUT
127     * "gpio"     : only GPIO out
128     */
129     lvs_ch0-type = "rgb565";//1、双通道都需要设置为rgb565
130     lvs_ch1-type = "rgb565";
131
132     /*
133     * rgb ggb bgb
134     * rrb rgb rbb
135     * rgr rgg rgb
136     */
137     rgb_src_sel = "rgb";//2、可以改变输出源，默认rgb
138     rgb-backlight-gpios = <&gpio_L 0 GPIO_ACTIVE_HIGH>;
139     rgb-backlight-gpios-rtos = <0>;// 3、gpio背光
140
141     /*
142     * 0: E_SRC_SEL_FXDE = 0
143     * 1: E_SRC_SEL_4KDE
144     * 2: E_SRC_SEL_HDMI_RX
145     */
146     src_sel = <CONFIG_DE4K_OUTPUT_SUPPORT>;
147     status = "okay";
148 };
149
150 5.2 配置lvs
151 board\hichip\hc16xx\common\dts\lcd\lcd_lvs_1024_600_vesa.dtsi //配置lvs vesa的节点
152 &lvs {
153     /*
154     * "lvs"      : LVDS screen
155     * "rgb888"   : RGB screen
156     * "rgb666"   : RGB screen
157     * "rgb565"   : RGB screen
158     * "i2so"     : I2S OUT
159     * "gpio"     : only GPIO out
160     */
161     lvs_ch0-type = "lvs";//1、设置输出的通道 LVDS D0~D3
162     lvs_ch1-type = "lvs";//          LVDS D4~D7
163
164     /*
165     * 0: E_CHANNEL_MODE_SINGLE_IN_SINGLE_OUT ,
166     * 1: E_CHANNEL_MODE_SINGLE_IN_DUAL_OUT
167     */
168     channel-mode = <0>;//2、输出

```

```

169  /*
170  * 0: E_MAP_MODE_VESA_24BIT ,
171  * 1: E_MAP_MODE_VESA_18BIT_OR_JEDIA
172  */
173  map-mode = <0>;//3、设置屏的信号格式 0:VESA 1:JEDIA
174  ch0-src_sel = <0>;
175  ch1-src_sel = <0>;
176  ch0-invert_clk_sel = <0>;
177  ch1-invert_clk_sel = <0>;
178  ch0-clk_gate = <0>;
179  ch1-clk_gate = <0>;
180  hsync-polarity = <1>;//与 de-engine 的h-sync-level 保持一致
181  vsync-polarity = <1>;//与 de-engine 的v-sync-level 保持一致
182  /*
183  * 0: E_ADJUST_MODE_FRAME_START
184  * 1: E_ADJUST_MODE_HSYNC_POS
185  * 2:E_ADJUST_MODE_VSYNC_POS
186  */
187  even-odd-adjust-mode = <0>;
188  even-odd-init-value = <0>;
189
190  /*
191  * 0: E_SRC_SEL_FXDE = 0
192  * 1: E_SRC_SEL_4KDE
193  * 2: E_SRC_SEL_HDMI_RX
194  */
195  src_sel = <CONFIG_DE4K_OUTPUT_SUPPORT>;
196  status = "okay";
197 };
198

```

199 步骤六：编译

200 6.1 修改了hcboot上的编译，重新编译

201 cd buildroot/

202 make linux-rebuild hcboot-kernel-rebuild hcboot-all hcboot-rebuild all

203 或

204 make linux-rebuild hcboot-dirclean all

205 6.2 修改了linux上的编译

206 make linux-rebuild all

207

208 步骤七：注意事项：

209 7.1 假设hclinux/board/hichip/hc16xx/dts/hc16xx-db-b300-v10-avp.dtsi配置了mipi: dsi0，请将
status = "disable"，并重新编译：

210

211 cd buildroot/

212 make linux-rebuild hcboot-kernel-rebuild hcboot-all hcboot-rebuild all

213 或

214 make linux-rebuild hcboot-dirclean all

215

216 7.2 lvds通道的说明

217 正确的配置，通道D0~D3可以配置成 "lvds","i2so","gpio",D4~D7可以配置成 "lvds","i2so","gpio"

218 例如正确的配置：

219 lvds_ch0-type = "lvds"

220 lvds_ch1-type = "gpio"

221

222 lvds_ch0-type = "i2so"

223 lvds_ch1-type = "lvds"

224

225 lvds_ch0-type = "i2so"

226 lvds_ch1-type = "i2so"

227

228 错误的配置：

229 不能够出现 rgb565 rgb666 rgb888 gpio i2so 同时组合的配置，例如：

230 lvds_ch0-type = "rgb565";//rgb666 rgb888

```

231 lvds_ch1-type = "gpio";
232
233 lvds_ch0-type = "rgb565";//rgb666 rgb888
234 lvds_ch1-type = "i2so";
235
236 lvds_ch0-type = "gpio";
237 lvds_ch1-type = "i2so";
238

```

15.4 MIPI 的配置

```

1  MIPI的相关配置只能在hcboot里面进行配置
2
3  步骤一：配置
4  cd buildroot/
5  make BR2_EXTERNAL=$PWD/./ hichip_hc16xx_db-c300_v10_defconfig
6
7  步骤二：
8      为了提前能够展示需要显示的内容，已经支持在boot里面显示一张logo图片；
9      MIPI的配置只能在hcboot里面。
10
11  2.1 mipi的配置（默认打开）
12  x There is no help available for this option.
13  x Symbol: CONFIG_DRV_MIPI [=y] (y=打开 n=关闭)
14  x Type   : bool
15  x Prompt: mipi
16  x   Location:
17  x     -> Components
18  x       -> kernel (BR2_PACKAGE_KERNEL [=y])
19  x   -> Driversio
20  x   Defined at drivers:163
21  x   Depends on: BR2_PACKAGE_KERNEL [=y]
22
23  2.2 bootloader show logo（默认打开）
24  make hcboot-menuconfig
25  x Symbol: CONFIG_BOOT_SHOWLOGO [=y] (y=打开 n=关闭)
26  x Type   : bool
27  x Prompt: Support boot show logo/av
28  x   Location:
29  x     -> Components
30  x       -> Applications Configuration
31  x   Defined at source:5x
32  x   Depends on: BR2_PACKAGE_APPS_BOOTLOADER [=y]
33
34  步骤三：配置DE
35  例如在board\hichip\hc16xx\common\dts\hc16xx-db-c300-v10.dts配置的是1080*1920的mipi显示屏
36  #include "lcd_mipi_ILI7807D_1080_1920_rgb888.dtsi"
37  打开board\hichip\hc16xx\common\dts\lcd\lcd_mipi_ILI7807D_1080_1920_rgb888.dtsi
38
39  &DE {
40      tvtype = <15>;//1、一般配置为15
41      VPIInitInfo {
42          rgb-cfg{
43              b-rgb-enable = <1>;//2、使能RGB设置为 1
44              /*
45               * 0: MODE_PRGB
46               * 1: MODE_SRGB
47               */
48              rgb-mode = <0>;//3、设置为并行模式
49              /*

```



```

50         * 0: MODE_PRGB_888_10bit
51         * 1: MODE_PRGB_666
52     */
53     prgb-mode = <0>; //4、设置为RGB888的模式
54     lcd-width = <1080>; //5、lcd-width与h-active-len需要一致
55     screen_timing: timing-para {
56         /* bool type, 0 or 1*/
57         b-enable = <1>; //6、使能为，使能之后才能配置以下的参数
58         /*
59             * VPO_RGB_CLOCK_27M = 1,
60             * VPO_RGB_CLOCK_33M = 2,
61             * VPO_RGB_CLOCK_49M = 3,
62             * VPO_RGB_CLOCK_66M = 4,
63             * VPO_RGB_CLOCK_74M = 5,
64             * VPO_RGB_CLOCK_85M = 6,
65             * VPO_RGB_CLOCK_108M = 7,
66             * VPO_RGB_CLOCK_6_6M = 8,
67             * VPO_RGB_CLOCK_9M = 9,
68             * VPO_RGB_CLOCK_39_6M = 10,
69             * VPO_RGB_CLOCK_74_25M = 11, //H1600
70             * VPO_RGB_CLOCK_148_5M = 12, //H1600
71         */
72         output-clock = <12>; //7、output-clock = h-total-len * v-total-len * 频率
(50~60H)
73                                     //8、按照屏厂提供参数填写相关屏参
74         h-total-len = <1315>; //h-total-len = h-active-len + h-front-len + h-sync-
len+h-back-len
75         v-total-len = <1980>; //v-total-len = v-active-len + v-front-len + v-sync-
len+v-back-len
76
77         h-active-len = <1080>;
78         v-active-len = <1920>;
79
80         h-front-len = <200>;
81         h-sync-len = <5>;
82         h-back-len = <30>;
83
84         v-front-len = <30>;
85         v-sync-len = <10>;
86         v-back-len = <20>;
87         /* bool type, 0 or 1*/
88         h-sync-level = <1>;
89         /* bool type, 0 or 1*/
90         v-sync-level = <1>;
91         frame-rate = <60000>; //9、设置帧率 60HZ
92     };
93 };
94 };
95 };
96
97 步骤四：配置mipi
98 打开board\hichip\hc16xx\common\dts\lcd\lcd_mipi_ILI7807D_1080_1920_rgb888.dtsi
99
100 &dsi0 {
101     reg = <0xb884A000 0x300>, <0xb8800444 0x8>, <0xb8800080 0x8>;
102     //2、配置gpio
103     enable-gpios=<11 0 1>; //设置 GPIO L11引脚，设置为GPIO输出模式，高电平起作用
104     reset-gpios=<0 0 0>; //设置 GPIO L0引脚，设置为GPIO输出模式，低电平起作用
105     //3、设置延时
106     init-delay-ms = <20>; //初始化enable-gpios reset-gpios引脚之后，输出均为低，然后延时20毫
秒，之后
107     enable-delay-ms = <120>; //将enable-gpios 引脚使能有效值（本设置拉高），然后延时120毫秒，
之后

```

```

108     reset-delay-ms = <120>;//将reset-gpios 引脚使能有效值（本设置拉低），然后延时120毫秒，然
    后将复位引脚释放（本设置拉高），之后
109     prepare-delay-ms = <120>;//延时120毫秒，开始发送命令
110
111     /*
112     * MIPI_COLOR_RGB565_0 = 0
113     * MIPI_COLOR_RGB565_1 = 1
114     * MIPI_COLOR_RGB565_2 = 2
115     * MIPI_COLOR_RGB666_0 = 3
116     * MIPI_COLOR_RGB666_1 = 4
117     * MIPI_COLOR_RGB888 = 5
118     */
119     dsi,format = <5>;//4、设置输出的格式
120
121     dsi,lanes = <4>;// 1= <dsi,lans <=4 //5、设置lane数
122
123     /*
124     * bit 6:5 MIPI_Virtual_Channel_Generic_ID
125     * bit 4 :MIPI_CRC
126     * bit 3 :MIPI_ECC
127     * bit 2 :MIPI_Bus_Turn_request.
128     * bit 1 :MIPI_EOTp
129     * bit 0 :MIPI_EOTp
130     */
131     dsi,cfg = <0x1c>;
132
133     /*
134     * 0: E_SRC_SEL_FXDE = 0
135     * 1: E_SRC_SEL_4KDE
136     * 2: E_SRC_SEL_HDMI_RX
137     */
138     src_sel = <CONFIG_DE4K_OUTPUT_SUPPORT>;
139     clock-frequency = <0>; //6、输出时钟，大于0有效；等于0，默认会自动计算
140     //一般 clock-frequency = h-total-len x v-total-len x 3 x
    8 x (frame-rate\1000)HZ / dsi,lanes
141     status = "okay";
142 };
143
144 步骤五: mipi初始化配置
145 &mipi{
146     panel-init-sequence = [
147         39 00 04 FF 78 07 01
148         15 00 02 42 11
149         .....
150         05 78 01 11
151         05 78 01 29
152     ];
153 };
154 5.1 初始化配置说明:
155 15 00 02 80 77
156 | | | | |
157 | | | | 数据
158 | | | 数据
159 | | 数据长度
160 | 延时
161 命令类型（0x05: 单字节数据 0x15: 双字节数据 0x39: 多字节数据）
162
163 5.2 单字节数据举例:
164 05 78 01 29
165
166 5.3 双字节数据举例:
167 15 00 02 42 11
168

```

```
169 5.4 多字节数据举例：
170 39 00 04 FF 78 07 01
171
172
173 步骤六：重新编译
174 cd buildroot/
175 make linux-rebuild hcboot-kernel-rebuild hcboot-all hcboot-rebuild all
176 或
177 make linux-rebuild hcboot-dirclean all
178
179 步骤七：注意事项：
180 6.1 假设hclinux/board/hichip/hc16xx/dts/hc16xx-db-c300-v10.dts配置了lvds，可将status =
    "disable"(这只是关闭lvds的功能，C300芯片可以实现LVDS与MIPI双屏显示，但是需要显示屏的相关屏参是一致才能
    让显示效果一致)，重新编译
181 cd buildroot/
182 make linux-rebuild hcboot-kernel-rebuild hcboot-all hcboot-rebuild all
183 或
184 make linux-rebuild hcboot-dirclean all
185
186
```

16. 快速开发指南

16.1 SDK 配置、编译和打包

16.1.1 sdk编译和打包

一个`sdk`要编译多个方案板，可以在编译时增加制定输出目录

编译sdk：

```
1 cd buildroot
2 make O=output/d3100 BR2_EXTERNAL=$PWD/./ hichip_hc16xx_db_d3100_v20_defconfig //选择方案配置
3 make O=output/d3100 all //sdk编译和打
  包
4
5 备注： output/d3100是目标输出目录
```

编译单个package：

```
1 以投影仪主应用为例：
2 部分编译：
3 cd output/d3100 //进入方案编译目录
4 make hcprojectorapp-rebuild all
5
6 完整的重新编译：
7 cd output/d3100 //进入方案编译目录
8 make hcprojectorapp-dirclean hcprojectorapp-rebuild all
9
10 代码修改较少，建议使用部分编译。代码修改较多，建议完整编译。
```

方案的编译命令请都在`output/d3100`路径下执行

16.1.2 常用命令

快捷命令：

```
1 | cd hclinux
2 | source support/scripts/envsetup.sh //sdk顶层目录执行
3 | cd buildroot/output/d3100 //进入方案编译目录
```

在方案编译目录，可以执行如下命令：

```
javen.xu@hichip01:hclinux$ source support/scripts/envsetup.sh
HC_TOP_DIR=/data/home/javen.xu/workspace/linux/hclinux/hclinux
command list:
  croot : jump to the hclinux directory
  ckernel : jump to the kernel directory
  cboot : jump to the bootloader directory
  cdtls : jump to the dts directory
  cbr : jump to the buildroot directory
  cout : jump to the buildroot/output directory
  cdriver : jump to the SOURCE/linux-drivers
  csource : jump to the SOURCE
  cavp : jump to the SOURCE/avp
  cavp_kernel : jump to the SOURCE/avp/components/kernel/source

  mkboot : compile hcboot and packaging firmware, eg: hcboot kernel/bootloader/dts
  mkkernel : compile linux kernel and packaging firmware, eg: kernel/dts
  mkavp : compile avp and packaging firmware, eg: avp kernel/cmds
  mkhclinux : compile hcboot/avp/kernel/dts and packaging firmware
  mkall : rebuild all hichip package and packaging firmware, eg: hcboot/avp/kernel/lib/app
  mkall-clean: clean and rebuild all hichip package and packaging firmware
```

类别	动作行为	命令	详细说明
配置	方案配置	make menuconfig	buildroot配置
	hcboot	make hcboot-menuconfig	bootloader配置
	linux kernel	make linux-menuconfig	linux kernel配置
	avp	make avp-menuconfig	avp配置
	lvgl	make lvgl-menuconfig	lvgl配置
	busybox	make busybox-menuconfig	配置busybox
		make busybox-update-config	保存busybox配置
整体编译	编译sdk	make all	编译sdk，并且打包固件
	修改dts	mkhclinux	编译dts相关的hcboot和linux kernel，并且打包固件
	单独编译bootloader	mkboot	单独编译bootloader，并且打包固件
	单独编译linux kernel	mkkernel	单独编译linux kernel，并且打包固件
	单独编译avp	mkavp	单独编译avp，并且打包固件
	编译所有海奇软件包	mkall	编译除编译工具链以外的软件包

类别	动作行为	命令	详细说明
	清除并且编译海奇软件包	mkall-clean	删除并且编译除编译工具链以外的软件包
单个软件包	查看依赖项	make -show-depends	显示构建程序包所需的依赖项，必须与hclinux/package目录中的文件夹名字保持一致
	删除编译的软件包	make -dirclean	删除output/build中的package软件包
	重新编译软件包	make -rebuild	重新编译package软件包
	编译同屏器app	make hcscreenapp-rebuild	重新编译hcscreenapp
		make hcscreenapp-dirclean make hcscreenapp-rebuild	删除output/build中的hcscreenapp目录，并且重新编译hcscreenapp
	编译投影仪app	make hcprojectorapp-rebuild	删除output/build中的hcprojectorapp目录，并且重新编译hcprojectorapp
		make hcprojectorapp-dirclean make hcprojectorapp-rebuild	删除output/build中的hcprojectorapp目录，并且重新编译hcprojectorapp
	编译lvgl	make liblvgl-rebuild	重新编译liblvgl

备注: *package*为*buildroot/package*目录下软件包名或*hclinux/package*目录下软件包名

更多命令可以参考《buidroot_user_manual.pdf》中《8.12.5 Package-specific make targets》

官网链接为: <http://buildroot.org/downloads/manual/manual.html>

快捷跳转命令

命令	说明	备注
croot	跳转到sdk顶层目录，即hclinux目录	
ckernel	跳转到linux kernel代码目录 即output\xxx\build\linux-4.4.186	
cboot	跳转到hcboot的apps-bootloader代码目录	
cdts	跳转到dts所在目录	
cbr	跳转到buildroot所在目录	
cout	跳转到buildroot/output目录	
cdriver	跳转到SOURCE/linux-drivers目录	
csource	跳转到SOURCE目录	

命令	说明	备注
cavp	跳转到SOURCE/avp目录	
cavp_kernel	跳转到SOURCE/avp/components/kernel/source目录	

16.1.3 sdk配置

16.1.3.1 SDK涉及的配置

以D3100方案为例，涉及的配置有：

1	.	
2	— board	
3	— hichip	
4	— hc16xx	
5	— ddrinit	
6	— aux_code_ddr2_128M_1066MHz.abs	//linux系统ddr配置文件
7	— gdb	
8	— H1600_0001_ddr2_clk800MHz_128M_20200923.abs	//gdb ddr配置文件
9		
10	— dts	
11	— hc16xx-db-d3100-v20-avp.dtsi	//avp dts配置
12	— hc16xx-db-d3100-v20.dts	//linux kernel dts配置
13		
14	— kernel-configs	
15	— 4.4.186	
16	— kernel-squashfs.config	//kernel config配置
17	— configs	
18	— hichip_hc16xx_db_d3100_v20_defconfig	//buildroot config配置
19		
20	— SOURCE	
21	— avp	
22	— configs	
23	— hichip_hc16xx_linux_avp_defconfig	//avp config配置
24	— hichip_hc16xx_linux_b1_defconfig	//hcboot config配置

hichip_hc16xx_db_d3100_v20_defconfig作为总的配置入口，记录了kernel、busybox、hcboot、avp、dts、ddr配置文件的位置，如果这些配置名称有修改，请务必更新hichip_hc16xx_db_d3100_v20_defconfig文件。

1	//buildroot配置文件位置
2	BR2_DEFCONFIG="\$(BR2_EXTERNAL_HCLINUX_PATH)/configs/hichip_hc16xx_db_d3100_v20_defconfig"
3	
4	//linux kernel配置文件位置
5	BR2_LINUX_KERNEL_CUSTOM_CONFIG_FILE="\$(BR2_EXTERNAL_HCLINUX_PATH)/board/hichip/hc16xx/kernel-configs/\$(LINUX_VERSION)/kernel-squashfs.config"
6	
7	//busybox配置文件位置
8	BR2_PACKAGE_BUSYBOX_CONFIG="\$(BR2_EXTERNAL_HCLINUX_PATH)/board/hichip/hc16xx/busybox-configs/busybox.config"
9	
10	//hcboot配置文件位置
11	BR2_TARGET_HCBOOT_DEFCONFIG="hichip_hc16xx_linux_b1_defconfig"
12	
13	//avp配置文件位置
14	BR2_PACKAGE_AVP_DEFCONFIG="hichip_hc16xx_linux_avp_defconfig"
15	
16	//ddr配置文件位置

```

17 BR2_EXTERNAL_BOARD_DDRINIT_FILE="${BR2_EXTERNAL_HCLINUX_PATH}/board/hichip/hc16xx/ddrinit/
aux_code_ddr2_128M_800MHz.abs"
18
19 //dts配置文件位置
20 BR2_LINUX_KERNEL_CUSTOM_DTS_PATH="$(BR2_EXTERNAL_HCLINUX_PATH)/board/hichip/hc16xx/dts/hc1
6xx-db-d3100-v20.dtsi $(BR2_EXTERNAL_HCLINUX_PATH)/board/hichip/hc16xx/dts/hc16xx-
common.dtsi $(BR2_EXTERNAL_HCLINUX_PATH)/board/hichip/hc16xx/dts/hc16xx-db-d3100-v20-
avp.dtsi"

```

16.1.3.2 修改buildroot配置

参考《7. Buildroot》

```

1 修改配置：
2 # cd buildroot/output/d3100
3 # make menuconfig          //进入配置界面
4
5 保存配置：
6 # cp output/d3100/.config ../../../../configs/hichip_hc16xx_db_d3100_v20_defconfig
7
8 修改后的编译：（建议重新编译整个buildroot）
9 # rm -rf output
10 # make BR2_EXTERNAL=$PWD/../../hichip_hc16xx_db_d3100_v20_defconfig
11 # make all

```

16.1.3.3 修改kernel配置

参考《8.2.3 linux kernel的配置》

```

1 修改配置：
2 # cd buildroot/output/d3100
3 # make linux-menuconfig    //进入配置界面
4 （如果新增模块，请在hc16xx-db-d3100-v20.dtsi文件中确认模块是否enable，一般每个模块通过 status =
"disabled"或者 status = "okay"来使能或非使能。没有使能的驱动就算编译了也不会初始化）
5
6 保存配置：
7 # cp output/d3100/build/linux-4.4.186/.config ../../../../board/hichip/hc16xx/kernel-
configs/4.4.186/kernel-squashfs.config
8
9 修改配置后的编译：
10 # make linux-rebuild all
11 或者
12 #mkall

```

如果是特殊方案，为不影响现有的方案而不想修改kernel-squashfs.config，可以将kernel config保存为自定义名称，方法如下：

```

1 保存配置：
2 #cp output/d3100/build/linux-4.4.186/.config ../../../../board/hichip/hc16xx/kernel-
configs/4.4.186/d3100_v20.config
3
4
5 修改buildroot配置文件让其生效：
6 #vi ../../../../configs/hichip_hc16xx_db_d3100_v20_defconfig
7 BR2_LINUX_KERNEL_CUSTOM_CONFIG_FILE="$(BR2_EXTERNAL_HCLINUX_PATH)/board/hichip/hc16xx/kerne
l-configs/${(LINUX_VERSION)}/d3100_v20.config"

```

16.1.3.4 修改hcboot配置

```
1  修改配置：
2  # buildroot/output/d3100
3  # make hcboot-menuconfig          //进入配置界面
4  (如果新增模块，请在hc16xx-db-d3100-v20-avp.dtsi文件中确认模块是否enable)
5
6  保存配置：
7  # cp output/d3100/build/hcboot-custom/output/.config
  ../../../SOURCE/avp/configs/hichip_hc16xx_linux_b1_defconfig
8
9  修改配置后的编译：
10 # make hcboot-kernel-rebuild hcboot-all hcboot-rebuild all
11 或者
12 #mkall
13
14 如果修改地方比较多，建议清除代码重新编译，命令如下：
15 # make hcboot-dirclean hcboot-rebuild all
```

16.1.3.5 修改avp配置

```
1  修改配置：
2  # cd buildroot
3  # make avp-menuconfig            //进入配置界面
4
5  保存配置：
6  # cp output/d3100/build/avp-custom/output/.config
  ../../../SOURCE/avp/configs/hichip_hc16xx_linux_avp_defconfig
7
8  修改配置后的编译：
9  # make avp-kernel-rebuild avp-all avp-rebuild all
10 如果修改的配置影响的是其他的模块，比如output/build/avp-custom/components/cmds，则需要
11 # make avp-kernel-rebuild avp-cmds-rebuild avp-all avp-rebuild all
12 或者
13 #mkall
14
15 如果修改地方比较多，建议清除代码重新编译，命令如下：
16 # make avp-dirclean avp-rebuild all
```

16.2 如何增加一个新方案

下面以d3100_v20 demo板为例

1.涉及的目录和文件

```
1  .
2  |— board
3  |   |— hichip
4  |       |— hc16xx
5  |           |— ddrinit
6  |               |— aux_code_ddr2_128M_1066MHz.abs          //linux系统ddr配置文件
7  |               |   |— gdb
8  |               |       |— H1600_0001_ddr2_c1k800MHz_128M_20200923.abs //gdb ddr配置文件
9  |               |
10 |               |— dts
```


11				└─ hc16xx-db-d3100-v20-avp.dtsi	//hcboot avp dts配置
12				└─ hc16xx-db-d3100-v20.dts	//linux kernel dts配置
13					
14				└─ kernel-configs	
15				└─ 4.4.186	
16				└─ kernel-squashfs.config	//kernel config配置
17				└─ configs	
18				└─ hichip_hc16xx_db_d3100_v20_defconfig	//buildroot config配置
19					
20				└─ SOURCE	
21				└─ avp	
22				└─ configs	
23				└─ hichip_hc16xx_linux_avp_defconfig	//avp config配置
24				└─ hichip_hc16xx_linux_bl_defconfig	//hcboot config配置

2.增加一个新方案

第1步：配置ddr

- (1). 如果ddrinit目录中没有相关ddr配置，请联系我们。
 - (2). 修改configs/hichip_hc16xx_db_d3100_v20_defconfig文件，配置ddr
- BR2_EXTERNAL_BOARD_DDRINIT_FILE="\${BR2_EXTERNAL_HCLINUX_PATH}/board/hichip/hc16xx/ddrinit/aux_code_ddr2_128M_800MHz.abs"

第2步：配置hcboot

通过配置dts来实现，不建议通过hcboot config来实现。

第3步：配置kernel

- (1). 通过配置dts来实现
- (2). 如果要新增模块，请配置kernel config，方法见《16.1.2 sdk配置》。

第4步：配置buildroot

检查hichip_hc16xx_db_d3100_v20_defconfig中，kernel、busybox、hcboot、avp、dts、ddr配置文件的位置是否正常。

经过上述4步，新方案增加完成。

16.3 如何新增一个软件包

下面以新增getevent软件包为例

1.涉及的目录和文件

1				└─ .	
2				└─ Config.in	//buildroot config配置规则
3				└─ configs	
4				└─ hichip_hc16xx_db_d3100_v20_defconfig	//buildroot配置文件
5				└─ local.mk	//buildroot编译规则
6				└─ package	

```

7 |   └─ getevent                               //getevent编译规则
8 |       └─ Config.in
9 |       └─ getevent.mk
10 └─ SOURCE
11 |   └─ getevent                               //getevent源码
12 |       └─ getevent.c
13 |       └─ getevent.h
14 |       └─ Makefile

```

2.增加一个软件包

第1步：移植源码

将getevent放到SOURCE目录

第2步：编写getevent编译规则

(1). 在package目录新建getevent

(2). 编写Config.in

```

1 config BR2_PACKAGE_GETEVENT
2     bool "getevent"
3     help
4         getevent package

```

(3). 编写getevent.mk

```

1 #####
2 #
3
4 # getevent package
5
6 #
7 #####
8 GETEVENT_SITE_METHOD = UNDEFINED
9 GETEVENT_SITE = UNDEFINED
10 GETEVENT_LICENSE = GPLv2+, GPLv3+
11 GETEVENT_LICENSE_FILES = Copyright COPYING
12 GETEVENT_DEPENDENCIES =                               //依赖的软件包
13 GETEVENT_CFLAGS = $(TARGET_CFLAGS)
14 GETEVENT_CFLAGS += -O2 -fPIC -Wall -DCOMPLETE_TIMEOUT //COMPLETE_TIMEOUT宏定义开关
15 GETEVENT_CFLAGS += -I$(STAGING_DIR)/usr/include -I$(@D) //编译时依赖的头文件
16 GETEVENT_LDFLAGS = $(TARGET_LDFLAGS)
17 GETEVENT_LDFLAGS += -L$(TARGET_DIR)/usr/lib/           //编译依赖的库文件
18
19 define GETEVENT_BUILD_CMDS
20     $(MAKE) CC="$(TARGET_CC)" CFLAGS="$(GETEVENT_CFLAGS)" \
21         LDFLAGS="$(GETEVENT_LDFLAGS)" -C $(@D) all
22 endef
23
24 define GETEVENT_INSTALL_TARGET_CMDS
25     $(INSTALL) -D -m 0755 $(@D)/getevent $(TARGET_DIR)/usr/bin
26 endef
27
28 $(eval $(generic-package))

```

第3步：加入到buildroot编译系统中

(1). 加入到Config.in

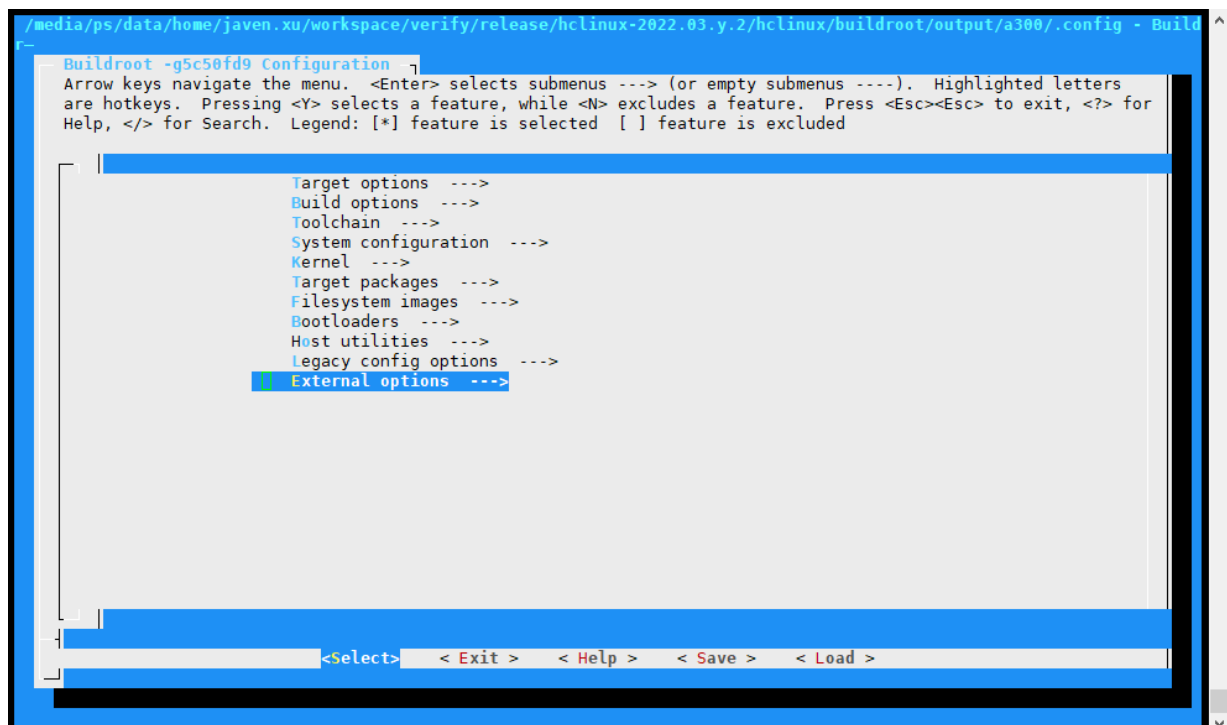
```
1 | source "$BR2_EXTERNAL_HCLINUX_PATH/package/getevent/Config.in"
```

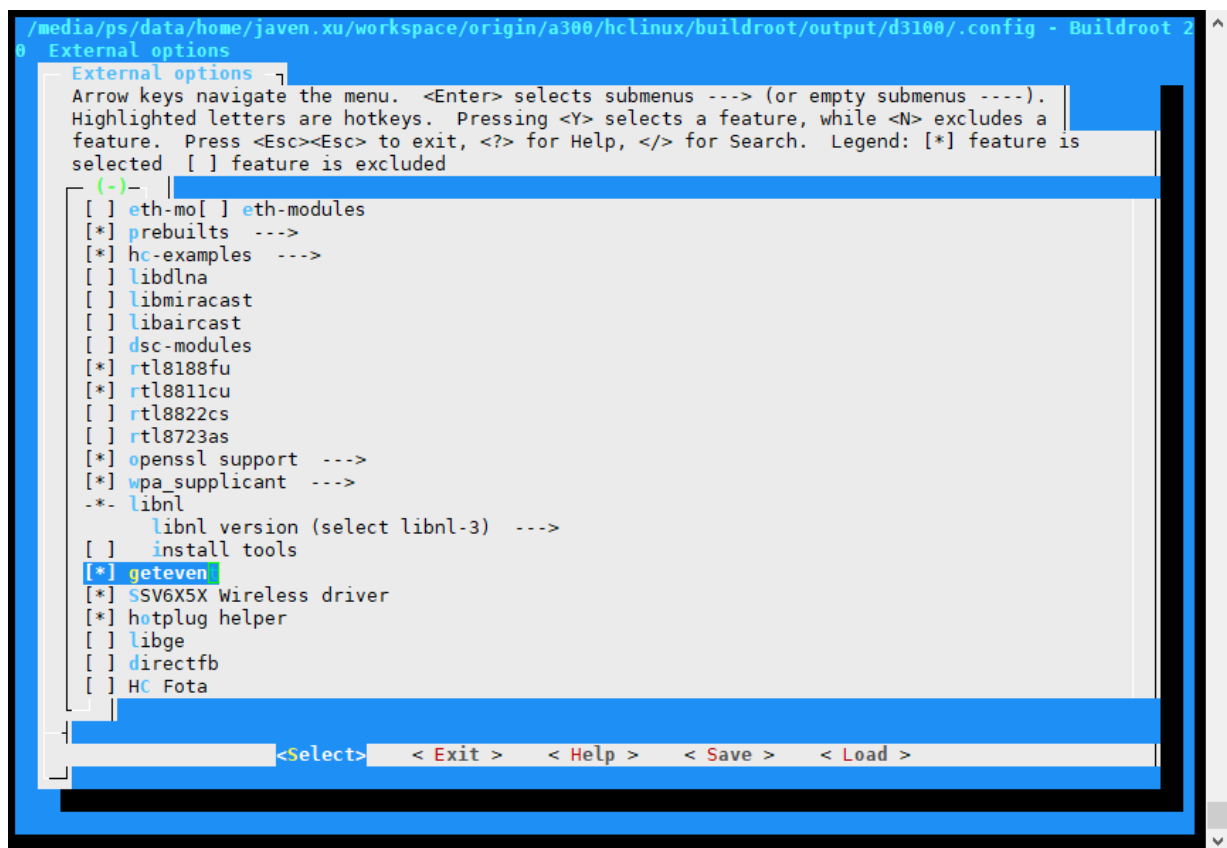
(2). 加入到local.mk

```
1 | GETEVENT_OVERRIDE_SRCDIR=$(BR2_EXTERNAL_HCLINUX_PATH)/SOURCE/getevent
```

第4步：配置buildroot config

```
1 | # make menuconfig
2 | 选择External options ---> getevent
3 | # make savedefconfig //保存配置
```





第5步：编译打包

```
1 # make getevent-rebuild all
```

至此，新软件包增加完成。

16.4 buildroot config的宏定义如何在package代码中生效

buildroot config中的宏定义在package代码中是不生效的，需要做一些处理，下面讲解一下如何处理。

下面以getevent为例：

第1步：在buildroot config中增加宏定义

修改package/getevent/Config.in文件，增加宏定义

```
1 config BR2_MACRO_DEFINITION_TEST
2     bool "Macro definition test"
3     depends on BR2_PACKAGE_GETEVENT
4     help
5         getevent package
```

第2步：将宏定义在编译时传递给package代码

修改package/getevent/getevent.mk文件

```

1 | #ifeq ($(BR2_MACRO_DEFINITION_TEST),y)
2 | GETEVENT_CFLAGS += -DMACRO_DEFINITION_TEST
3 | #endif

```

第3步：宏定义在package代码中生效

修改SOURCE/getevent/getevent.c文件，

```

1 | int main(int argc, char *argv[])
2 | {
3 |     .....
4 |
5 | #ifdef MACRO_DEFINITION_TEST
6 |     printf("\nMacro definition test\n\n");
7 | #endif
8 |
9 |     .....
10 | }

```

测试验证：

```

# ./getevent -lr
add device 1: /dev/input/event0
  name:      "HC Remote Control Receiver"
poll 2, returned 1
/dev/input/event0: EV_MSC      MSC_SCAN      0000001a
poll 2, returned 1
/dev/input/event0: EV_KEY      KEY_UP      DOWN
poll 2, returned 1
/dev/input/event0: EV_SYN      SYN_REPORT  00000000
poll 2, returned 1
/dev/input/event0: EV_KEY      KEY_UP      UP
poll 2, returned 1
/dev/input/event0: EV_SYN      SYN_REPORT  00000000      rate 4
poll 2, returned 1
/dev/input/event0: EV_MSC      MSC_SCAN      00000048
poll 2, returned 1
/dev/input/event0: EV_KEY      KEY_DOWN     DOWN
poll 2, returned 1
/dev/input/event0: EV_SYN      SYN_REPORT  00000000      rate 0
poll 2, returned 1
/dev/input/event0: EV_MSC      MSC_SCAN      00000048
poll 2, returned 1
/dev/input/event0: EV_SYN      SYN_REPORT  00000000      rate 21
poll 2, returned 1
/dev/input/event0: EV_MSC      MSC_SCAN      00000048
poll 2, returned 1
/dev/input/event0: EV_SYN      SYN_REPORT  00000000      rate 183
poll 2, returned 1
/dev/input/event0: EV_KEY      KEY_DOWN     UP
poll 2, returned 1
/dev/input/event0: EV_SYN      SYN_REPORT  00000000      rate 4

```

16.5 分区操作

16.5.1 如何调整分区大小

下面以d3100方案为例

1.分区涉及的文件

```

1  .
2  └─ board
3      └─ hichip
4          └─ hc16xx
5              |
6              └─ dts
7                  └─ hc16xx-db-d3100-v20-avp.dtsi //hcboot分区划分，必须与linux 分区一致
8                  └─ hc16xx-db-d3100-v20.dts      //linux 分区划分
9                  └─ post-build.sh                //固件打包

```

2.分区划分说明

1.hcboot分区划分

hc16xx-db-d3100-v20-avp.dtsi

```

1  spi_nor_flash {
2      spi-tx-bus-width = <1>;
3      spi-rx-bus-width = <1>;
4      reg = <0>;
5      status = "okay";
6      partitions {
7          part-num = <7>;                //分区的个数
8
9          /* part0 is for entire norflash by default */
10
11         part1-label = "boot";           //分区名
12         part1-reg = <0x0 0x80000>;      //boot分区从0x00地址开始，大小为0x80000
13
14         part2-label = "dtb";           //分区名
15         part2-reg = <0x80000 0x10000>;  //dtb分区从大小为0x80000地址开始，大小为0x10000
16                                         //dtb起始地址 = boot分区起始地址 + boot分区大小
17                                         //即：0x80000 = 0x0 + 0x80000
18
19         part3-label = "avp";
20         part3-reg = <0x90000 0x120000>;
21
22         part4-label = "linux";
23         part4-reg = <0x1b0000 0x400000>;
24
25         part5-label = "rootfs";
26         part5-reg = <0x5b0000 0xa00000>;
27
28         part6-label = "eromfs";
29         part6-reg = <0xfb0000 0x300000>;
30
31         part7-label = "persistentmem";
32         part7-reg = <0xfe0000 0x20000>; //最好一个分区的结束地址不能大于flash总大小
33                                         //0xfe0000 + 0x20000 = 0x1000000 (16m)
34     };
35 };

```

2.linux 分区划分

hc16xx-db-d3100-v20.dts

```

1  &spi_flash {
2      spi-tx-bus-width = <1>;
3      spi-rx-bus-width = <1>;
4
5      partitions {
6          compatible = "fixed-partitions";

```

```

7      #address-cells = <1>;
8      #size-cells = <1>;
9
10     nor@0 {                                //说明flash的总大小
11                                           //nor@0 : 分区名@起始地址
12         label = "nor";
13         reg = <0x0 0x1000000>;           //flash起始地址和结束地址
14                                           //nor flash的总大小为16M
15     };
16
17     boot@0 {                                //dtb分区从0x00地址开始
18         label = "boot";                  //分区名
19         reg = <0x0 0x80000>;             //起始地址: 0x00, 分区大小: 0x80000
20     };
21
22     dtb@80000 {                             //dtb分区从0x80000地址开始
23                                           //dtb起始地址 = boot起始地址 + boot分区大小
24         label = "dtb";                  //分区名
25         reg = <0x80000 0x10000>;         //起始地址: 0x80000, 分区大小: 0x10000
26     };
27
28     avp@90000 {
29         label = "avp";
30         reg = <0x90000 0x120000>;
31     };
32
33     linux@1b0000 {
34         label = "linux";
35         reg = <0x1b0000 0x400000>;
36     };
37
38     rootfs@5b0000 {
39         label = "rootfs";
40         reg = <0x5b0000 0xa00000>;
41     };
42
43     eromfs@f90000 {
44         label = "eromfs";
45         reg = <0xfb0000 0x30000>;
46     };
47
48     persistentmem@fe0000 {
49         label = "persistentmem";
50         reg = <0xfe0000 0x20000>;         //分区的结束地址 不能超过 nor@0中的总大小
51     };
52 };
53
54 };

```

3.如何调整分区大小

以调整kernel 分区大小为例，将kernel分区大小从4M调整为3M。

第1步：修改hcboot dts中的分区表

修改hc16xx-db-d3100-v20-avp.dtsi文件，调整分区表

```

1      spi_nor_flash {
2          spi-tx-bus-width = <1>;
3          spi-rx-bus-width = <1>;

```

```

4         reg = <0>;
5         status = "okay";
6         partitions {
7             part-num = <7>;
8
9             /* part0 is for entire norflash by default */
10
11            part1-label = "boot";
12            part1-reg = <0x0 0x80000>;
13
14            part2-label = "dtb";
15            part2-reg = <0x80000 0x10000>;
16
17            part3-label = "avp";
18            part3-reg = <0x90000 0x120000>;
19
20            part4-label = "linux";                //linux分区之前的分区不做调整
21            part4-reg = <0x1b0000+ 0x300000>;    //起始地址不变，只调整分区大小，4M调整为3M
22
23            part5-label = "rootfs";
24            part5-reg = <0x4b0000 0xa00000>;    //起始地址要调整，起始地址=前分区起始地址+前分区大小
25                                                //0x1b0000+0x300000=0x4b0000
26            part6-label = "eromfs";
27            part6-reg = <0xeb0000 0x30000>;    //起始地址要调整，起始地址=前分区起始地址+前分区大小
28
29            part7-label = "persistentmem";
30            part7-reg = <0xee0000 0x20000>;    //起始地址要调整，起始地址=前分区起始地址+前分区大小
31                                                //剩余空间还有1M未分配，可以划给其他分区，
32                                                //也可以保留不使用
33        };
34    };
35

```

第2步：修改kernel dts中的分区表

修改hc16xx-db-d3100-v20.dtsi文件，调整分区表

```

1  &spi_flash {
2      spi-tx-bus-width = <1>;
3      spi-rx-bus-width = <1>;
4
5      partitions {
6          compatible = "fixed-partitions";
7          #address-cells = <1>;
8          #size-cells = <1>;
9
10         nor@0 {
11             label = "nor";
12             reg = <0x0 0x1000000>;
13         };
14
15         boot@0 {
16             label = "boot";
17             reg = <0x0 0x80000>;
18         };
19
20         dtb@80000 {
21             label = "dtb";
22             reg = <0x80000 0x10000>;
23         };
24

```



```

25     avp@90000 {
26         label = "avp";
27         reg = <0x90000 0x120000>;
28     };
29
30     linux@1b0000 {                                //linux分区之前的分区不做调整,起始地址不变
31         label = "linux";
32         reg = <0x1b0000 0x300000>;                //起始地址不变, 只调整分区大小, 4M调整为3M
33     };
34
35     rootfs@4b0000 {                                //起始地址要调整, 起始地址=前分区起始地址+前分区大小
36                                                     //0x1b0000+0x300000=0x4b0000
37         label = "rootfs";
38         reg = <0x4b0000 0xa00000>;                //起始地址要调整, 起始地址=前分区起始地址+前分区大小
39     };
40
41     eromfs@e90000 {                                //起始地址要调整, 起始地址=前分区起始地址+前分区大小
42         label = "eromfs";
43         reg = <0xeb0000 0x30000>;                //起始地址要调整, 起始地址=前分区起始地址+前分区大小
44     };
45
46     persistentmem@ee0000 {                          //起始地址要调整, 起始地址=前分区起始地址+前分区大小
47         label = "persistentmem";
48         reg = <0xee0000 0x20000>;                //起始地址要调整, 起始地址=前分区起始地址+前分区大小
49                                                     //剩余空间还有1M未分配, 可以划给其他分区,
50                                                     //也可以保留不使用
51     };
52 };
53 };

```

第3步: 重新编译和打包

```

1 # make hcboot-kernel-rebuild hcboot-all hcboot-rebuild //重新编译hcboot
2 # make linux-rebuild                                //重新编译linux kernel
3 # make all                                           //整体编译和打包

```

或者

```

1 # make hcboot-dirclean hcboot-rebuild linux-rebuild all

```

如果有修改xxx-avp.dtsi里面的内存mapping, 则还需要重新编译avp, 因为avp和hcboot的入口地址是动态生成的, 当内存mapping发生变化的时候, 它们的入口地址也会发生变化。下面的命令会把hcboot和avp的编译临时文件夹都删除。如果有通过make hcboot-menuconfig或make avp-menuconfig修改hcboot/avp的配置的话, 在执行下面命令时先保存一下。

```

1 # make hcboot-dirclean avp-dirclean linux-rebuild all

```

第4步: 验证分区表是否修改成功

烧写固件, 进入到命令行, 执行#cat /proc/partitions查看

```
1 # cat /proc/partitions
2 major minor #blocks name
3
4 31 0 16384 mtdblock0 //nor@0
5 31 1 512 mtdblock1 //boot@0
6 31 2 64 mtdblock2 //dtb@80000
7 31 3 1152 mtdblock3 //avp@90000
8 31 4 3072 mtdblock4 //linux@1b0000
9 31 5 10240 mtdblock5 //rootfs@4b0000
10 31 6 192 mtdblock6 //eromfs@eb0000
11 31 7 128 mtdblock7 //persistentmem@ee0000
```

4.如何新增分区

- (1). 新增分区步骤与修改分区大小的步骤和方法一样
- (2). 注意，新增分区不能超过flash总大小

16.5.2 如何增加用户数据分区

目前只有d3100_p1增加了用户数据分区支持，其他方案可以参考d3100_p1修改。

1.d3100_p1用户数据分区介绍

分区名：data

大小：112k

文件系统格式：jffs2

mount目录：/mnt/data

特点：

- 1.支持编译时，将用户数据拷贝到output/images/data目录。固件打包时，将data目录生成jffs2格式文件一起打包到固件中。
- 2.系统启动时，会自动将其挂载到/mnt/data目录
- 3.app可以直接对/mnt/data目录进行文件读写

2.分区涉及的文件

```
1 .
2 |— board
3 |   |— hichip
4 |       |— hc16xx
5 |           |
6 |               |— d3100_p1
7 |                   |— dts
8 |                       |— hc16xx-db-d3100-v20-avp.dtsi //hcboot分区划分，必须与linux 分区一致
9 |                       |— hc16xx-db-d3100-v20.dts //linux 分区划分
10 |                   |— kernel-configs //kernel增加jffs2文件系统支持
11 |                   |— rootfs_overlay //d3100 rootfs启动相关
12 |                       //与common中的rootfs_overlay的区别：
13 |                       //1.common公共配置，d3100是方案配置
14 |                       //2.编译时，d3100会覆盖common配置
15 |                   |— logo.m2v //开机logo
16 |                   |— post-image.sh //打包生成data.img
```

```
17 | — configs
18 |   — hichip_hc16xx_db_d3100_v20_p1_defconfig //增加rootfs overlay配置和post-build配置
```

3.使用方法

系统启动以后直接对/mnt/data进行文件读写即可。

16.5.3 如何增加新分区

新增一个分区要做3件事：

- 1 1.修改dts，加入新分区
- 2 2.新分区的格式化
- 3 3.挂载新分区

1.修改dts，加入新分区

参考《16.5.1 如何调整分区大小》

2.新分区的格式化

(1).什么时候格式化新分区？

- PC上格式化，如果新分区要预装数据，那么需要在PC格式化，然后随着固件一起写到flash中。
- 系统启动时格式化，如果不需要预装数据，那么在系统启动时将分区格式化。

(2).系统启动时格式化新分区

参考board\hichip\hc16xx\rootfs_overlay\rootfs\etc\preinit。

(3).PC上格式化新分区

参考board\hichip\hc16xx\post-build.sh中littlefs文件系统镜像的制作。

PC上文件系统镜像制作工具有：

-----fs-----	-----description-----	-----usage-----
mklittlefs	制作带有 mklittlefs文件系统的镜像	将数据拷贝到目录中，然后将目录生成镜像
mke2img	制作带有 ext2文件系统的镜像	将数据拷贝到目录中，然后将目录生成镜像
mkfs.jffs2	制作带有 jffs2文件系统的镜像	将数据拷贝到目录中，然后将目录生成镜像
mkfs.vfat	格式化成fat文件系统	先生成镜像，再文件系统格式化，然后拷贝数据到镜像中

采用何种文件系统，取决于分区大小：

小于1M空间，推荐使用mklittlefs、mkfs.jffs2

大于1M空间，推荐使用mke2img、mkfs.vfat

(3-1). mklittlefs 用法

```
mklittlefs -c data/ -d 0 -b 4096 -p 256 -s 0x20000 data.img
```

- -c : 数据源目录
- -b: block size
- -p: page size
- -s: 分区大小
- data.img : 生成的镜像名

(3-2). mkfs.vfat用法

//步骤1：生成data.img镜像

```
#dd if=/dev/zero of=${BINARIES_DIR}/data.img bs=1k count=128
```

//步骤2：格式化文件系统

```
#mkfs.vfat -F 16 ${BINARIES_DIR}/data.img
```

//步骤3：将data目录中文件拷贝到data.img中

```
#mcopy -si BINARIES_DIR/data.img{BINARIES_DIR}/data/* ::/
```

其他文件系统镜像的制作请自行在网上查找。

3.挂载新分区

参考board\hchip\hc16xx\rootfs_overlay\rootfs\etc\preinit

16.5.4 如何将用户数据预装到固件中

下面以hccast_demo为例

1.将文件预装到固件中

修改package\lvgl.mk文件，在xxx_INSTALL_TARGET_CMDS中，将文件拷贝到\${DATA_DIR}即可。

2.开机后如何访问

data分区会自动挂载到/mnt/data目录，访问/mnt/data目录即可。

```

24
25 define LVGL_INSTALL_TARGET_CMDS
26 if [ "$BR2_PACKAGE_LVGL_DEMO" = "y" ];then $(INSTALL) -D -m 0755 $(@D)/demo $(TARGET_DIR)/usr/bin/demo; fi
27 $(INSTALL) -D -m 0755 $(@D)/$(DEMO_NAME) $(TARGET_DIR)/usr/bin/$(DEMO_NAME)
28 $(SED) "/\<$(DEMO_NAME)\>/d" $(TARGET_DIR)/etc/inittab
29 echo 'null::respawn:/usr/bin/$(DEMO_NAME) > /dev/console 2>&l' >> $(TARGET_DIR)/etc/inittab
30
31 $(INSTALL) -m 755 -d $(TARGET_DIR)/hccast_files
32 $(INSTALL) -m 755 $(@D)/$(DEMO_DIR)/*.m2v $(TARGET_DIR)/hccast_files
33
34 $(INSTALL) -m 755 -d $(DATA_DIR)/hccast_files
35 $(INSTALL) -m 755 $(@D)/$(DEMO_DIR)/*.m2v $(DATA_DIR)/hccast_files
36
37
38 $(TARGET_MAKE_ENV) $(MAKE) -C $(@D) install INSTALL="$(INSTALL)"
39 endef
40
41 $(eval $(generic-package))

```

16.5.5 如何让squashfs变为可写

16.6 按键类支持

16.6.1 如何增加ir遥控器

1.涉及的文件

```

1 1.驱动代码
2 hclinux/SOURCE/linux-drivers/drivers/hcdriivers/rc
3 |— hc_rc.c           // ir platform driver
4 |— keymaps
5 |   |— Makefile
6 |       |— rc-hcdemo.c    // 遥控器驱动，负责按键键值映射
7 |— Makefile
8
9 2.dts配置
10 hclinux/board/hichip/hc16xx/dts/hc16xx-db-d3100-v20.dts
11 &irc {
12     linux,rc-map-name = "rc-hcdemo";    // 对应rc_map_list中name，见rc-hcdemo.c中hcdemo_map
    数组定义
13     pinctrl-names = "active";
14     pinctrl-0 = <&pctl_irc>;
15     status = "okay";
16 };

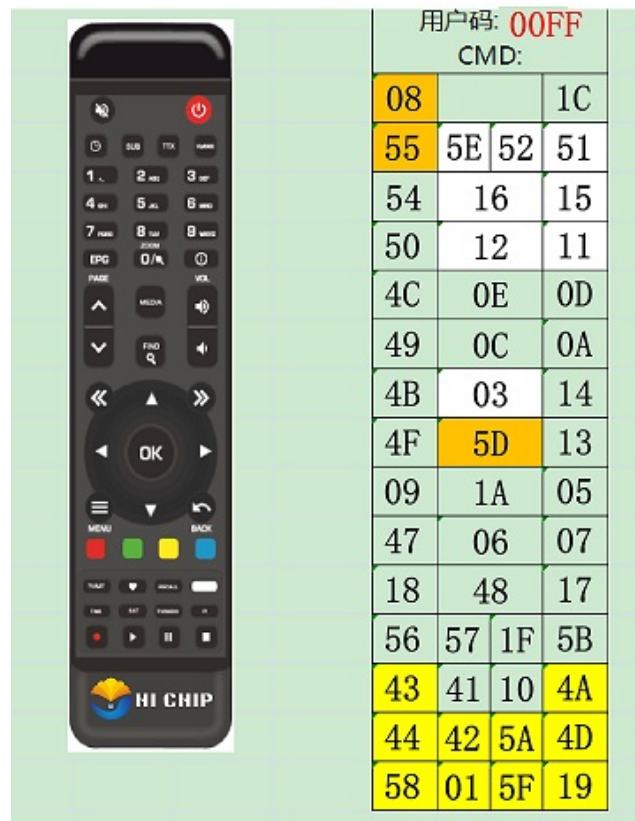
```

2.准备工作

拿到遥控器产品规格书，至少弄清楚如下两件事情：

(1).遥控器协议，例如：NEC、RC-5、JVC等

(2).遥控器按键码值表，如下图：

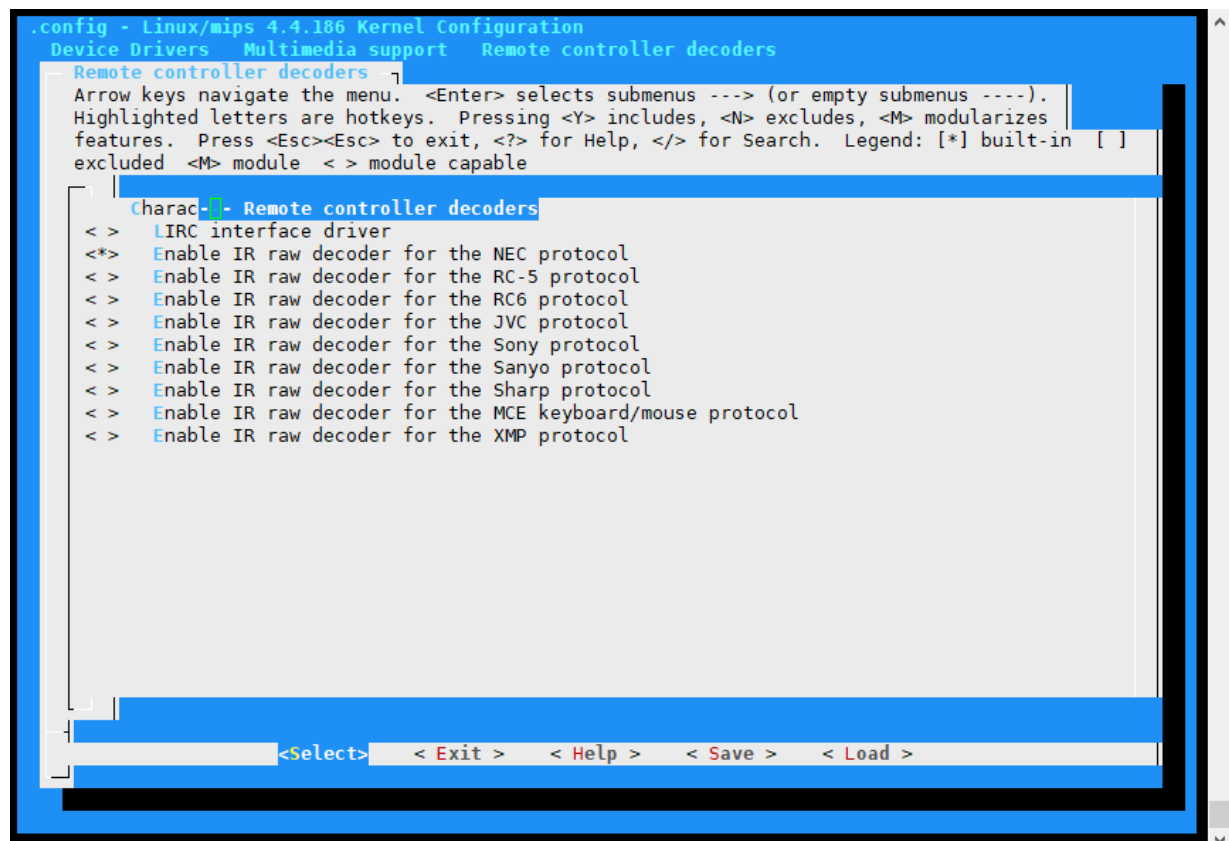


3.配置kernel config, 选择对应的遥控器协议

```

1 # cd buildroot
2 # make linux-menuconfig
3 Device Drivers ---> Multimedia support ---> Remote controller decoders --->
4

```



4.确认遥控器按键码值

通过getevent可以看到驱动发送的码值。

- 1.如果有遥控器键码表，请检查getevent获取的键值是否与码值表一致
- 2.如果没有键码表，那么将遥控器每一个按键按一下，并且记录getevent获得的对应码值

```
# ./getevent
add device 1: /dev/input/event0
  name:      "HC Remote Control Receiver"
poll 2, returned 1
/dev/input/event0: 0004 0004 00000054 按键1的keycode是0x54
poll 2, returned 1
/dev/input/event0: 0001 0201 00000001
poll 2, returned 1
/dev/input/event0: 0000 0000 00000000
poll 2, returned 1
/dev/input/event0: 0004 0004 00000054
poll 2, returned 1
/dev/input/event0: 0000 0000 00000000
poll 2, returned 1
/dev/input/event0: 0004 0004 00000054
poll 2, returned 1
/dev/input/event0: 0000 0000 00000000
poll 2, returned 1
/dev/input/event0: 0001 0201 00000000
create msg id: 0
poll 2, returned 1
/dev/input/event0: 0000 0000 00000000
poll 2, returned 1
/dev/input/event0: 0004 0004 00000016 按键2的keycode是0x16
poll 2, returned 1
/dev/input/event0: 0001 0202 00000001
poll 2, returned 1
/dev/input/event0: 0000 0000 00000000
poll 2, returned 1
/dev/input/event0: 0004 0004 00000016
poll 2, returned 1
/dev/input/event0: 0000 0000 00000000
poll 2, returned 1
/dev/input/event0: 0004 0004 00000016
poll 2, returned 1
/dev/input/event0: 0000 0000 00000000
poll 2, returned 1
/dev/input/event0: 0001 0202 00000000
poll 2, returned 1
/dev/input/event0: 0000 0000 00000000
```

5.制作遥控器码值与按键映射表

- 1.linux按键定义位于：hclinux/buildroot/output/build/linux-4.4.186/include/uapi/linux/input-event-codes.h
- 2.制作映射表，参考rc-hcdemo.c中struct rc_map_table hcdemo 填写映射表，上述遥控器的映射表如下：

```
1 static struct rc_map_table hcdemo[] = {
2     .....
3     { 0x54, KEY_NUMERIC_1 },
4     { 0x16, KEY_NUMERIC_2 },
5     { 0x15, KEY_NUMERIC_3 },
6     { 0x50, KEY_NUMERIC_4 },
7     { 0x12, KEY_NUMERIC_5 },
8     { 0x11, KEY_NUMERIC_6 },
9     { 0x4c, KEY_NUMERIC_7 },
10    { 0x0e, KEY_NUMERIC_8 },
```

```

11     { 0x0d, KEY_NUMERIC_9 },
12     .....
13
14     { 0x47, KEY_LEFT  },
15     { 0x1a, KEY_UP    },
16     { 0x07, KEY_RIGHT },
17     { 0x48, KEY_DOWN  },
18     { 0x06, KEY_OK   },
19     .....
20 };

```

6.将映射表添加到rc-hcdemo驱动中

将映射表添加到rc-hcdemo.c文件的hcdemo[]数组中。

7.同时支持多款遥控器

- 1.按照“5.制作遥控器码值与按键映射表”，给每个遥控器制定映射表
- 2.将多个表合成一个映射表，如果码值有重复并且功能有冲突，那么说明某款遥控器无法同时支持
- 3.将“6.将映射表添加到rc-hcdemo驱动中”，更新hcdemo驱动

至此新遥控器支持已经完成，重新编译内核(make linux-rebuild all)，烧写固件即可。

8.app如何响应按键

可以参考SOURCE\lg\hccast_app\key.c中代码

16.7 如何添加flash支持

16.7.1 如何添加nand支持

下面以a3200为例

1.涉及的目录和文件

```

.
├── board
│   └── hichip
│       └── hc16xx
│           └── dts
│               └── hc16xx-db-a3200-v10.dts           //linux kernel dts配置

```

2.在dts中增加nand支持

```

1     &nand0 {
2         pinctrl-names = "active";
3         pinctrl-0 = <&pctl_nf>;
4         status = "okay";
5         partitions {

```



```

6         compatible = "fixed-partitions";
7         #address-cells = <1>;
8         #size-cells = <1>;
9
10        partition@0 {
11            label = "nand";
12            reg = <0x00000000 0x1f000000>;
13        };
14    };
15};
16
17//在pctl_uart3: pctl_uart3后面添加nand pin
18pctl_nf: pctl_nf {
19    name = "pctl_nf";
20    groups = "GPIO_B_10", "GPIO_B_11", "GPIO_B_12", "GPIO_B_13", "GPIO_B_14", \
21            "GPIO_B_15", "GPIO_B_16", "GPIO_B_17", "GPIO_B_18", "GPIO_B_19", \
22            "GPIO_B_20", "GPIO_B_21", "GPIO_B_22", "GPIO_B_23", "GPIO_B_24";
23    field-value-0 = <0x1>;
24    field-value-1 = <0x1>;
25    field-value-2 = <0x1>;
26    field-value-3 = <0x1>;
27    field-value-4 = <0x1>;
28    field-value-5 = <0x1>;
29    field-value-6 = <0x1>;
30    field-value-7 = <0x1>;
31    field-value-8 = <0x1>;
32    field-value-9 = <0x1>;
33    field-value-10 = <0x1>;
34    field-value-11 = <0x1>;
35    field-value-12 = <0x1>;
36    field-value-13 = <0x1>;
37    field-value-14 = <0x1>;
38};

```

3.修改完成后，需要重新编译kernel

16.7.2 如何添加emmc支持

下面以C3100为例

1.涉及的目录和文件

```

1  .
2  ├── board
3  │   └── hichip
4  │       └── hc16xx
5  │           |
6  │           ├── common
7  │               ├── dts
8  │                   └── hc16xx-db-c3100-v20.dts    //linux 分区划分
9  │                       |
10 │                           ├── rootfs_overlay
11 │                               └── rootfs_nor_emmc    //hcboot分区划分，必须与linux 分区一致
12 ├── configs
13 │   └── hichip_hc16xx_db_c3100_v20_defconfig    //增加rootfs overlay配置
14

```

2.修改配置

(1). 修改hc16xx-db-c3100-v20.dts配置, 使能emmc

```
1 &mmc {
2     pinctrl-names = "default";
3     pinctrl-0 = <&pctl_emmc>; // pinctrl for EMMC
4     //pinctrl-0 = <&pctl_sdio>; // pinctrl for SD
5     //pinctrl-0 = <&pctl_sdio_wifi>; // pinctrl for SD wifi
6     //mmc-pwrseq = <&sdio_pwrseq>;
7     status = "okay";
8
9     // bus-width = <8>;
10    bus-width = <4>;
11    // bus-width = <1>;
12
13    broken-cd;
14    cd-gpios = <&gpio_T 13 GPIO_ACTIVE_LOW>;
15    //non-removable;
16
17    cap-sd-highspeed;
18    sd-uhs-sdr12;
19    sd-uhs-sdr25;
20    sd-uhs-ddr50;
21    sd-uhs-sdr50;
22    sd-uhs-sdr104;
23    cap-sdio-irq;
24
25 };
```

(2). 修改方案配置, hichip_hc16xx_db_c3100_v20_defconfig, 增加emmc配置

```
1 BR2_ROOTFS_OVERLAY="$(BR2_EXTERNAL_HCLINUX_PATH)/board/hichip/hc16xx/common/rootfs_overlay/
  rootfs_nor_emmc"
```

3.修改完成后, 需要重新编译kernel

16.8 如何更换bootlogo

1.涉及的目录和文件

```
1 |─ board
2 |   └─ hichip
3 |       └─ hc16xx
4 |           └─ logo.m2v //bootlogo
5 |           └─ post-build.sh //bootlogo图片解码
6 |       └─ SOURCE
7 |           └─ avp
8 |               └─ configs
9 |                   └─ hichip_hc16xx_linux_bl_defconfig //hcboot配置文件
```

hcboot当前只支持如下格式的图片或视频: mpg2、jpeg。为了加快开机速度, 系统会在编译时通过post-build.sh脚本将logo.m2v解码成logo.hc, 并且存放在romfs.img中。

2. 更换bootlogo

系统默认支持mpg2格式，下面以更换为jpeg图片为例

(1). 将logo.jpg图片拷贝到board/hichip/hc16xx目录

(2). 修改post-build.sh文件

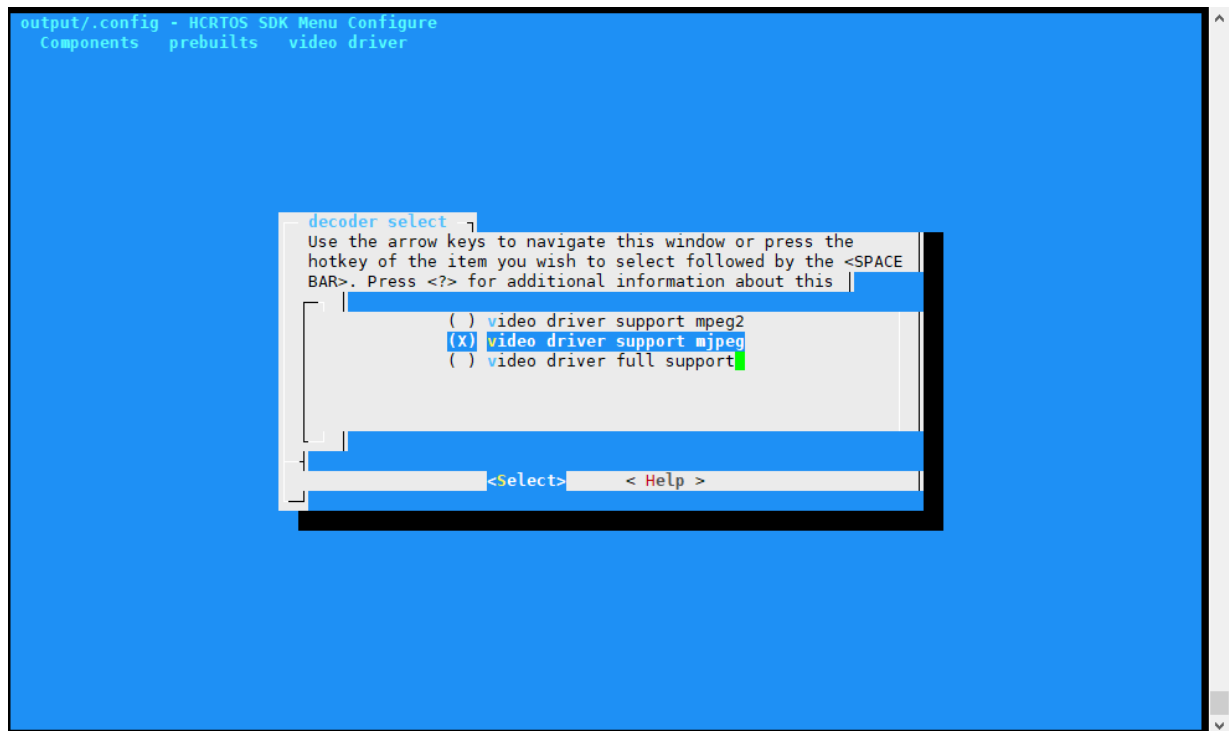
```
1 | ${BR2_EXTERNAL_HCLINUX_PATH}/support/scripts/genbootmedia -i ${current_dir}/logo.jpg -o  
  | ${BINARIES_DIR}/romfs-root/logo.hc
```

(3). 修改eromfs分区大小

根据图片大小修改eromfs分区大小，修改方法请参考《16.5 如何调整分区大小和新增分区》。

(4). 修改hcboot配置增加jpeg支持

```
1 | # make hcboot-menuconfig  
2 |  
3 | Components ---> prebuilts ---> video driver ---> decoder select (video driver support  
  | mjpeg) --->  
4 |  
5 | 选择“video driver support mjpeg”
```



(5). 将修改的配置保存

参考《16.1.2.4 修改hcboot配置》

16.9 如何申请mmz内存

1.mmz说明

mmz操作定义位于：linux-drivers/include/uapi/hcuapi/mmz.h

```

1 struct mmz_blk {
2     int id;          //<! mmz ID to request memory
3     size_t alignment; //<! alignment of memory to request
4     int size;         //<! Size of memory to request
5     uint32_t addr;    //<! Address to return or address to free
6 };
7
8 #define MMZ_TOTAL      _IOWR(MMZ_IOCBASE, 0, struct mmz_blk)
9 #define MMZ_MEMALIGN   _IOWR(MMZ_IOCBASE, 1, struct mmz_blk)
10 #define MMZ_FREE       _IOWR(MMZ_IOCBASE, 2, struct mmz_blk)

```

通过MMZ_MEMALIGN获得的内存是内核态内存，不能在用户空间直接使用，必须通过mmap映射为用户态内存。

2.演示代码

```

1 #include <hcuapi/mMZ.h>
2
3 /* 1.app使用user_addr地址
4    2.如果想让disp驱动使用mmz内存中的，必须通过ioctl传递kernel_addr地址 */
5 struct mmz_buffer{
6     struct mmz_blk blk;
7
8     void *kernel_addr; //kernel space address
9     void *user_addr;   //user space address
10    int size;
11 };
12 int mmz_fd = -1;
13 struct mmz_buffer mmz_buf;
14
15 static void mmz_test_init(void)
16 {
17     char *mmz_name = "/dev/mMZ";
18     int ret = 0;
19     int buf_size = 1280*720*8;
20
21     memset(&mmz_buf, 0, sizeof(struct mmz_buffer));
22
23     mmz_fd = open(mMZ_name, O_RDWR);
24     if( mmz_fd < 0){
25         printf("open %s failed\n", mmz_name);
26         return;
27     }
28
29     mmz_buf.blk.id = 0;          //默认id为0
30     mmz_buf.blk.alignment = 32; //内存地址为32byte对齐
31     mmz_buf.blk.size = buf_size;
32     ret = ioctl(mMZ_fd, MMZ_MEMALIGN, &mmz_buf.blk);
33     if(ret != 0){
34         close(mMZ_fd);
35         mmz_fd = -1;
36         return;
37     }
38
39     mmz_buf.size = mmz_buf.blk.size;
40     mmz_buf.kernel_addr = mmz_buf.blk.addr;
41     mmz_buf.user_addr = (void *)mmap(mMZ_buf.kernel_addr, mmz_buf.size, PROT_READ |
42     PROT_WRITE, MAP_SHARED, mmz_fd, 0);
43
44     printf("kernel_addr=0x%x\n", (unsigned int)mmz_buf.kernel_addr);
45     printf("user_addr=0x%x\n", (unsigned int)mmz_buf.user_addr);
46     printf("size=%d\n", mmz_buf.size);

```

```

46 |
47 |     return;
48 | }
49 |
50 | static void mmz_test_exit(void)
51 | {
52 |     /* free mmz memory */
53 |     ioctl(mmz_fd, MMZ_MEMALIGN, &mmz_buf.blk);
54 |     close(mmz_fd);
55 |
56 |     return;
57 | }

```

16.10 如何调试wifi

16.10.1 加载wifi 驱动

下面以调试r1t-8811cu为例

1.涉及的目录和文件

```

1 | .
2 | |
3 | |─ package
4 | |   └─ wpa_supplicant
5 | |       └─ wpa_supplicant-rtl
6 | |           └─ wpa_supplicant
7 | |               └─ wpa_supplicant.conf    //
8 | |
9 | |─ SOURCE
10 | |   |
11 | |   └─ rtl8811cu                        //wifi driver
12 | |       └─ wpa_supplicant
13 | |           └─ wpa_supplicant-rtl        //realtek wpa_supplicant

```

2.wifi驱动配置

(1). 修改buildroot配置

buildroot配置、保存配置、编译等方法参考《16.1.2.2 修改buildroot配置》

配置选择wifi驱动：

```
/media/ps/data/home/javen.xu/workspace/verify/release/hclinux-2022.03.y.2/hclinux/buildroot/output/a300/.config - Buildroot -g5c50fd9 Configuration
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters are hotkeys. Pressing <Y> selects a feature, while <N> excludes a feature. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] feature is selected [ ] feature is excluded

Target options --->
Build options --->
Toolchain --->
System configuration --->
Kernel --->
Target packages --->
Filesystem images --->
Bootloaders --->
Host utilities --->
Legacy config options --->
[*] External options --->

<Select> <Exit> <Help> <Save> <Load>
```

```
/media/ps/data/home/javen.xu/workspace/a300/hclinux/buildroot/output/d3100/.config - Buildroot 2021.05- External options
External options
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters are hotkeys. Pressing <Y> selects a feature, while <N> excludes a feature. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] feature is selected [ ] feature is excluded

(-)-
[ ] eth-mo[ ] eth-modules
[*] prebuilts --->
[*] hc-examples --->
[ ] libdlna
[ ] libmiracast
[ ] libaircast
[ ] dsc-modules
[*] rtl8188fu
[*] rtl8811u
[ ] rtl8822cs
[ ] rtl8723as
[*] openssl support --->
[*] wpa_supplicant --->
-*- libnl
libnl version (select libnl-3) --->
[ ] install tools
[ ] getevent
[*] SSV6X5X Wireless driver
[*] hotplug helper
[ ] libge
[ ] directfb
[ ] HC Fota

<Select> <Exit> <Help> <Save> <Load>
```

配置选择realtek wpa_supplicant:

```
/media/ps/data/home/javen.xu/workspace/a300/hclinux/buildroot/.config - Buildroot 2021.05-rc2-59-gf2d9821c42 Configurati
i External options
- External options -
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters
are hotkeys. Pressing <Y> selects a feature, while <N> excludes a feature. Press <Esc><Esc> to exit, <?> for
Help, </> for Search. Legend: [*] feature is selected [ ] feature is excluded

[ ] eth-modules
[*] prebuilts --->
[*] ffmpeg-example
[ ] Neptune ----
[ ] Platinum
[ ] libdlina-porting
[ ] libdlina
[ ] libmiracast
[ ] airplay
-* libgoahead
[ ] dsc-modules
[ ] rtl8188fu
[*] rtl8811cu
[ ] rtl8822cs
[ ] rtl8723as
-* openssl support --->
[*] wpa_supplicant --->
-* libnl
    libnl version (select libnl-3) --->
[ ] install tools
[*] getevent
[ ] SSV6X5X Wireless driver

<Select> <Exit> <Help> <Save> <Load>
```

```
/media/ps/data/home/javen.xu/workspace/a300/hclinux/buildroot/output/d3100/.config - Buildroot 2021.05-rc2-59-gf2d9821c42 Configurati
r External options wpa_supplicant
- wpa_supplicant -
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters
are hotkeys. Pressing <Y> selects a feature, while <N> excludes a feature. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] feature is
selected [ ] feature is excluded

--- wpa_supplicant
select wpa_supplicant version (wpa_supplicant 2.0 provided by Realtek) --->
[*] Enable nl80211 support
[*] Enable wext (deprecated)
[ ] Enable wired support
[*] Enable IBSS RSN
[*] Enable AP mode
[*] Enable Wi-Fi Display
[ ] Enable mesh networking
[*] Enable autoscan
-* Enable EAP
[*] Enable HS20
[ ] Enable syslog support
[*] Enable WPS
[ ] Enable WPA3 support
[*] Install wpa_cli binary
[*] Install wpa_client shared library
[*] Install wpa_passphrase binary
-* Enable the Unix-socket control interface
[ ] Enable support for the DBus control interface

<Select> <Exit> <Help> <Save> <Load>
```

```
/media/ps/data/home/javen.xu/workspace/a300/hclinux/buildroot/output/d3100/.config - Buildroot 2021.05-  
r External options wpa_supplicant
```

select wpa_supplicant version

Use the arrow keys to navigate this window or press the
hotkey of the item you wish to select followed by the <SPACE
BAR>. Press <?> for additional information about this

(X) wpa_supplicant 2.0 provied by Realteck
() buildroot original wpa_supplicant

<Select>

< Help >

```
/media/ps/data/home/javen.xu/workspace/a300/hclinux/buildroot/output/d3100/.config - Buildroot 2021.05-  
r External options - wpa_supplicant
```

External options

Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----).
Highlighted letters are hotkeys. Pressing <Y> selects a feature, while <N> excludes a
feature. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] feature is
selected [] feature is excluded

(-)-

[] eth-modules
[*] prebuilts --->
[*] hc-examples --->
[] libdlua
[] libmiracast
[] libaircast
[] dsc-modules
[*] rtl8188fu
[*] rtl8811cu
[] rtl8822cs
[] rtl8723as
[*] openssl support --->
[*] wpa_supplicant --->
-- libnl
libnl version (select libnl-1) --->
[] install tools
[] getevent
[*] SSV6X5X Wireless driver
[*] hotplug helper
[] libge
[] directfb
[] HC Fota

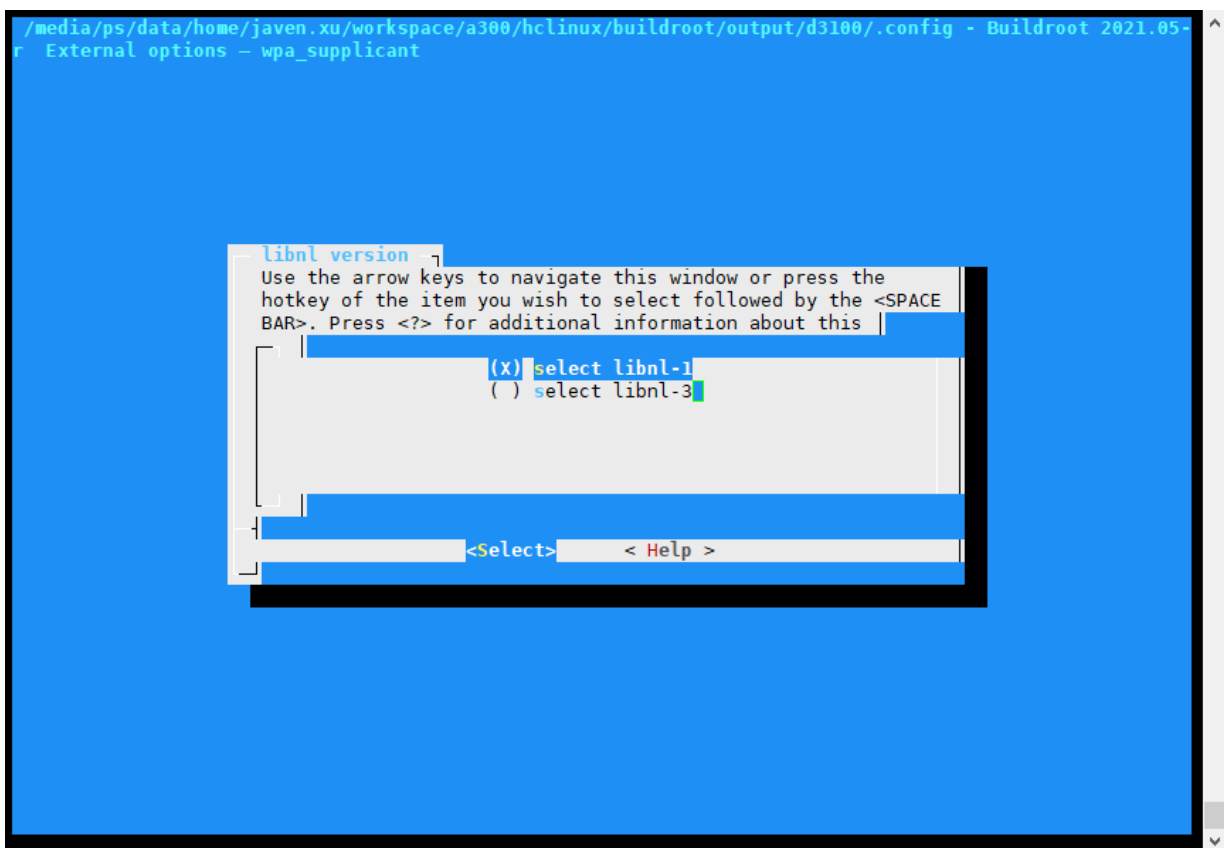
<Select>

< Exit >

< Help >

< Save >

< Load >



(2). 编译

make rtl8811cu-rebuild wpa_supplicant-rebuild all

(3). 驱动加载

开机启动加载

16.10.2 wifi配网和调试

(1). 相关代码

```
1 | SOURCE\hccast\wifi_mgr          //libhccast-net.so wifi管理库
2 | SOURCE\hc-examples\wifi_test    //wifi测试代码
```

(2). 配置wifi_test

make menuconfig进行配置

```
1 | External options
2 |     -> hc-examples
3 |         -> wifi test
```

(3). 编译

```
1 | make hc-examples-rebuild all
```

(4). 测试

在控制台输入wifi_test命令，就可以进入测试模式

命令	使用方法	说明
help	help	查看支持的命令及其用法
scan	scan	扫描wifi热点
connenct	connect -s ssid -p password	配网，ssid:热点名称，password:密码
get_status	get_status	获得当前的wifi连接状态
get_connect_ssid	get_connect_ssid	查看当前连接的热点名称

```
# wifi_test
wifi_test:~# [WIFI][INFO]: ctrl_iface: /var/run/wpa_supplicant/wlan0

wifi_test:~# help

Commands available:
  help          Show available cmds
  exit          Exit from current cmd set
  history       Show history cmds
  scan          scan
  connect       connect -s ssid -p password
  get_status    get_status
  get_connect_ssid get_connect_ssid
wifi_test:~#
```

```
wifi_test:~# scan
wifi_mgr_callback_func:41: event: HCCAST_WIFI_SCAN
[WIFI][INFO]: wifi_ctrl_wait_scan_timeout ret = 145.
[WIFI][INFO]: scan status = 0.
[WIFI][INFO]: scan ap_num = 16.
wifi_mgr_callback_func:47: event: HCCAST_WIFI_SCAN_RESULT
ssid          encryptMode  keyIdx      pwd          numCharInPwd  quality      special_ap
[0]: TP-LINK_A422 6          0           0            100           0
[1]: 中文测试 6          92          xe8\xaf\x95 0            100           0
[2]: TP-LINK_WPA3 6          0           0            100           0
[3]: xiedong-dlna 6          0           0            100           0
[4]: tp-link-a32 6          0           0            100           0
[5]: HICHIP_TEST111 6          0           0            98            0
[6]: Meeting_Room 6          0           0            72            0
[7]: TEST11 6          0           0            100           0
[8]: javen 6          0           0            100           0
[9]: redmi_hichip 6          0           0            100           0
[10]: HCCast-9D3C14 6          0           0            100           0
[11]: Hichiptech 6          0           0            100           0
[12]: Aisteed 6          0           0            100           0
[13]: hichip3 6          0           0            84            0
[14]: Hccast-08C576 6          0           0            76            0
[15]: DIRECT-A9-HP 6          0           0            76            0
```

```
-----
wifi_test:~# connect -s "javen" -p "j12345678"
wifi_connect:161: ssid=javen, pass_word=j12345678
wifi_mgr_callback_func:41: event: HCCAST_WIFI_SCAN
[WIFI][INFO]: wifi_ctrl_wait_scan_timeout ret = 145.
[WIFI][INFO]: scan status = 0.
[WIFI][INFO]: scan ap_num = 18.
wifi_mgr_callback_func:47: event: HCCAST_WIFI_SCAN_RESULT
[WIFI][INFO]: Event: WLAN CONNECTED SUCCESS
[WIFI][INFO]: wifi_ctrl_wait_connect_timeout ret = 0.
[WIFI][INFO]: Event: WLAN CONNECTED SUCCESS
[WIFI][INFO]: connect_status = connected.
wifi_connect:202:err: udhcpd start failed, ret=0
wifi_test:~# udhcpd: started, v1.33.0
udhcpd: sending discover
udhcpd: sending select for 192.168.43.219
udhcpd: lease of 192.168.43.219 obtained, lease time 3599
deleting routers
adding dns 192.168.43.1
```

```
wifi_test;#
wifi_test;# get_status
wifi_staus: connect
wifi_test;#
wifi_test;#
wifi_test;# get_connect_ssid
wifi_mgr_callback_func:60: event: HCCAST_WIFI_CONNECT_SSID, ssid: javen
wifi_get_connect_ssid:229: cur_ssid=javen
wifi_test;#
```

16.10.3 如何保存和读取wifi密码

1.自动配网和回连

wifi密码保存在/etc/wpa_supplicant.conf里面，wpa_supplicant进程启动时，会自动去配网。

由于`rootfs`只读，无法修改`/etc/wpa_supplicant.conf`文件，所以此方法不支持。

2.主应用(hcscreen_app)配网和回连

wifi密码的保存由hcscreen_app完成的，保存在persistentmem分区中。hcscreen_app启动时，从persistentmem分区中读取wifi密码，进行配网。

代码位于：SOURCE\applications\hcscreenapp\hcscreen_app\data_mgr.c

相关函数：

```
void data_mgr_wifi_ap_save(hccast_wifi_ap_info_t *wifi_ap)
```

```
int data_mgr_save(void)
```

16.11 显示相关

16.11.1 如何调试显示效果(亮度、对比度等)

1.涉及的文件

```
1 1. 驱动接口
2 SOURCE\linux-drivers\include\uapi\hcuapi\dis.h //显示效果的接口
3
4 2.demo代码
5 SOURCE\ffplayer-example\dis_test.c //显示接口调用demo
```

2.dis.h涉及代码

```
1 typedef enum DIS_VIDEO_ENHANCE {
2     DIS_VIDEO_ENHANCE_BRIGHTNESS = (1 << 0), //亮度, value[0, 100], default 50
3     DIS_VIDEO_ENHANCE_CONTRAST = (1 << 1), //对比度, value[0, 100], default 50
4     DIS_VIDEO_ENHANCE_SATURATION = (1 << 2), //饱和度, value[0, 100], default 50
5     DIS_VIDEO_ENHANCE_SHARPNESS = (1 << 3), //锐度, value[0, 10 ], default 5
6     DIS_VIDEO_ENHANCE_HUE = (1 << 4), //色调, value[0, 100], default 50
7 } dis_video_enhance_e;
8
9 typedef struct video_enhance_param {
10     uint16_t enhance_type; // typedef enum DIS_VIDEO_ENHANCE
11     uint16_t grade; //!< The enhancement value. Range of 0 ~ 100, default 50 means no
    enhancement.
12     bool enable; //!< 0: disable video enhancement, 1: enable video enhancement
13 } video_enhance_param_t;
14
```

```

15 typedef struct dis_video_enhance {
16     enum DIS_TYPE distype;
17     struct video_enhance_param enhance;
18 } dis_video_enhance_t;
19
20 //设置显示效果参数
21 #define DIS_SET_VIDEO_ENHANCE _IOW(DIS_IOCBASE, 6, struct dis_video_enhance)
22 #define DIS_SET_VIDEO_ENHANCE_ENABLE _IOW(DIS_IOCBASE, 7, struct dis_video_enhance)

```

3.dis_test.c涉及的代码

DIS_SET_VIDEO_ENHANCE和DIS_SET_VIDEO_ENHANCE_ENABLE的使用方法参考下面的函数：

```
int venhance_test(int argc, char *argv[])
```

```
int enhance_enable_test(int argc, char *argv[])
```

4.demo测试方法

1.buildroot配置需要选择上ffmplayer（发布sdk默认已经配置）

2.烧写固件后，执行mplayer命令，如下图：

```

# mplayer
current volume is 10
cat: write error: No space left on device
mp:# dis_test
mp:(dis_test)# help

Commands available:
  help          Show available cmds
  exit          Exit from current cmd set
  history       Show history cmds
  aspect_ratio  aspect_ratio -d DIS_TYPE(0 sd, 1 hd, 2 uhd) -t TV_MODE -m DISPLAY_MODE
  tvsys        tvsys -c cmd(0:DIS_SET_TVSY 1:DIS_GET_TVSY) -l DIS_LAYER -d DIS_TYPE -t TVTYPE -p progressive -s 1:dual ou
t 0:single out -a dac type
  zoom         zoom -c cmd -l DIS_LAYER -d DIS_TYPE source area:(-x -y -w -h ) dst area:(-o x-offset of the area -k y-offs
et of the area -j Width of the area -g Height of the area
  winon        winon -l DIS_LAYER(0x1 main, 0x10 auxp) -d DIS_TYPE(0 sd, 1 hd, 2 uhd) -o ON(1 on, 0 off)
  venhance     venhance -c Picture-enhancement-type(0x01 BRIGHTNESS,0x02 CONTRAST,0x04 SATURATION,0x08 SHARPNESS,0x10 HUE) -d
DIS_TYPE(0 sd, 1 hd, 2 uhd) -g picture enhancement value(0-100,default 50)
  hanen       hanen -e ENHANCE_EN(0 disable, 1 enable) -d DIS_TYPE(0 sd, 1 hd, 2 uhd)
  outfmt      outfmt -f HDMI_OUT_FMT(1 YCBCR_420,2 YCBCR_422,3 YCBCR_444,4 RGB_MODE1,5 RGB_MODE2) -d DIS_TYPE(0 sd, 1 hd,
2 uhd)
  layerorder   layerorder -d 1 -o 0 -l 1 -r 2 -g 3 ->set layer order: main_layer 0, auxp_layer 1, gmas_layer 2, gmaf_layer 3
  bppic       bppic -d DIS_TYPE(0 sd, 1 hd, 2 uhd)
  cutoff      cutoff -d DIS_TYPE(0 sd, 1 hd, 2 uhd) -c cutoff
  awonoff     awonoff -d DIS_TYPE(0 sd, 1 hd, 2 uhd) -w WinOnOff
  singleout   singleout -d DIS_TYPE(0 sd, 1 hd, 2 uhd) -o single output
  keystone    keystone -d DIS_TYPE(0 sd, 1 hd, 2 uhd) -c bg color enable(0 disable 1 enable) -e keystone(0 disable 1 enable)
) -w up width -i down width -y bg y -b bg cb -r bg cr
  bgcolor     bgcolor -d DIS_TYPE(0 sd, 1 hd, 2 uhd) -y color uY -b color uCb -r color uCr
  regdac      regdac -d DIS_TYPE(0 sd, 1 hd, 2 uhd) -t eDacType -e enable -f uDacFirst -s uDacSecond -h uDacThird -p bPregr
essive(1 DVI_SCAN_PROGRESSIVE,2 DVI_SCAN_INTERLACE) -v tvtype
  underdac    underdac -d DIS_TYPE(0 sd, 1 hd, 2 uhd) -u type
mp:(dis_test)# venhance -d 1 -c 1 -g 60 调整屏幕亮度到60,-c 1表示屏幕亮度
-d distype 1
-c enhance_type 1
-g grade 60
enter ioctl
ioctl done
mp:(dis_test)#

```

16.11.2 屏幕旋转

下面以D3100为例

1.涉及的文件

board/hichip/hc16xx/common/dts/hc16xx-db-d3100-v20-avp.dtsi

2.配置旋转属性

```

1 rotate{
2     /*
3     * anticlockwise,
4     * support 0/90/180/270 degree.
5     */

```

```

6      rotate = <0>;    /* 逆时针旋转 0/90/180/270 */
7
8      /*
9      * value is 0 or 1
10     * h_flip = 1 is horizontally flipped
11     * v_flip = 1 is vertically flipped
12     */
13     h_flip = <0>;    /* 水平翻转 */
14     v_flip = <0>;    /* 垂直翻转 */
15
16     status = "disabled";
17 };

```

3.实现旋转

旋转涉及模块有：

- bootlogo：hcboot已经实现，默认会去解析dts
- UI：liblvgi中有实现，默认会去解析dts
- 视频播放：支持旋转，app需要解析dts，在启动视频/图片播放之前将旋转属性配置给解码器

4.编译

rotate涉及很多模块，建议make clean，重新编译。

16.12 hdmi in调试

下面以a300方案为例，调试hdmi in。

1.涉及的文件

```

1  1. 驱动
2  SOURCE\avp\components\prebuilts\sysroot\usr\lib\libviddrv_hdmirx.a  //hdmi rx驱动库文件
3  SOURCE\linux-drivers\include\uapi\hcuapi\hdmi_rx.h                  //hdmi rx驱动头文件
4
5  2. 配置文件
6  board\hichip\hc16xx\dts\hc16xx-db-a300-v10-avp.dtsi
7  board\hichip\hc16xx\dts\hc16xx-db-a300-v10.dtsi
8  SOURCE\avp\configs\hichip_hc16xx_linux_avp_defconfig
9
10 3. demo代码
11 SOURCE\hc-examples\hdmi_rx_test.c //hdmi rx 测试程序
12 SOURCE\hc-examples\hdmi_switch   //hdmi tx/rx demo程序

```

2.配置hdmi in

(1). dts中使能hdmi rx

修改hc16xx-db-a300-v10-avp.dtsi，使能hdmi rx

```

1  #define CONFIG_HDMI_RX_SUPPORT 1    //置1，使能hdmi rx
2
3  //配置的buffer大小不一样

```

```

4  #define CONFIG_MM_VIN_FB_SIZE (1920*1088*2*4)
5
6  //sdk默认没有配置fb0, 请配置上
7  fb0 {
8      bits_per_pixel = <32>;
9      xres = <1280>;
10     yres = <720>;
11     xres_virtual = <1280>;
12     yres_virtual = <720>;
13     xoffset = <0>;
14     yoffset = <0>;
15
16     scale = <1280 720 1920 1080>;
17
18     reg = <CONFIG_FB0_REG 0x1000>;
19     /*
20      * frame buffer memory from:
21      * system : malloc buffer from system heap
22      * mmz0    : malloc buffer from mmz0
23      * none    : set buffer by user
24      * default is system if the property is missing
25      */
26     buffer-source = "system";
27     status = "okay";
28 };
29
30 i2s {
31     pinmux-clock = <2 2 3 2 4 2>;
32     status = "okay";           //确保i2s使能
33     ejtag = "disabled";
34 };
35
36 //如果需要录制hdmi rx的声音, 需要将i2si enable
37 i2si {
38     volume = <255>;
39     status = "okay";
40 };

```

(2). 配置defconfig

(2-1). 进行avp配置

```
1 | make avp-menuconfig
```

选上hdmi rx driver plugin

```
1 | Components ---> prebuilts ---> hdmi rx driver
```

选上hdmi rx driver

```
1 | Components ---> kernel ---> Drivers ---> video Support ---> hdmi tx Support
```

选上fb

```
1 | Components ---> kernel ---> Drivers ---> video ---> hcfb
```

保存avp配置, 参考《16.1.2.5 修改avp配置》

```
95 javen.xu@hichip01: ~/workspace/verify/release/hclinux/hclinux-2022.10.y.rc1/hclinux/buildroot/output/d3100 - Xshell 5
文件(F) 编辑(E) 查看(V) 工具(T) 选项卡(B) 窗口(W) 帮助(H)
ssh://javen.xu:*****@172.16.100.95:22
95
195 x 295 x 395 x +
output/.config - HCRTOS SDK Menu Configure
Components prebuilts -
prebuilts
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenu ----). Highlighted letters
are hotkeys. Pressing <Y> selects a feature, while <N> excludes a feature. Press <Esc><Esc> to exit, <?> for
Help, </> for Search. Legend: [*] feature is selected [ ] feature is excluded

--- prebuilts
[ ] ffmpeg
[*] audio driver
*- video driver --->
[*] decoder & driver plugins --->
[*] hdmi rx driver
[ ] usb driver
[ ] sd-mmc driver
[ ] ntfs support
[ ] libge
[ ] libdsc
[ ] libspectrum
[ ] lib3a ----
[ ] libdrc
[ ] liblrbalance
[ ] libtwotone
[ ] libws2811
[*] efuse driver
[ ] rtl8188fu
[ ] wpa_supplicant
[ ] wireless tools

<Select> <Exit> <Help> <Save> <Load>

tail -f /dev/uart
SSH2 xterm 120x36 14.26 3 会话 CAP NUM
```

```
output/.config - HCRTOS SDK Menu Configure
Components kernel Drivers Video Support ins
Video Support
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenu ----). Highlighted letters
are hotkeys. Pressing <Y> selects a feature, while <N> excludes a feature. Press <Esc><Esc> to exit, <?> for
Help, </> for Search. Legend: [*] feature is selected [ ] feature is excluded

[*] Video Support
[*] hdmi tx Support
[*] hdmi rx Support
[*] vin dvp Support
[*] tv decoder Support
[*] video decoder Support
[*] video sink Support

<Select> <Exit> <Help> <Save> <Load>
```

```
output/.config - HCRTOS SDK Menu Configure
Components  kernel  Drivers  video Support ins
video
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters
are hotkeys. Pressing <Y> selects a feature, while <N> excludes a feature. Press <Esc><Esc> to exit, <?> for
Help, </> for Search. Legend: [*] feature is selected [ ] feature is excluded

-[- video river ----
  [*] hcfb

<Select> < Exit > < Help > < Save > < Load >
```

(2-2). 进行buildroot配置

```
1 | make menuconfig
```

选上hdmi_switch和hdmi rx test, 他们都在hc_examples里面

```
1 | External options ---> hc-examples
```

```
/media/ps/data/home/javen.xu/workspace/verify/release/hclinux-2022.03.y.2/hclinux/buildroot/output/a300/.config - Build
r External options
External options
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters
are hotkeys. Pressing <Y> selects a feature, while <N> excludes a feature. Press <Esc><Esc> to exit, <?> for
Help, </> for Search. Legend: [*] feature is selected [ ] feature is excluded

*** HiChipTech Linux sdk distribution (in /media/ps/data/home/javen.xu/workspace/verif
({BR2_EXTERNAL_HCLINUX_PATH}/board/hichip/hcl6xx/ddrinit/aux_code_ddr3_128M_1333MHz.abs)
Bootloaders --->
Extra Linux Configuration --->
[ ] ffmpeg ----
[*] Avpconsole
[ ] hccast
[*] avp --->
[*] lvgl
[ ] eth-modules
[*] prebuilts --->
  [*] hc-examples
  [ ] libdlua
  [ ] libmiracast
  [ ] libaircast
  [ ] dsc-modules
  [ ] rtl8188fu
  [ ] rtl8811cu
  [ ] rtl8822cs
  [ ] rtl8723as
  [*] openssl support --->
  [*] wpa_supplicant --->

  (+)

<Select> < Exit > < Help > < Save > < Load >
```

3.编译hdmi in

编译之前，请保存好avp配置，方法参考《16.1.2.5修改avp配置》

```
#source support/scripts/envsetup.sh
```

```
#mkall //编译avp和linux
```

4.debug测试

(1). hdmi_switch

在命令行中输入：hdmi_switch

(2). hdmi rx

在命令行中输入hdmi_rx，进入测试模式。

图像输出到TV，命令：start -a 0 -v 1 //声音输出到hdmi tx

图像输出到LCD，命令：start -a 1 -v 1 //声音输出到hdmi i2so

录制hdmi rx数据，命令：start -a 3 -v 3 //声音和视频录制到U盘中

5.问题排查方法

情况1.hdmi设备有插入，但是没有图像出来

问题现象：hdmi设备有插入，但是没有图像出来

串口打印：

```
1 #tail -f /dev/virtuart & //打开后台打印
2
3 #hdmi_rx
4 hdmi_rx:# start -a 0 -v 1
5 apath 0, vpath 1
6 hdmi_rx start ok```
7 hdmi_rx:# hdmi_rx_enc_enable = 0
8 hdmi_rx_enc_play_enable = 0
9 hdmi_rx_enc_store_enable = 0
10 vin_param.fb_size = 0x17e8000
11 vin_param.fb_addr = 0xa5d67000
12 The framebuffer device was opened successfully.
13 variable screen: 1280x720, 32bpp
14 The framebuffer device was mapped to memory successfully.
15 hdmi_rx_open 422
16 HDMI_RX_FLAG_HPD_CONNECT
17 hdmi_rx_cb_video_clk_pre_chg_for_osd
18 exit hdmi_rx_cb_video_clk_pre_chg_for_osd
19 hdmi_rx_phy_init done
20 hdmi_rx_phy_wait_clock_lock TIME OUT
21 disconnect
22 hdmi_rx_ctrl_hal_enable_md_interrupt
```

问题原因：hdmi设备被识别，但是没有收到图像

1. hdmi设备本身就没有出图像

2. hdmi设备和D3100 hdmi rx存在兼容性问题，造成 hdmi设备不输出设备

排查方法：将hdmi设备查到TV，看看是否有图像输出。

- | | |
|---|--|
| 1 | 1. 如果没有图像输出，那就是问题原因1，需要对hdm设备重新上电。 |
| 2 | 2. 如果有图像输出，那就是问题原因2，是hdm rx兼容性问题，需要联系原厂支持。 |

16.13 libge调试

1.涉及的文件

```
1 1.lib库
2 SOURCE\prebuilts\usr\lib\libge.so //libge动态库
3 SOURCE\prebuilts\usr\lib\libge.a //libge静态库
4 SOURCE\prebuilts\usr\include\hcge\ge_api.h
5
6 2.配置文件
7 board\hichip\hc16xx\dts\hc16xx-common.dtsi
8 board\hichip\hc16xx\dts\hc16xx-db-d3100-v20.dts
9 configs\hichip_hc16xx_db_d3100_v20_defconfig
10
11 3.demo代码
12 SOURCE\hc-examples\ge //ge 测试程序
13
```

2.lib库函数说明

```
1 //初始化ge
2 int hcge_open(hcge_context **pctx);
3
4 //注销ge
5 void hcge_close(hcge_context *ctx);
6
7 //ge硬件reset
8 void hcge_hw_reset(hcge_context *ctx);
9
10 //ge 软件引擎reset
11 void hcge_reset(struct hcge_context *ctx);
12
13 //this api for directfb, only directfb used
14 //检查ge引擎状态，提供给directfb使用，其他情况不建议使用
15 void hcge_check_state(hcge_state *state, HCGEAccelerationMask accel);
16
17 //等待ge完成所以命令，用于同步
18 int hcge_engine_sync(hcge_context *ctx);
19
20 //ge 软件引擎reset，提供给directfb使用，其他情况不建议使用
21 void hcge_engine_reset( void *drv, void *dev);
22
23 //flush cache，提供给directfb使用，其他情况不建议使用
24 void hcge_flush_texture_cache(void *drv, void *dev);
25
26 //让ge立即执行，提供给directfb使用，其他情况不建议使用
27 void hcge_emit_jommands(void *drv, void *dev);
28
29 //画矩形
30 bool hcge_draw_rect(hcge_context *ctx, HCGERectangle *rect);
31
32 //填充矩形
33 bool hcge_fill_rect(hcge_context *ctx, HCGERectangle *rect);
34
35 //数据搬运，不做拉升
```

```

36 bool hcge_blit(hcge_context *ctx, HCGERectangle *srect, int dx, int dy);
37
38 //数据搬运并且拉升
39 bool hcge_stretch_blit(hcge_context *ctx, HCGERectangle *srect, HCGERectangle *drect);
40
41 //设置ge状态，主要是设置ge引擎参数，在调用图像处理函数
42 (hcge_draw_rect,hcge_fill_rect,hcge_blit,hcge_stretch_blit)之前必须调用
43 void hcge_set_state(hcge_context *ctx, hcge_state *state, HCGEAccelerationMask accel);
44
45 //填充矩形
46 void hcge_fill_rect_ext(hcge_context *ctx, HCGE_CoreSurfaceBuffer *dst, HCGE_CoreSurface
47 *surface, HCGERectangle *rect, HCGEColor *color);

```

函数接口如何使用，请参考SOURCE\hc-examples\ge。

3.配置

(1). dts配置

hc16xx-common.dtsi:

```

1 ge: ge@18806000 {
2     compatible = "hichip,hc16xx-ge";
3     reg = <0x18806000 0x2000>, <0x18800000 0x1000>;
4     interrupts = <4>;
5     cmdq_buf_size = <0x3c000>;
6     status = "okay";    //确认是否打开，默认都是打开的
7 };

```

hc16xx-db-d3100-v20.dts:

```

1 &fb0 {
2     bits_per_pixel = <32>;
3     xres = <1280>;
4     yres = <720>;
5     xres_virtual = <1280>;
6     yres_virtual = <720>;
7     xoffset = <0>;
8     yoffset = <0>;
9
10    scale = <1280 720 1920 1080>;
11
12    reg = <CONFIG_FB0_REG 0x1000>;
13    /*
14     * frame buffer memory from:
15     * system : malloc buffer from system heap
16     * mmz0    : malloc buffer from mmz0
17     * default is system if the property is missing
18     */
19    buffer-source = "system";
20    extra-buffer-size = <0xc00000>; /* 8Mb */    //ge使用到的内存大小
21    status = "okay";
22
23 };

```

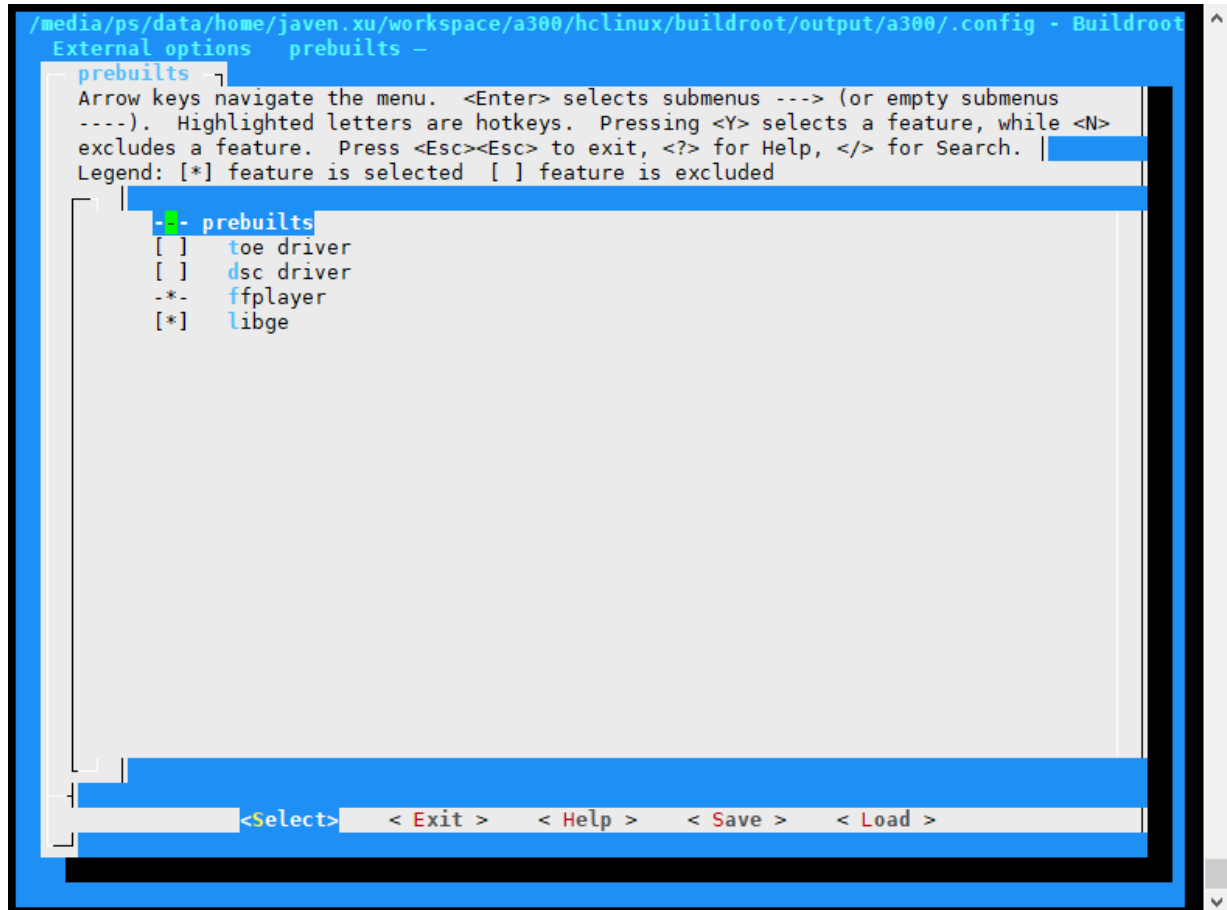
(2). defconfig配置

libgo在buildroot配置中，位于:configs\hichip_hc16xx_db_d3100_v20_defconfig

```

1 #make menuconfig
2
3 ---> External options
4     ----> prebuilts
5         ----> libge //配置libge.so
6
7 ---> External options
8     ----> hc-examples //配置ge demo程序
9

```



4.编译

```

1 make prebuilts-rebuild all //编译libge.so
2 make hc-examples-rebuild all //编译ge demo

```

5.测试

ge demo说明:

1.hcge_fill_rect_test 绘制矩形

```

1 #hcge_fill_rect_test //直接运行即可

```

2.hcge_cust_api_test 测试color key

```
1 | #hcge_cust_api_test //会弹出使用说明
2 | usage: hcge_cust_api_test <ppm file> <source color> <source colorkey value>
3 | for example: hcge_cust_api_test /nfs/dfbtest_scale.ppm 0xff800000 0xff700000
4 |               hcge_cust_api_test /nfs/dfbtest_scale.ppm 0xff800000 0xff800000
5 |
6 | #hcge_cust_api_test /media/hdd/dfbtest_scale.ppm 0xff800000 0xff700000
```

3.hcge_rotate_test //旋转

```
1 | #hcge_rotate_test /media/hdd/dfbtest_scale.ppm
```

4.hcge_blit_test //测试搬运图像数据到显存，测试旋转和拉升

```
1 | #hcge_blit_test /media/hdd/dfbtest_scale.ppm
```

16.14 USB Device调试

1.usb device涉及的文件

```
1 | 1. 配置文件
2 | board\hichip\hc16xx\dtb\hc16xx-db-d3100-v20.dts
3 | board\hichip\hc16xx\kernel-configs\4.4.186\kernel-squashfs.config //kernel defconfig
4 |
```

2.修改配置

(1).配置linux kernel

执行 make linux-menuconfig 进入配置界面

```
1 | -> Device Drivers
2 |     -> USB support
3 |         -> MUSB Mode Selection
4 |             -> Dual Role mode
5 |
```

.config - Linux/mips 4.4.186 Kernel Configuration

- Device Drivers

Linux/mips 4.4.186 Kernel Configuration

Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus --->). Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search.
Legend: [*] built-in [] excluded <M> module < > module capable

- Machine selection --->
- Endianness selection (Little endian) --->
- CPU selection --->
- Kernel type --->
- General setup --->
- [*] Enable loadable module support --->
- [*] Enable the block layer --->
- Bus options (PCI, PCMCIA, EISA, ISA, TC) --->
- Executable file formats --->
- Power management options --->
- CPU Power Management --->
- [*] Networking support --->
- Device Drivers --->
- Firmware Drivers --->
- File systems --->
- Kernel hacking --->
- Security options --->
- *- Cryptographic API --->
- Library routines --->
- [] Virtualization ----

<Select> < Exit > < Help > < Save > < Load >

.config - Linux/mips 4.4.186 Kernel Configuration

Device Drivers

Device Drivers

Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus --->). Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search.
Legend: [*] built-in [] excluded <M> module < > module capable

- (-)
- < > HSI support ----
- PPS support --->
- PTP clock support --->
- Pin controllers --->
- *- GPIO Support --->
- < > Dallas's 1-wire support ----
- [] Power supply class support ----
- [] Adaptive Voltage Scaling class support ----
- < > Hardware Monitoring support ----
- < > Generic Thermal sysfs driver ----
- [*] Watchdog Timer Support --->
- Sonics Silicon Backplane --->
- Broadcom specific AMBA --->
- Multifunction device drivers --->
- [] Voltage and Current Regulator Support ----
- <*> Multimedia support --->
- Graphics support --->
- < > Sound card support ----
- HID support --->
- [*] USB support --->
- < > Ultra Wideband devices ----
- < > MMC/SD/SDIO card support ----

! (+)

<Select> < Exit > < Help > < Save > < Load >

```

.config - Linux/mips 4.4.186 Kernel Configuration
Device Drivers  USB support - USB Gadget Support
  USB support
  Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted
  letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc>
  to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module <> module
  capable
  (-)
  <> Olympus MAUSB-10/Fuji DPC-R1 support
  <> Support OneTouch Button on Maxtor Hard Drives
  <> Support for Rio Karma music player
  <> SAT emulation on Cypress USB/ATA Bridge with ATACB ATACB
  <> USB ENE card reader support
  <> USB Attached SCSI SCSI
  *** USB Imaging devices ***ces ***
  <> USB Mustek MDC800 Digital Camera support
  <> Microtek X6USB scanner support
  <> USB/IP support
  -* Inventra Highspeed Dual Role Controller (TI, ADI, AW, ...)
  [USB Mode Selection (Dual Role mode) --->) --->
  *** Platform Glue Layer ***yer ***
  *** MUSB DMA mode ***ode ***
  [ ] Disable DMA (always use PIO)
  [*] Inventra
  <> DesignWare USB3 DRD Core Support
  <> DesignWare USB2 DRD Core Support
  <> ChipIdea Highspeed Dual Role Controller
  <> NXP ISP 1760/1761 support
  *** USB port drivers ***
  <> USB Serial Converter support ----
  *** USB Miscellaneous drivers ***
  <> EMI 6|2m USB Audio interface support
  (+)
  <Select> <Exit> <Help> <Save> <Load>

```

- 1 -> Device Drivers
- 2 -> USB support
- 3 -> USB Gadget Support
- 4 -> USB Gadget Drivers
- 5 -> Mass Storage Gadget

```

.config - Linux/mips 4.4.186 Kernel Configuration
Device Drivers  USB support
  USB support
  Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted
  letters are hotkeys. Pressing <Y> includes, <N> excludes, <M>
  modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search.
  Legend: [*] built-in [ ] excluded <M> module <> module capable
  (-)
  <> USB Diamond Rio500 support
  <> USB Lego Infrared Tower support
  <> USB LCD driver support
  <> USB LED driver support
  <> Cypress CY7C63xxx USB driver support
  <> Cypress USB thermometer driver support
  <> Siemens ID USB Mouse Fingerprint sensor support
  <> Elan PCMCIA CardBus Adapter USB Client
  <> Apple Cinema Display support
  <> USB 2.0 VGA dongle support (Net2280/SiS315)
  <> USB LD driver
  <> PlayStation 2 Trance Vibrator driver support
  <> IO Warrior driver support
  <> USB testing driver
  <> USB EHSET Test Fixture driver
  <> iSight firmware loading support
  <> USB YUREX driver support
  <> Functions for loading firmware on EZUSB chips
  <> USB3503 HSIC to USB20 Driver
  <> USB Link Layer Test driver
  <> USB Physical Layer drivers --->
  <> USB Gadget Support --->
  <Select> <Exit> <Help> <Save> <Load>

```

```
.config - Linux/mips 4.4.186 Kernel Configuration
Device Drivers  USB support  USB Gadget Support -
  USB Gadget Support
  Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus
  ----). Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M>
  modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search.
  Legend: [*] built-in [ ] excluded <M> module < > module capable

  --- USB Gadget Support
  [ ] Debugging messages (DEVELOPMENT)
  [ ] Debugging information files (DEVELOPMENT)
  [ ] Debugging information files in debugfs (DEVELOPMENT)
  (2) Maximum VBUS Power usage (2-500 mA)
  (2) Number of storage pipeline buffers
  USB Peripheral Controller --->
  <M> USB Gadget Drivers
  < > USB functions configurable through configs
  < > Gadget Zero (DEVELOPMENT)
  < > Ethernet Gadget (with CDC Ethernet support)
  < > Network Control Model (NCM) support
  < > Gadget Filesystem
  < > Function Filesystem
  <M> Mass Storage Gadget
  < > Serial Gadget (with CDC ACM and CDC OBEX support)
  < > Printer Gadget
  < > CDC Composite Device (Ethernet and ACM)
  < > CDC Composite Device (ACM and mass storage)
  < > Multifunction Composite Gadget
  < > HID Gadget
  < > EHCI Debug Device Gadget

  <Select>  < Exit >  < Help >  < Save >  < Load >
```

3.编译

make linux-rebuild all

4.测试

```
1 /*
2  * 切换到device模式
3  * musb-hdrc.0.auto    usb0控制器
4  * musb-hdrc.1.auto    usb1控制器
5  */
6 echo "peripheral" > /sys/devices/platform/soc/18844000.usb/musb-hdrc.0.auto/mode
7
8 modprobe libcomposite    //加载usb驱动
9
10 mkdir -p /sys/kernel/config
11
12 mount none /sys/kernel/config -t configfs
13
14 mkdir -p /sys/kernel/config/usb_gadget/g1
15
16 echo 0x18d1 > /sys/kernel/config/usb_gadget/g1/idVendor
17
18 echo 0x4ee7 > /sys/kernel/config/usb_gadget/g1/idProduct
19
20 mkdir -p /sys/kernel/config/usb_gadget/g1/configs/b.1/strings/0x409
21
```



```

22 echo "Mass Storage" >
   /sys/kernel/config/usb_gadget/g1/configs/b.1/strings/0x409/configuration
23
24 mkdir -p /sys/kernel/config/usb_gadget/g1/functions/mass_storage.gs1
25
26 /* 在ddr中生成一个ramfs, 并且挂载到usb上 */
27 dd if=/dev/zero of=/tmp/test.img bs=1M count=8
28 echo /tmp/test.img >
   /sys/kernel/config/usb_gadget/g1/functions/mass_storage.gs1/lun.0/file
29
30
31 ln -sf /sys/kernel/config/usb_gadget/g1/functions/mass_storage.gs1
   /sys/kernel/config/usb_gadget/g1/configs/b.1/f1
32
33 echo musb-hdrc.0.auto > /sys/kernel/config/usb_gadget/g1/UDC
34

```

16.15 无线投屏介绍

16.15.1 无线投屏代码介绍

```

1  SOURCE\libcast                //dlna/miracast/aircast库文件
2  SOURCE\applications\hcscreenapp  //无线投屏app
3  SOURCE\hccast
4      |
5      ├── aircast                //aircast中间件
6      ├── app                    //hccast测试app,hccast_simple_wireless
7      ├── dlna                  //dlna中间件
8      ├── httpd                 //网页配网代码
9      ├── inc                    //hccast头文件
10     ├── miracast               //miracast中间件
11     ├── udhcp                  //分配ip地址, 开关udhcpd
12     └── wifi_mgr                //wifi manager, wifi配网、状态等

```

16.15.2 dlna接口函数

位于: SOURCE\hccast\inc\hccast_dlna.h

```

1  hccast_dlna_event_callback      //回调函数
2  int hccast_dlna_service_init(hccast_dlna_event_callback func); //初始化
3  int hccast_dlna_service_uninit(); //注销
4  int hccast_dlna_service_start(); //开启dlna服务
5  int hccast_dlna_service_stop(); //关闭dlna服务

```

16.15.3 miracast接口函数

位于: SOURCE\hccast\inc\hccast_mira.h

```

1  int hccast_mira_service_start(); //开启miracast服务
2  int hccast_mira_service_stop(); //关闭miracast服务
3  int hccast_mira_player_init(); //初始化播放器
4  int hccast_mira_get_stat(void); //获得状态
5  int hccast_mira_service_init(hccast_mira_event_callback func); //初始化
6  int hccast_mira_service_uninit(); //注销

```

16.15.4 aircast接口函数

位于: SOURCE\hccast\inc\hccast_air.h

```
1  int hccast_air_ap_mirror_stat(void);
2  int hccast_air_ap_audio_stat(void);
3  int hccast_air_service_init(hccast_air_event_callback aircast_cb);
4  int hccast_air_service_start();
5  int hccast_air_service_stop();
6  void hccast_air_mdnsd_start();
7  void hccast_air_mdnsd_stop();
8  void hccast_air_mediaplayer_2_aircast_event(int type, void *param);
9  int hccast_air_ap_get_mirror_frame_num(void);
10 int hccast_air_service_is_start();
```

16.15.5 wifi manager接口函数

位于: SOURCE\hccast\inc\hccast_wifi_mgr.h

16.16 如何调试hdmi wireless display

16.16.1 如何调试hdmi wireless display

下面以A3300为例，Hdmi无线投屏有发送端和接收端两部分，因此有两份配置和两个固件。

1.usb device涉及的文件

```
1  1. 配置
2  board\hichip\hc16xx\a3300_hwd_source      //source端板级配置
3  configs\hichip_hc16xx_db_a3300_v10_hdmi_wifi_sink_defconfig
4  configs\hichip_hc16xx_db_a3300_v10_hdmi_wifi_source_defconfig
5
6  2. 源码
7  SOURCE\hc-examples\hdmi_wireless_display  //hdmi_wireless_display 源代码
8  SOURCE\hc-examples\hdmi_wireless_display\S41display_source  //发送端开机启动脚本
9  SOURCE\hc-examples\hdmi_wireless_display\S42display_sink    //接收端开机启动脚本
10 SOURCE\hc-examples\hdmi_wireless_display\hostapd_2G.conf    //softap配置脚本
11
```

2.编译source固件

//生成a3300source方案专属编译目录

```
1  #cd buildroot
2  #make O=output/a3300_source BR2_EXTERNAL=$PWD/../../
   hichip_hc16xx_db_a3300_v10_hdmi_wifi_source_defconfig
3  #cd output/a3300_source
4  #make all
```

3.编译sink固件

```
1 | #cd buildroot
2 | #make O=output/a3300_sink BR2_EXTERNAL=$PWD/../
   | hichip_hc16xx_db_a3300_v10_hdmi_wifi_sink_defconfig
3 | #cd output/a3300_sink
4 | #make all
```

4.测试

- (1). 发送端，hdmi in接入设备，系统会自动识别设备，并且将hdmi rx收到的视频通过wifi发送出去。
- (2). 接收端，hdmi out连接TV，系统会自动与发送端联网，收取wifi数据，并且显示到TV。

16.16.2 其他方案如何配置hdmi wireless display

下面以A3200为例

1.生成source端和sink端的config配置

基于hichip_hc16xx_db_a3200_v10_defconfig生成原始config配置

```
1 | #cp configs/hichip_hc16xx_db_a3200_v10_defconfig
   | configs/hichip_hc16xx_db_a3200_v10_hdmi_wifi_source_defconfig
2 |
3 | #cp configs/hichip_hc16xx_db_a3200_v10_defconfig
   | configs/hichip_hc16xx_db_a3200_v10_hdmi_wifi_sink_defconfig
```

2. 编译source固件

(1). 配置source端config配置

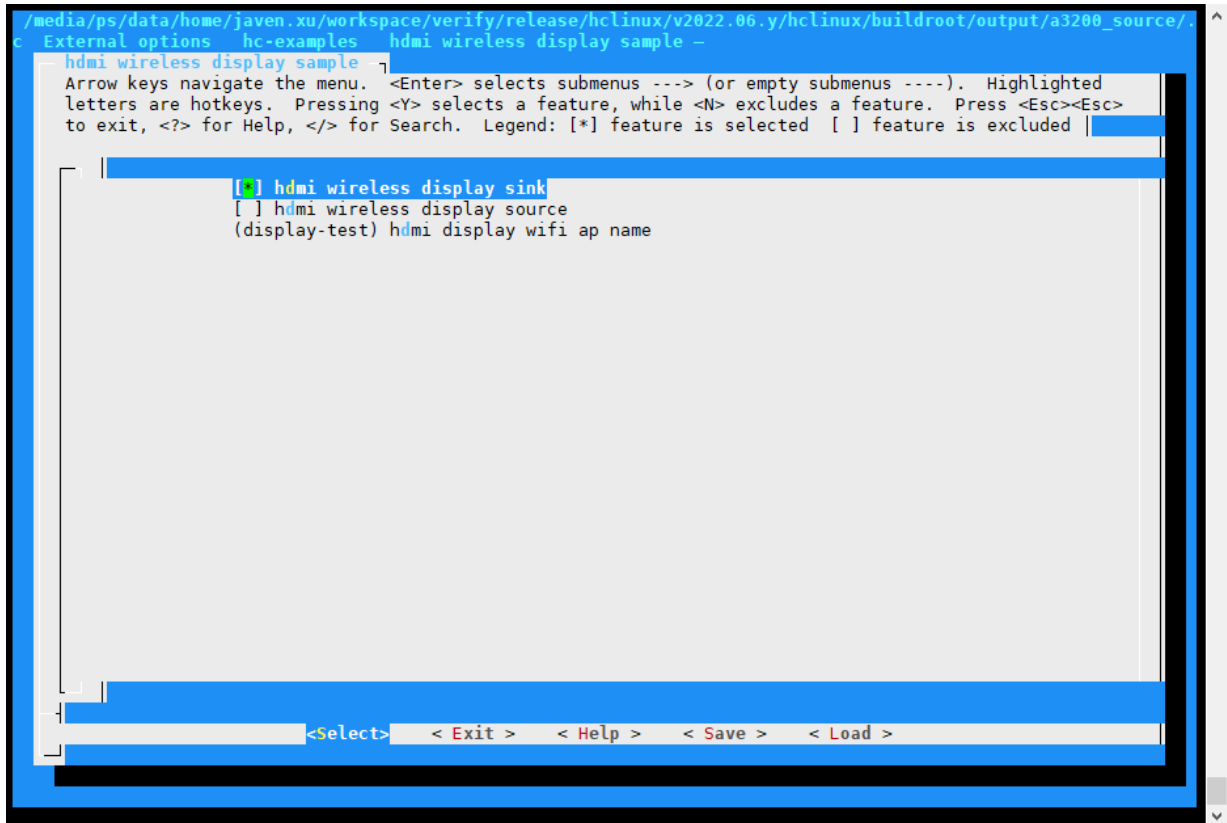
```
1 | #cd buildroot
2 | #make O=output/a3200_source BR2_EXTERNAL=$PWD/../
   | hichip_hc16xx_db_a3200_v10_hdmi_wifi_source_defconfig
3 | #cd output/a3200_source
4 | #make menuconfig
5 | -> External options
6 |     -> hc-examples
7 |         -> hdmi wireless display sample
8 |             ->hdmi wireless display source
9 | #cp .config ../../../configs/hichip_hc16xx_db_a3200_v10_hdmi_wifi_source_defconfig //保存配置
10 | #make all //编译固件
```



```

1 #cd buildroot
2 #make O=output/a3200_sink BR2_EXTERNAL=$PWD/../
  hichip_hc16xx_db_a3200_v10_hdmi_wifi_sink_defconfig
3 #cd output/a3200_sink
4 #make menuconfig
5 -> External options
6     -> hc-examples
7         -> hdmi wireless display sample
8             ->hdmi wireless display source
9 #cp .config ../../../../configs/hichip_hc16xx_db_a3200_v10_hdmi_wifi_sink_defconfig //保存配置
10 #make all //编译固件

```



16.17 hcfota固件升级

16.17.1 hcfota介绍

hcfota是hichip固件升级程序，其升级方式有：

- usb device，待支持，使用PC工具通过usb device进行升级
- U盘升级，已支持，通过U盘进行升级
- sd 卡升级，已支持，通过sd卡进行升级
- network，已支持，通过网络进行升级

支持的方案有：

- nor flash only，已支持
- spi-nand only，待支持
- nor+emmc，已支持
- nor+nand，待支持

进入升级模式的方法有：

- adc_key升级键：开机时，长按升级键，即可进入升级模式。升级键在dts中定义
- hcfota接口：app调用hcfota接口

由于红外遥控器协议原因，不支持遥控器按键升级

升级固件：

hcfota.bin：系统固件

system_data.bin：用户数据固件，针对nor+emmc方案

16.17.2 hcfota实现原理

linux系统阶段，app发起升级请求，hcfota会向persistentmem分区写入标志位，系统重启后，hcboot检测到persistentmem标志位，会去读取U盘中的hcfota.bin文件，然后进行升级。

16.17.3 hcfota支持的方案

公版板型，都未配置hcfota，需要用户自己配置

方案板型，c3100_emmc默认有配置

16.17.4 hcfota相关代码介绍

下面以c3100_emmc为例：

1.配置

```
1 board\hichip\hc16xx\c3100_emmc\dts\hc16xx-db-c3100-v20-avp.dtsi //板级配置
2 SOURCE\avp\configs\hichip_hc16xx_linux_b1_c3100_emmc_defconfig //方案配置
```

2.代码

```
1 SOURCE\hcfota //hcfota源码
2 SOURCE\avp\components\applications\apps-bootloader\source\cmd\upgrade.c //hcboot升级源码
```

16.17.5 hcfota配置

1.板级配置

修改hc16xx-db-c3100-v20-avp.dtsi

```
1 key-adc@0 {
2     status = "disabled";
3     key-num = <6>; //按键的数量
4     /*key_map = <voltage_min, voltage_max, code>*/
5     key-map = <200 500 103>,
6             <501 800 108>,
7             <901 1000 105>,
8             <1101 1200 106>,
9             <1301 1500 0x160>,
```

```

10         <1600 1750 174>;
11     };
12
13     hcfota-upgrade {
14         status = "disabled";    //使能
15         adc_key = <103>;        //升级键键值，需是input-event-codes.h定义的keycode
16     };

```

hclinux/buildroot/output/build/linux-4.4.186/include/uapi/linux/input-event-codes.h

2.hcboot方案配置

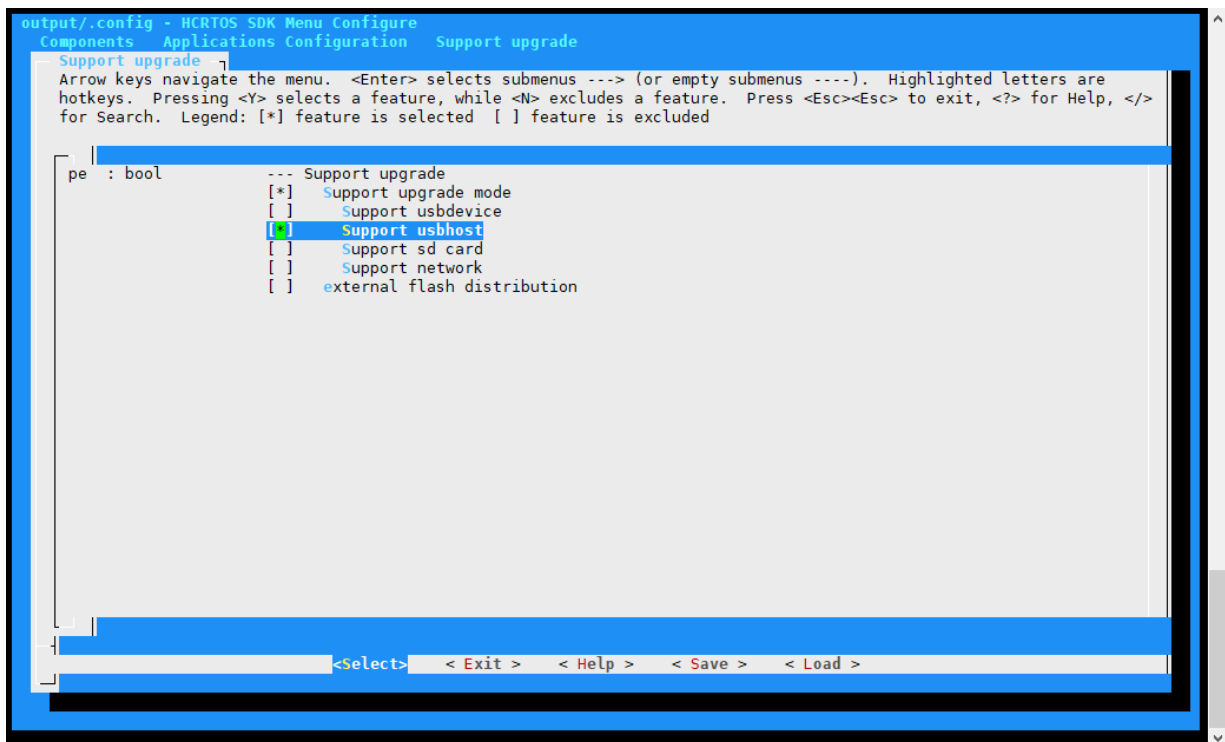
参考《16.1.2.4 修改hcboot配置》方法进入配置，**切记一定要保存配置**

(1). 配置升级方式

```

1  -> Components
2      -> Applications Configuration
3          -> Support upgrade (CONFIG_BOOT_UPGRADE [=y])
4              -> Support upgrade mode
5                  -> external flash distribution //only nor+emmc方案选配

```



(2). 配置按键

```

1  -> Components
2      -> kernel (BR2_PACKAGE_KERNEL [=y])
3          -> Drivers
4              -> input event (CONFIG_DRV_INPUT [=y]) x (NEW)
5                  -> key adc driver

```

```
output/.config - HCRTOS SDK Menu Configure
Components  kernel  Drivers  input event  saradc menu
saradc menu
--- saradc menu
[*] key adc driver
[ ] poll adc driver (NEW)
[ ] touch adc driver 16xx (NEW)

<Select>  < Exit >  < Help >  < Save >  < Load >
```

3.linux方案配置

- 1 -> External options
- 2 -> HC Fota (BR2_PACKAGE_HCFOTA [=y])
- 3 -> execute bin

```
/media/ps/data/home/javen.xu/workspace/verify/release/hclinux/v2022.09.y/hclinux/buildroot/output/d3100/.config - Buildroot -
g External options  HC Fota
HC Fota
--- HC Fota
[*] execute bin
[ ] gzip
[ ] lzma

<Select>  < Exit >  < Help >  < Save >  < Load >
```


16.17.6 hcfota调试

hcfota命令:

```
1 Usage: hcfota <file-path>
2   upgrade from local file
3
4 Usage: hcfota <url>
5   upgrade from network file
6
7 Usage: hcfota info <file-path>
8   Show information of local file
9
10 Usage: hcfota info <url>
11   Show information of network file
12
13 Usage: hcfota download <url> <file-path>
14   Download network file to local file
15
16 Usage: hcfota reboot <mode> <url>
17   Reboot with ota detect mode <mode>: [none | usbdevice | usbhost | sd | network]
18                                     <url> mus be set if <mode> is [usbhost | sd | network]
```

```
*
# hcfota
Usage: hcfota <file-path>
  upgrade from local file

Usage: hcfota <url>
  upgrade from network file

Usage: hcfota info <file-path>
  Show information of local file

Usage: hcfota info <url>
  Show information of network file

Usage: hcfota download <url> <file-path>
  Download network file to local file

Usage: hcfota reboot <mode> <url>
  Reboot with ota detect mode <mode>: [none | usbdevice | usbhost | sd | network]
                                     <url> mus be set if <mode> is [usbhost | sd | network]

# hcfota reboot usbhost
```

16.17.7 app如何调用hcfota接口

hfota头文件位于: SOURCE\hcfota\hcfota.h

可以参考SOURCE\hcfota\main.c

16.18 usb device固件升级

16.18.1 介绍

在启动阶段, 进入USB device升级模式, 电脑上会显示一个U盘, 向U盘中拷贝hcfota升级固件, 就可以自动升级了。

此方式操作方便, 既适用于开发阶段, 又适用于终端用户。

16.18.2 升级原理

- (1). hcboot检测到按键按下就进入U盘模式，电脑上就会弹出一个u盘
- (2). 将升级固件拷贝到USB盘中
- (3). hcboot检测到固件后，系统会自动升级。
- (4). 升级成功后会重启系统。
- (5). 用户发现电脑上U盘消失了，可以认为升级完成。

16.18.3 进入升级方法

方法1：按住升级键开机

16.18.4 配置

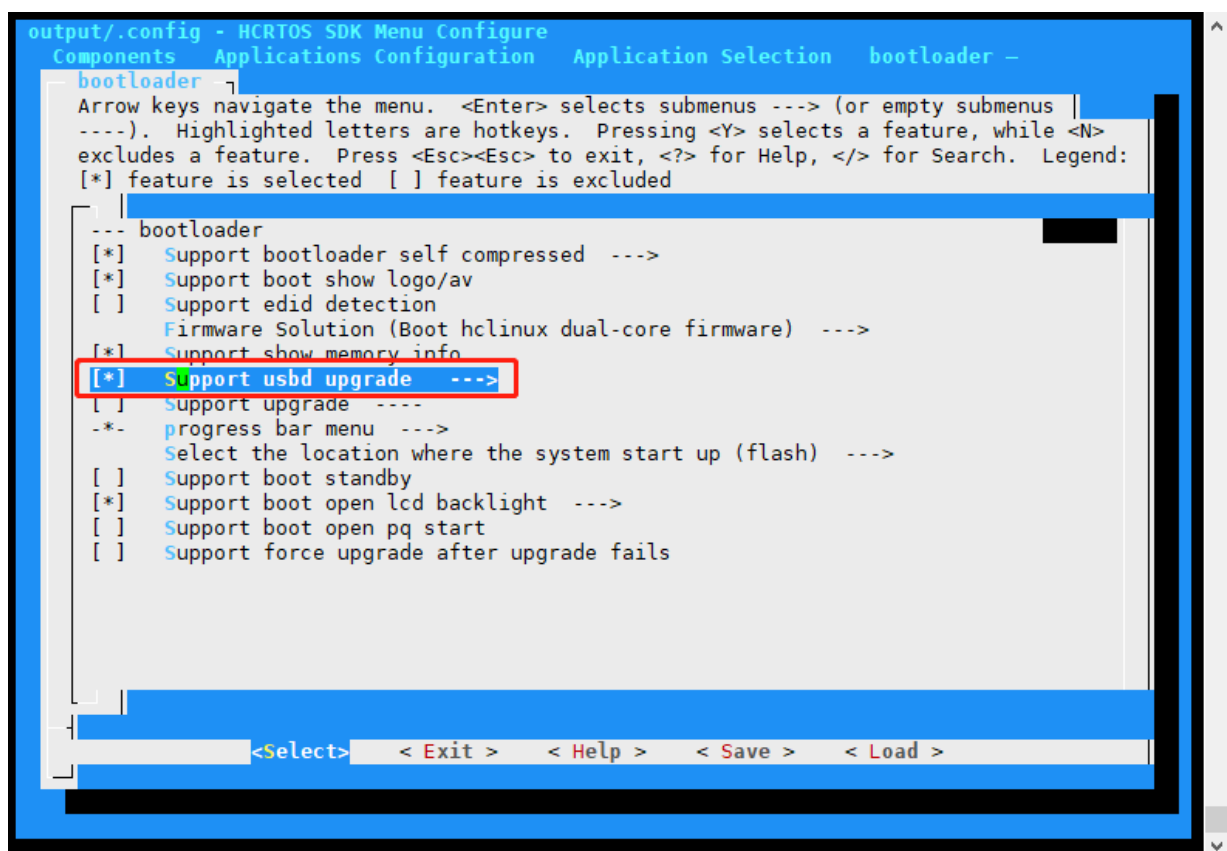
1.配置xxx-avp.dtsi

参考《16.17.5 hcfota配置》中的板级配置

2.配置hcboot

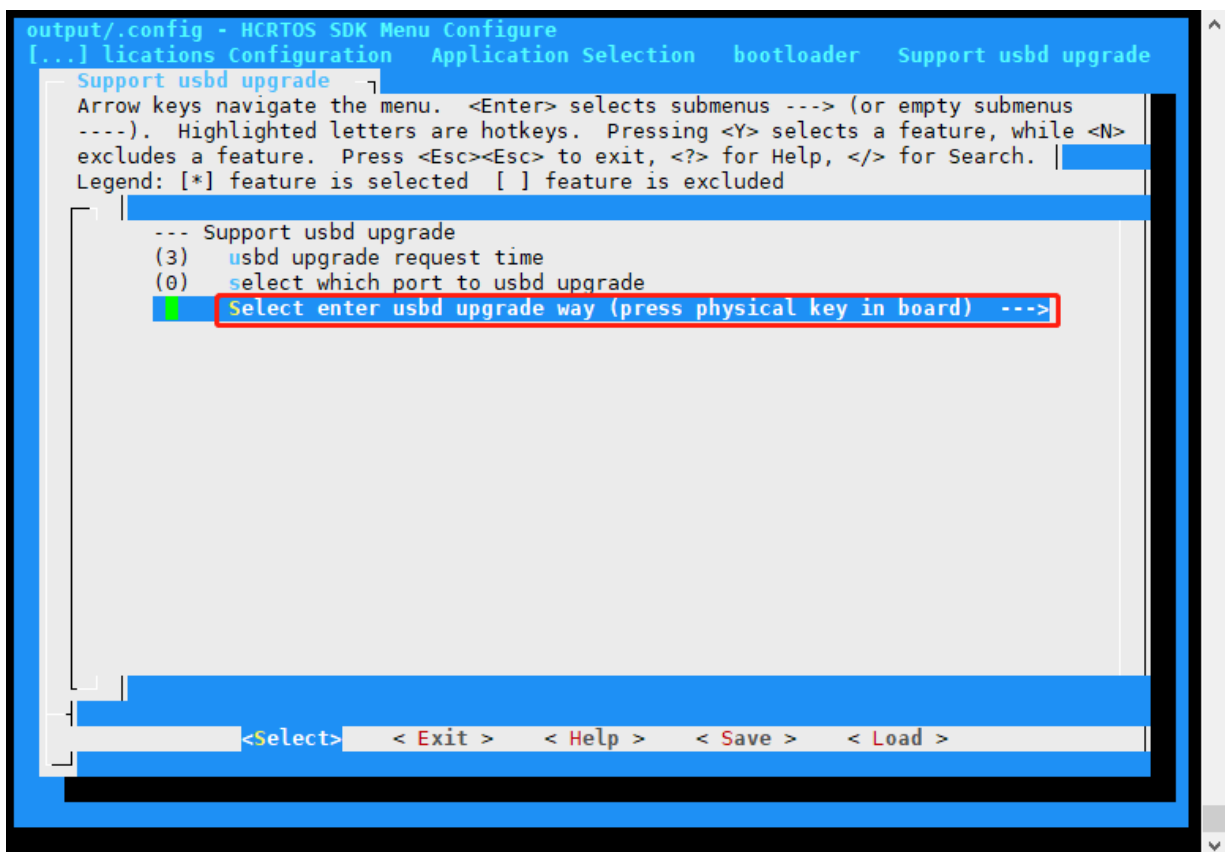
输入命令进行配置

```
1 | make hcboot-menuconfig
```



```
output/.config - HCRTOS SDK Menu Configure
Components Applications Configuration Application Selection bootloader -
bootloader
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus |
----). Highlighted letters are hotkeys. Pressing <Y> selects a feature, while <N>
excludes a feature. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend:
[*] feature is selected [ ] feature is excluded
--- bootloader
[*] Support bootloader self compressed --->
[*] Support boot show logo/av
[ ] Support edid detection
Firmware Solution (Boot hclinux dual-core firmware) --->
[*] Support show memory info
[*] Support usbd upgrade --->
[ ] Support upgrade ----
-- progress bar menu --->
Select the location where the system start up (flash) --->
[ ] Support boot standby
[*] Support boot open lcd backlight --->
[ ] Support boot open pq start
[ ] Support force upgrade after upgrade fails

<Select> < Exit > < Help > < Save > < Load >
```



3.保存配置和编译

参考《16.1.2.4 修改hcboot配置》

16.18.5 调试

1.按住升级键上电开机，会进入usb device升级模式

```
*****
Flash Geometry:
  blocksize: 4096
  erasesize: 4096
  neraseblocks: 8192
  chipsize: 0x2000000
sysdata:tvtype is 15
VC TV_LINE_800X480_60 F:30
VC TV_LINE_800X480_60

DDR Info:
  id:19
  type:DDR3
  size:256M
  clock:1066MHz

PRESS ANY KEY ON KEYBOARD TO ENTER USB D UPGRADE MODE ==> 3
PRESS ANY KEY ON KEYBOARD TO ENTER USB D UPGRADE MODE ==> 2
--> usb#0 reset
--> usb notify connect v1D6Bp0002
--> usb#0 reset
--> usb#0 mode : 0x1 (0x1:host, 0x2:peripheral, 0x3:otg)
--> usb#1 reset
--> usb notify connect v1D6Bp0002
--> usb#1 reset
--> usb#1 mode : 0x1 (0x1:host, 0x2:peripheral, 0x3:otg)
PRESS ANY KEY ON KEYBOARD TO ENTER EXECUTE FROM RAM MODE,ELSE WILL ENTER USB D UPGRADE MODE ==> 3
--> usb connect, devctl 5d
--> usb notify connect v1A40p0101
PRESS ANY KEY ON KEYBOARD TO ENTER EXECUTE FROM RAM MODE,ELSE WILL ENTER USB D UPGRADE MODE ==> 2
--> usb notify connect v0BDApF179
PRESS ANY KEY ON KEYBOARD TO ENTER EXECUTE FROM RAM MODE,ELSE WILL ENTER USB D UPGRADE MODE ==> 1

    ### USB D UPGRADE MODE ###

--> usb#0 reset
--> usb#0 mode : 0x2 (0x1:host, 0x2:peripheral, 0x3:otg)
==> (usb gadget) (lun:0) file:/dev/ram0
```

2.将HCFOTA_HC16C3100V20_2303281947.bin拷贝到电脑U盘中

![[hcboot_upgrade_usb_device_4](pic/hcboot_upgrade_usb_device_4.png)

	flashwr_unify.abs	2023/3/28 19:47	ABS 文件	74 KB
	hc16xx-db-c3100-v20.dtb	2023/3/28 19:47	DTB 文件	30 KB
	hcboot.bin	2023/3/28 19:46	{42042206-2D85...	358 KB
	hcboot.out	2023/3/28 19:46	OUT 文件	474 KB
	hcboot.out.map	2023/3/28 19:46	MAP 文件	9 KB
	hcboot.uncompress.bin	2023/3/28 19:46	{42042206-2D85...	943 KB
	hcboot.uncompress.out	2023/3/28 19:46	OUT 文件	6,078 KB
	hcboot.uncompress.out.map	2023/3/28 19:46	MAP 文件	2,187 KB
	HCfota_Generator.exe	2023/3/28 19:47	应用程序	182 KB
	HCFOTA_HC16C3100V20_2303281947.bin	2023/3/28 19:47	{42042206-2D85...	11,922 KB
	hcprog.ini	2023/3/28 19:47	配置设置	2 KB
	HCProgram_bridge.exe	2023/3/28 19:47	应用程序	2,532 KB
	persistentmem.bin	2023/3/28 19:47	{42042206-2D85...	1 KB
	romfs.img	2023/3/28 19:47	光盘映像文件	34 KB
	rootfs.squashfs	2023/3/28 19:47	SQUASHFS 文件	7,648 KB
	sfburn.bin	2023/3/28 19:47	{42042206-2D85...	16,384 KB
	sfburn.bin.290572dbab9786b900451a8276351595	2023/3/28 19:47	290572DBAB978...	16,384 KB
	sfburn.ini	2023/3/28 19:47	配置设置	1 KB
	system_data.bin	2023/3/28 19:47	{42042206-2D85...	52,224 KB
	updater.bin	2023/3/28 19:47	{42042206-2D85...	64 KB
	vmlinux	2023/3/28 19:47	文件	69 599 KB

升级工具

升级固件

升级脚本

U 盘 (I:)

17.7 MB 可用, 共 17.7 MB

)					

名称	修改日期	类型	大小
HCFOTA_HC16C3100V20_2303281947...	2023/3/28 19:47	{42042206-2D85...	11,922 KB

```

viddec_mjpg_set_frame_output
pkt_info.flag=2 pkt_info.size=0
mjpg_set_display_frame
get_pkt_eos = 1
reset
D/HEX JEDEC id bytes:: 0000-0010: FF FF FF FF FF FF .....
Flash Geometry:
  blocksize: 4096
  erasesize: 4096
  neraseblocks: 8192
  chipsize: 0x2000000
sysdata:tvtype is 15
VC_TV_LINE_800X480_60 F:30
VC_TV_LINE_800X480_60

DDR Info:
  id:19
  type:DDR3
  size:256M
  clock:1066MHz

PRESS ANY KEY ON KEYBOARD TO ENTER USB D UPGRADE MODE ==> 3
PRESS ANY KEY ON KEYBOARD TO ENTER USB D UPGRADE MODE ==> 2
--> usb#0 reset
--> usb notify connect v1D6Bp0002
--> usb#0 reset
--> usb#0 mode : 0x1 (0x1:host, 0x2:peripheral, 0x3:otg)
--> usb#1 reset
--> usb notify connect v1D6Bp0002
--> usb#1 reset
--> usb#1 mode : 0x1 (0x1:host, 0x2:peripheral, 0x3:otg)
PRESS ANY KEY ON KEYBOARD TO ENTER EXECUTE FROM RAM MODE,ELSE WILL ENTER USB D UPGRADE MODE ==> 3
--> usb connect, devctl 5d
--> usb notify connect v1A40p0101
PRESS ANY KEY ON KEYBOARD TO ENTER EXECUTE FROM RAM MODE,ELSE WILL ENTER USB D UPGRADE MODE ==> 2
--> usb notify connect v0BDApF179
PRESS ANY KEY ON KEYBOARD TO ENTER EXECUTE FROM RAM MODE,ELSE WILL ENTER USB D UPGRADE MODE ==> 1

    ### USB D UPGRADE MODE ###

--> usb#0 reset
--> usb#0 mode : 0x2 (0x1:host, 0x2:peripheral, 0x3:otg)
==> (usb gadget) (lun:0) file:/dev/ram0

=====HCFOTA_HC16C3100V20_2303281947.bin start update!=====
Upgrade from /mnt/ram0/HCFOTA_HC16C3100V20_2303281947.bin
Upgrading: [>

```

1 0%

16.19 旋转

旋转分为:

- UI旋转
- 视频旋转

在SDK中, UI和视频是不同的接口, 因此要分别实现UI旋转和视频旋转。

c3100_emmc默认实现了mipi屏的旋转, 可供参考:

(1).配置

 dts配置: hclinux\board\hichip\hc16xx\c3100_emmc\dts

 lvgl配置: hclinux\board\hichip\hc16xx\c3100_emmc\liblvgl.config

(2). mipi屏参数

 分辨率: 800x1280

1.旋转配置

(1). 配置旋转参数

hc16xx-db-c3100-v20-avp.dtsi有旋转配置, hcboot和app应该读取配置, 从而实现旋转和翻转。

```

1 rotate{
2     /*
3     * anticlockwise,
4     * support 0/90/180/270 degree.
5     */
6     rotate = <90>;    //旋转角度

```

```

7
8  /*
9  * value is 0 or 1
10 * h_flip = 1 is horizontally flipped
11 * v_flip = 1 is vertically flipped
12 */
13 h_flip = <0>; //左右翻转
14 v_flip = <0>; //上下翻转
15
16     status = "disabled";
17 };
18

```

如果支持`hdmirx`，`avp.dtsi`中的`fb0`配置，可以部分旋转。

(2). 配置scale

参考`hc16xx-db-c3100-v20.dts`

```

1  &fb0 {
2      bits_per_pixel = <32>;
3      xres = <800>;           //配置xres和yres
4      yres = <1280>;
5      xres_virtual = <800>;   //配置xres_virtual和yres_virtual
6      yres_virtual = <1280>;
7      xoffset = <0>;
8      yoffset = <0>;
9
10     scale = <800 1280 1080 1920>; //配置scale
11
12     reg = <CONFIG_FB0_REG 0x1000>;
13     /*
14     * frame buffer memory from:
15     * system : malloc buffer from system heap
16     * mmz0    : malloc buffer from mmz0
17     * default is system if the property is missing
18     */
19     buffer-source = "system";
20     extra-buffer-size = <0xc00000>; /* 8Mb */
21
22     status = "okay";
23
24 };

```

2.bootlogo旋转

代码位置：`hclinux\SOURCE\avp\components\applications\apps-bootloader\source\cmd\showlogo.c`

默认已经实现旋转和翻转。

3.app旋转

(1). UI旋转

lvgi初始化时，会去读取`avp.dtsi`中配置，实现旋转。

app不需要做任何处理。

(2). 视频旋转

特别注意：视频旋转和**UI**旋转是相反的

视频：顺时针旋转

UI：逆时针旋转

app需要主动去处理视频旋转，可以参考hclinux\SOURCE\hc-examples\ffmpeg_examples.c文件中play_uri(char *uri)函数

```
1 static int play_uri(char *uri)
2 {
3     HCPlayerInitArgs init_args = {0};
4     if (!g_mp) {
5         return -1;
6     }
7
8     init_args.uri = uri;
9     init_args.snd_devs = g_snd_devs;
10    init_args.sync_type = g_sync_mode;
11    init_args.user_data = g_mp;
12    init_args.start_time = g_time_ms;
13    init_args.buffering_enable = g_buffering_enable;
14    init_args.play_attached_file = 1;
15    init_args.msg_id = (int)g_msgid;
16    init_args.img_dis_mode = g_img_dis_mode;
17    init_args.mirror_type = g_mirror_type;
18    init_args.rotate_type = g_rotate_type % 4;    //旋转类型,enum ROTATE_TYPE
19    init_args.callback = NULL;
20    init_args.disable_audio = g_disable_audio;
21    init_args.disable_video = g_disable_video;
22    if (g_en_subtitle) {
23
24    #ifdef __linux__
25        init_args.callback = subtitle_callback;
26    #endif
27    }
28    if (g_rotate_type || g_mirror_type) {
29        init_args.rotate_enable = 1;    //旋转使能
30    }
31    init_args.audio_flush_thres = g_audio_flush_thres;
32    init_args.bypass = g_bypass;
33
34    .....
35 }
36
```

hclinux/SOURCE/linux-drivers/include/uapi/hcuapi/vidmp.h

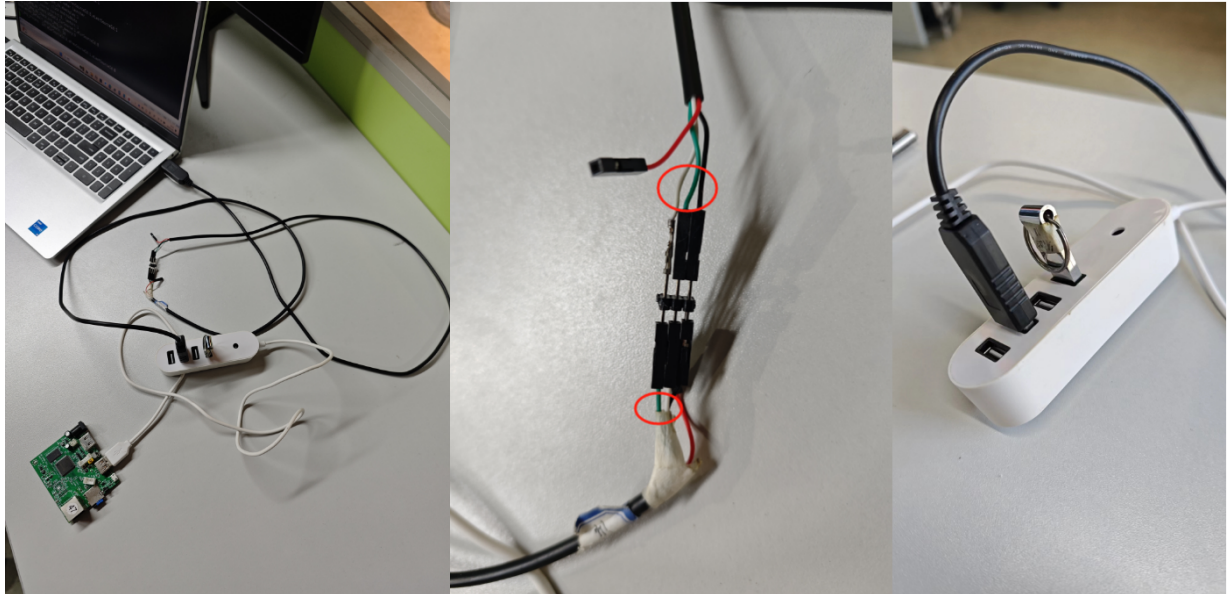
```
1 typedef enum ROTATE_TYPE {
2     82      /* Clockwise rotate 顺时针旋转*/
3     83      ROTATE_TYPE_0,
4     84      ROTATE_TYPE_90,
5     85      ROTATE_TYPE_180,
6     86      ROTATE_TYPE_270,
7     87 } rotate_type_e;
```

16.20 如何将usb串口配置为控制台串口

1. 准备工作

- (1). 两根usb串口打印线，A和B
- (2). A线的TX接B线的RX，A线的RX接B线的TX，A线的GND接B线的GND
- (3). A线接开发板，B线接电脑
- (4). 如果还想接U盘，需要一个usb hub

如图所示：



2. 修改dts，将ttyUSB0作为控制台串口

修改hc16xx-db-d3100-v10.dts中的bootargs参数，修改如下：

```
1 console=ttyS0,115200N8
2
3 改为：
4
5 console=tty0 console=ttyUSB0,115200N8
```

```
bootargs = "root=/dev/mtdblock5 rw rootfstype=squashfs init=/linuxrc console = ttyS0,115200N8
earlycon=uart8250,mmio,0x18818300,115200n8 no_console_suspend noirqdebug";
```

修改后：

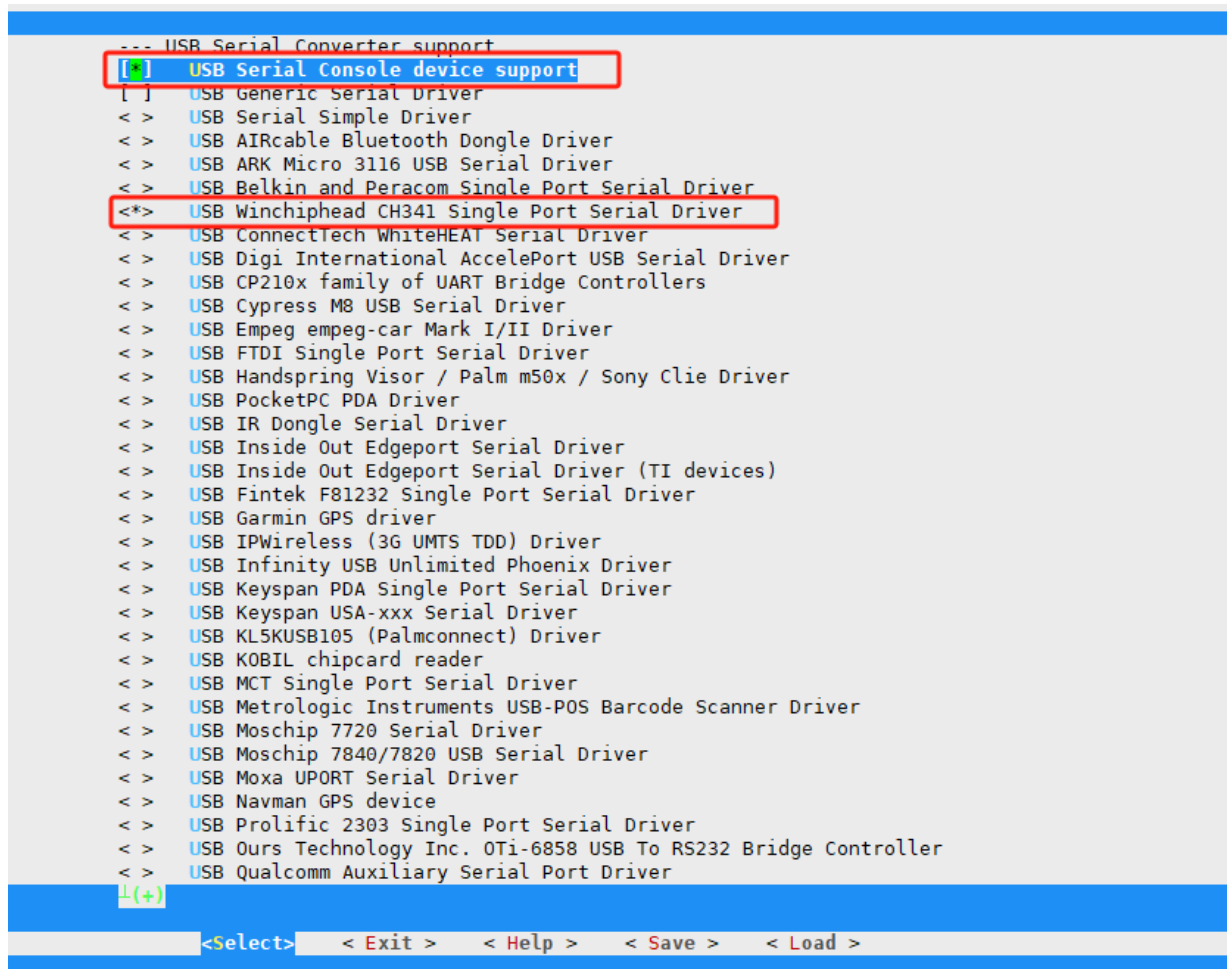
```
bootargs = "root=/dev/mtdblock5 rw rootfstype=squashfs init=/linuxrc console = tty0
console = ttyUSB0,115200N8 earlycon=uart8250,mmio,0x18818300,115200n8 no_console_suspend
noirqdebug";
```

3. 修改linux kernel defconfig配置，使能usb串口配置

```
Device Drivers
USB support
> USB Serial Converter support
[] USB Serial Console device support
```


[] USB Serial Console device support

<*> USB Winchiphead CH341 Single Port Serial Driver



注意：保存`kernel`配置到`board`目录

4. 注意事项

- (1). 不要修改`avp.dtsi`中的串口配置，boot阶段会从普通串口的打印。进到kernel以后会从usb出打印
- (2). 如果开机插着usb串口线，进到控制台后，串口可以输入也可以输出。如果开机后才插usb串口线，那么可以看到打印，但是不能输入。

FAQ. 常见问题解答

1. 中文版本ubuntu系统编译hclinux报错

问题现象：编译到`hcboot`时，系统会报错，如下：

```
Makefile:195: warning: ignoring old recipe for target 'synconfig'
[7m>>> apps-bootloader Building [27m
/data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/build/scripts/mkmakefile /data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/components/applications/apps-bootloader/source/ /data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/build/apps-bootloader/Makefile
GEN /data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/build/apps-bootloader/Makefile
rm -f /data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/build/apps-bootloader/.tmp-for-load-addr
mkdir -p /data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/build/apps-bootloader/.tmp-for-load-addr
cp -f /data/sky/work/hc/helinux-2022.03.y.1/board/hicchip/hc16xx/dts/hc16xx-db-d3100-v20.dts /data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/build/apps-bootloader/.tmp-for-load-addr/
cp -f /data/sky/work/hc/helinux-2022.03.y.1/board/hicchip/hc16xx/dts/hc16xx-common.dtsi /data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/build/apps-bootloader/.tmp-for-load-addr/
cp -f /data/sky/work/hc/helinux-2022.03.y.1/board/hicchip/hc16xx/dts/hc16xx-db-d3100-v20-avp.dtsi /data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/build/apps-bootloader/.tmp-for-load-addr/
echo "unsigned int bootloader_load_addr = (HCRTOS_BOOTMEM_OFFSET);" >> /data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/build/apps-bootloader/.tmp-for-load-addr/hc16xx-db-d3100-v20.dts
gcc -O2 -I/data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/components/kernel/source/include -E -Wp,-NMD /data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/build/apps-bootloader/.tmp-for-load-addr/hc16xx-db-d3100-v20.dts.tmp | grep bootloader_load_addr > /data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/build/apps-bootloader/.tmp-for-load-addr/hc16xx-db-d3100-v20.dtb.dts.tmp
echo -e "#include <stdio.h>\n\n int main(int argc, char **argv) { printf(\"0x%08x\\n\", bootloader_load_addr | 0x80000000); return 0; }" >> /data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/build/apps-bootloader/.tmp-for-load-addr/hc16xx-db-d3100-v20.dtb.dts.tmp
gcc -o /data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/build/apps-bootloader/.tmp-for-load-addr/a.out /data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/build/apps-bootloader/.tmp-for-load-addr/hc16xx-db-d3100-v20.dtb.dts.tmp
/data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/build/scripts/update_entry_addr.sh --genaddrout /data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/build/apps-bootloader/.tmp-for-load-addr/a.out /data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/host/bin/data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/host/usr/bin/data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/host/bin/data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/build/apps-bootloader/Makefile:77: warning: ignoring old recipe for target 'built-in.o'
PATH="/opt/mips32-mti-elf/2019.09-03/bin:/data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/host/bin:/data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/host/usr/bin:/data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/build/apps-bootloader/Makefile:77: warning: ignoring old recipe for target 'built-in.o'
make[3]: 警告: 子 make 中强制 -jN: 关闭 jobserver 模式。
"KBUILD_OUTPUT is /data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/build/apps-bootloader"
/data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/build/Makefile:build:54: warning: overriding recipe for target 'built-in.o'
/data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/components/applications/apps-bootloader/source/./Makefile:77: warning: ignoring old recipe for target 'built-in.o'
/opt/mips32-mti-elf/2019.09-03/bin/mips-mti-elf-ld: warning: section '.sbss' type changed to PROGBITS
/opt/mips32-mti-elf/2019.09-03/bin/mips-mti-elf-objcopy -O binary hcboot.out hcboot.bin
lzma -zxf -c hcboot.bin > hcboot.bin.z
/opt/mips32-mti-elf/2019.09-03/bin/mips-mti-elf-objcopy --remove-section=.reginfo --add-section=.image=hcboot.bin.z --set-section-flags=.image=contents,alloc,load,readonly,data boot-compressed/dummy.o boot-compressed/piggy.o
Compiling boot-compressed/head.o
/data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/components/applications/apps-bootloader/source/boot-compressed/head.S: Assembler messages:
/data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/components/applications/apps-bootloader/source/boot-compressed/head.S:59: 错误: invalid operands 'li $26,'
/data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/build/Makefile:rules:60: recipe for target 'boot-compressed/head.o' failed
make[5]: *** [boot-compressed/head.o] Error 1
/data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/components/applications/apps-bootloader/source/Makefile:83: recipe for target 'hcboot.z.out' failed
make[4]: *** [hcboot.z.out] Error 2
/data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/build/Makefile:entry:35: recipe for target 'sub-make' failed
make[3]: *** [sub-make] Error 2
build/pkg-generic.mk:250: recipe for target '/data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/build/apps-bootloader/.stamp_build' failed
make[2]: *** [data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/output/build/apps-bootloader/.stamp_build] Error 2
package/pkg-generic.mk:247: recipe for target '/data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/.stamp_build' failed
make[1]: *** [data/sky/work/hc/helinux-2022.03.y.1/buildroot/output/build/hcboot-custom/.stamp_build] Error 2
Makefile:84: recipe for target '_all' failed
make: *** [_all] Error 2
```

问题原因：

ubuntu中文版本造成的，在编译过程中readef、nm等工具返回值是中文，造成编译失败

解决方法：

将ubuntu语音从中文切换为英文，方法如下：

- 1.编辑文件 /etc/default/locale
- 2.将语言修改为英文

```
1 | LANG="zh_CN.UTF-8"
2 | LANGUAGE="zh_CN:zh"
3 |
4 | 修改为：
5 |
6 | LANG="en_US.UTF-8"
7 | LANGUAGE="en_US:en"
```

3.重启Ubuntu，即可恢复为英文的语言环境。

2 buildroot在编译过程中下载第三方软件包非常慢

问题现象： buildroot在编译过程中下载第三方软件包非常慢

问题原因： 第三方软件包通常在国外服务器，所以下载有时非常慢

解决方法：

- 方法1. 更换更快网络的电脑下载
- 方法2. 请参阅SDK发布说明，里面有常用第三方软件包的下载说明

3 烧写完固件后，播放影视文件时喇叭或者耳机没有声音

问题现象： 烧写完固件后，播放U盘中音视频文件，喇叭或者耳机没有声音

问题原因： jtag没有拔掉， jtag pin与audio pin存在复用

解决方法： 拔掉jtag线，重启机器

4 使用mplayer播放视频，可以听见声音，看不到播放画面，电视只显示主应用画面

问题现象： 使用mplayer播放视频，可以听见声音，看不到播放画面，电视只显示主应用画面

问题原因： 主应用画面把视频播放画面挡住了

解决方法：

方法1：将主应用UI透明化，执行命令：cat /dev/zero > /dev/fb0

方法2：开机步启动hccast_demo

(1). 修改package\lvgl\lvgl.mk文件，注释掉将hccast_demo添加到开机启动中。

```
LVGL_VERSION =
LVGL_SITE_METHOD = UNDEFINED
LVGL_SITE = UNDEFINED
LVGL_LICENSE = LGPL-2.0+
LVGL_LICENSE_FILES = COPYING
LVGL_CONF_OPTS =
LVGL_DEPENDENCIES += busybox wpa_supplicant

define LVGL_BUILD_CMDS
    $(TARGET_MAKE_ENV) $(MAKE) CC="$(TARGET_CC)" CFLAGS="$(TARGET_CFLAGS) $(TARGET_LDFLAGS)" -C $(@D)
endef

define LVGL_INSTALL_TARGET_CMDS
    $(INSTALL) -D -m 0755 $(@D)/demo $(TARGET_DIR)/usr/bin/demo
    $(INSTALL) -D -m 0755 $(@D)/hccast_demo $(TARGET_DIR)/usr/bin/hccast_demo
    #$(SED) "/\<hccast_demo\>/d" $(TARGET_DIR)/etc/inittab
    #echo 'null::respawn:/usr/bin/hccast_demo > /dev/console 2>&1' >> $(TARGET_DIR)/etc/inittab

    $(INSTALL) -m 755 -d $(TARGET_DIR)/hccast_files
    $(INSTALL) -m 755 $(@D)/hccast_app/HICHP_DLNA.m2v $(TARGET_DIR)/hccast_files

    $(TARGET_MAKE_ENV) $(MAKE) -C $(@D) install INSTALL="$(INSTALL)"
endef

$(eval $(generic-package))
```

(2). 在开机脚本中，删除之前添加的启动命令，修改buildroot\output\target\etc\inittab文件

```
4 #
5 # Note: BusyBox init doesn't support runlevels. The runlevels field is
6 # completely ignored by BusyBox init. If you want runlevels, use
7 # sysvinit.
8 #
9 # Format for each entry: <id>:<runlevels>:<action>:<process>
10 #
11 # id == tty to run on, or empty for /dev/console
12 # runlevels == ignored
13 # action == one of sysinit, respawn, askfirst, wait, and once
14 # process == program to run
15 #
16 # Startup the system
17 ::sysinit:/bin/mount -t proc proc /proc
18 ::sysinit:/bin/mount -o remount,rw /
19 ::sysinit:/bin/mkdir -p /dev/pts /dev/shm
20 ::sysinit:/bin/mount -a
21 ::sysinit:/etc/preinit
22 ::sysinit:/sbin/swapon -a
23 null::sysinit:/bin/ln -sf /proc/self/fd /dev/fd
24 null::sysinit:/bin/ln -sf /proc/self/fd/0 /dev/stdin
25 null::sysinit:/bin/ln -sf /proc/self/fd/1 /dev/stdout
26 null::sysinit:/bin/ln -sf /proc/self/fd/2 /dev/stderr
27 ::sysinit:/bin/hostname -f /etc/hostname
28 # now run any rc scripts
29 ::sysinit:/etc/init.d/rcS
30
31 # Put a getty on the serial port
32 #console::respawn:/sbin/getty -L console 0 vt100 # GENERIC_SERIAL
33 ::respawn:/bin/sh
34
35 # Stuff to do for the 3-finger salute
36 #::ctrlaltdel:/sbin/reboot
37
38 # Stuff to do before rebooting
39 ::shutdown:/etc/init.d/rcK
40 ::shutdown:/sbin/swapoff -a
41 ::shutdown:/bin/umount -a -x
42 null::respawn:/usr/bin/hccast_demo > /dev/console 2>&1
43
```

删除此行

(3). 执行make all，重新编译固件即可。

